

臺灣自編大學教科書

大模型導論

Introduction to Large Language Models

第 4 篇

預訓練、解碼與實驗科學

Pretraining, Decoding, and Experimental Science

第 14 章 解碼、取樣、受限生成與 KV Cache

第 15 章 預訓練迴圈、最佳化與訓練穩定性

第 16 章 訓練診斷、實驗科學與可重現報告

第 17 章 尺度定律、資料模型配比與算力預算

李世淵（機器人叫獸） 編著

助教 李天宇、李宇晴

2026 年 8 月 第一版

序言 從「跑得動」到「說得清楚」

第 3 篇結束時，你手上有一個會生成繁體中文的模型。那是一個里程碑——但如果你把它拿給別人看，很可能會遇到四個問題。

第一，它生成的文字一直重複。第二，它跑得很慢，生成一百個字要等好久。第三，你說「我改了某個設定之後變好了」，但別人問「你怎麼知道不是運氣」，你答不出來。第四，有人問「那訓練一個真正能用的模型要多少錢」，你只能說「應該不便宜」。

這一冊就是回答這四個問題。

四章，四個問題

第 14 章回答「輸出怎麼調」。一個訓練好的模型只提供機率分布，至於怎麼從中選詞元，是一個完全獨立的設計空間。溫度、top-p、重複懲罰這些旋鈕不花一分錢的訓練成本，卻能決定產品好不好用。這一章也會加上 KV cache——生成 2,048 個詞元時，它讓計算量降到千分之一。

第 15 章回答「訓練怎麼穩」。第 13 章的訓練迴圈是最小可行版本，能跑但沒有排程、沒有監控、中斷後只能大致續上。這一章把它升級為工業級：跑得久、跑得穩、出事時知道發生了什麼、而且能自動回滾。

第 16 章回答「結論可信嗎」。這是全書唯一一章不教任何新技術的——它教的是下結論的紀律。我在寫這章時做了一個模擬：在一個完全沒有效果的改動上只跑一個種子，有 50.6% 的機率會看到「變好了」。那個數字是這一章存在的理由。

第 17 章回答「資源夠不夠」。它把前面十六章的量化工具接起來，產出一份能拿去申請計畫、說服主管、或決定專題規模的預算書。如果你只能從本書帶走一份文件去說服別人，通常就是這一份。

這一冊的性格

前三篇教的是「怎麼做出來」，這一冊教的是「怎麼做得對」。這個轉變會讓部分讀者覺得節奏變慢——第 16 章沒有任何新的模型技術，第 17 章全是算術。但我認為這一冊是全書最能區分「會用工具的人」與「能負責的工程師」的部分。

一個具體的例子。第 16 章要求每個實驗至少跑三個種子、報告信賴區間、誠實列出跑過的所有設定。這些要求會讓你的實驗變慢、報告變長、結論變得沒那麼漂亮。但它們換來的是：**你的結論別人重做一次會得到同樣的答案**。而那正是一份技術報告與一則廣告的差別。

另一個例子。第 17 章會讓你算出「以我的算力，最適合訓練的是 87 M 參數的模型」——而多數人

的直覺會高出好幾倍。這個計算只要幾分鐘，卻可能省下你三個月的無效訓練。

幾個反直覺的結論

本冊有幾個結論值得先預告，因為它們與多數人的直覺相反。

****beam search 在開放式生成上比 greedy 更糟****（第 14.1.3 節）。把搜尋做得更好，結果反而更平淡——這說明「模型認為最可能的文字」與「人類覺得好的文字」之間有系統性落差。

****注意力不是計算的大宗****（第 11.7 節的延續）。在一般序列長度下，前饋網路才是；但記憶體的故事完全相反。

****梯度範數比 loss 更早反映異常****（第 15.4.1 節），提前量可達數百步。監控過程比監控結果有效。

****服務量大時應該刻意過度訓練較小的模型****（第 17.3 節）。7 B 配 140 B 詞元與 3 B 配 600 B 詞元的損益兩平點約在 6,150 億服務詞元——而且小模型還能部署在更便宜的硬體上。

****在臺灣，模型大小往往是被資料決定的，而不是被算力決定的****（第 17.4.3 節）。這與國際大廠的處境相反，也決定了我們的策略應該偏向「小而充分訓練」。

關於用語

本書一律使用臺灣的專業用語與教育部標準用字。以下是本冊特別容易混淆的幾組：

本書用語	對應英文	說明與區別
推論	inference	指模型執行、產生輸出的階段，如推論成本。
推理	reasoning	指模型的思考與邏輯推導能力，如推理模型。
最佳化	optimization	不使用「優化」。最佳化器即 optimizer。
正規化	normalization	層正規化與 Unicode 正規化。不使用「歸一化」。
標準化	standardization	統計上的減中位數除以離散度，如 IQR 標準化。
解碼	decoding	從機率分布產生詞元的整個過程。
取樣	sampling	解碼中「抽籤」的那一步，如核取樣。
截斷	truncation	取樣前先砍掉長尾，如 top-k、top-p。
退化	degeneration	生成品質崩壞（重複、無意義），非指模型退化。
快取	cache	KV cache 依慣例保留英文。不使用「緩存」。

本書用語	對應英文	說明與區別
預填 / 逐詞解碼	prefill / decode	生成的兩個階段，見第 14.6 節。
梯度範數	gradient norm	不使用「梯度模長」。
梯度裁切	gradient clipping	不使用「梯度剪裁」。
排程	schedule	學習率排程。不使用「調度」。
餘裕	headroom / buffer	預算中保留的緩衝。不使用「餘量」。
擬合 / 外推	fitting / extrapolation	不使用「配適 / 外插」。
信賴區間	confidence interval	不使用「置信區間」。
變異數 / 標準誤	variance / standard error	不使用「方差 / 標準差誤」。
管線	pipeline	資料管線與執行管線。不使用「流水線」。
尺度定律	scaling law	不使用「縮放定律」。

英文專有名詞在首次出現時並列原文，其後視情況直接使用英文（例如 KV cache、top-p、MFU、FLOPs）——這反映臺灣工程現場的真實習慣。

一個給教師的說明

本冊涵蓋第 14 到 17 週，是第一學期的後半。第 17 章結束時，學生應該完成第一學期的四項核心產出：可治理的語料、自訓 tokenizer、能生成繁中的模型，以及一份算力預算書。

第 16 章的教學特別建議放慢。它沒有新技術，但它的觀念會改變學生做研究的方式——而那個改變不會在一堂課裡發生。我的建議是：從第 16 章開始，所有的作業都要求附上種子數與信賴區間，讓紀律變成習慣而非知識。

李世淵（機器人叫獸）

2026 年 8 月 於雲林

目 錄

序言 從「跑得動」到「說得清楚」

目錄

全書架構總覽

第 14 章 解碼、取樣、受限生成與 KV Cache

Decoding, Sampling, Constrained Generation, and KV Cache

學習目標

- 14.1 解碼是一個獨立的設計空間
- 14.2 溫度：分布形狀的旋鈕
- 14.3 截斷取樣：top-k、top-p 與 min-p
- 14.4 退化輸出與重複懲罰
- 14.5 KV Cache：把平方變成線性
- 14.6 Prefill 與 Decode：兩種完全不同的工作
- 14.7 推測解碼：用小模型加速大模型
- 14.8 受限生成：強制輸出合法的結構
- 14.9 為臺灣任務設計解碼參數表

程式實作

系統案例：校務問答的解碼參數設計

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 15 章 預訓練迴圈、最佳化與訓練穩定性

Pretraining Loops, Optimization, and Training Stability

學習目標

- 15.1 批次大小與梯度累積
- 15.2 學習率排程
- 15.3 混合精度訓練
- 15.4 梯度範數：訓練健康度的主要指標
- 15.5 可中斷、可續訓的訓練
- 15.6 吞吐量與 MFU
- 15.7 完整的訓練腳本

程式實作

系統案例：24 小時不中斷訓練

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 16 章 訓練診斷、實驗科學與可重現報告

Training Diagnostics, Experimental Science, and Reproducible Reports

學習目標

16.1 把改動變成假設

16.2 隨機性：為什麼單一種子不算數

16.3 多重比較：跑很多組挑最好的

16.4 學習曲線診斷

16.5 記憶化偵測

16.6 三種自欺與它們的解藥

16.7 可重現的實驗紀錄

16.8 報告的寫法

程式實作

系統案例：詞彙表大小對繁中生成品質的完整實驗

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 17 章 尺度定律、資料模型配比與算力預算

Scaling Laws, Data-Model Balance, and Compute Budgets

學習目標

17.1 尺度定律的基本形式

17.2 最適配比

17.3 推論主導時：刻意過度訓練

17.4 資料受限：重複訓練的邊際效益

17.5 從小規模外推

17.6 從 FLOPs 到新臺幣

17.7 尺度定律的限制

程式實作

系統案例：臺灣本土 3B 模型的算力預算書

章末評量

研究延伸與版本注意事項

常見問題
教師使用建議
附：本章速查表
本章小結
成果檢核

索引

以下為 Word 自動目錄欄位。在 Word 中開啟後，於下方按右鍵選擇「更新功能變數」即可產生含頁碼的三層目錄；亦可使用左側導覽窗格依書籤瀏覽。

序言 從「跑得動」到「說得清楚」	2
四章，四個問題	2
這一冊的性格	2
幾個反直覺的結論	3
關於用語	3
一個給教師的說明	4
目 錄	5
全書架構總覽	14
本冊四章的能力遞進	14
第 4 篇	16
解碼、取樣、受限生成與 KV Cache	17
學習目標	17
14.1 解碼是一個獨立的設計空間	17
14.1.1 訓練與生成的分工	17
14.1.2 為什麼 greedy 不夠	18
14.1.3 beam search 為什麼在這裡失效	18
14.2 溫度：分布形狀的旋鈕	19
14.2.1 定義	19
14.2.2 用數值看清楚它做了什麼	19
14.2.3 實作	20
14.3 截斷取樣：top-k、top-p 與 min-p	21
14.3.1 為什麼光有溫度不夠	21
14.3.2 三種截斷方式	21
14.3.3 為什麼 top-p 通常優於 top-k	22
14.3.4 min-p 的動機	22
14.3.5 完整實作	23
14.4 退化輸出與重複懲罰	24
14.4.1 退化的三種形式	24

14.4.2 重複懲罰的兩種形式.....	24
14.5 KV Cache：把平方變成線性.....	26
14.5.1 沒有快取時在浪費什麼.....	26
14.5.2 快取什麼、不快取什麼.....	26
14.5.3 實作.....	27
14.5.4 記憶體代價.....	28
14.5.5 快取的三種失效情況.....	29
14.6 Prefill 與 Decode：兩種完全不同的工作.....	29
14.6.1 兩個階段.....	29
14.6.2 為什麼 decode 是頻寬受限的.....	30
14.7 推測解碼：用小模型加速大模型.....	30
14.7.1 核心想法.....	30
14.7.2 效益取決於接受率.....	31
14.7.3 實務考量.....	31
14.8 受限生成：強制輸出合法的結構.....	32
14.8.1 為什麼需要.....	32
14.8.2 三種層次.....	32
14.8.3 一個最小的 JSON 約束.....	32
14.9 為臺灣任務設計解碼參數表.....	33
14.9.1 任務分類.....	33
14.9.2 把設定寫成可版本控制的檔案.....	34
程式實作.....	36
程式 14A 完整解碼工具箱.....	36
程式 14B KV Cache 實作與效能量測.....	37
程式 14C 結構化輸出的合規率量測.....	38
系統案例：校務問答的解碼參數設計.....	39
問題分類與路由.....	39
交付要求.....	40
評分重點.....	40
章末評量.....	41
一、概念題.....	41
二、計算題.....	41
三、除錯題.....	41
四、系統設計題.....	42
五、臺灣情境與倫理題.....	42
評量提示（給自學者）.....	42
研究延伸與版本注意事項.....	42
常見問題.....	43
教師使用建議.....	43

本章小結	44
成果檢核	45
預訓練迴圈、最佳化與訓練穩定性	47
學習目標	47
15.1 批次大小與梯度累積	47
15.1.1 三種批次	47
15.1.2 累積時的損失平均：最常見的錯誤	48
15.1.3 批次大小該設多少	49
15.2 學習率排程	50
15.2.1 為什麼不能用固定學習率	50
15.2.2 三種排程	50
15.2.3 WSD 的實務優勢	52
15.2.4 學習率探測	52
15.3 混合精度訓練	53
15.3.1 為什麼要混合	53
15.3.2 loss scaling：只有 FP16 需要	54
15.3.3 哪些運算不該用低精度	55
15.4 梯度範數：訓練健康度的主要指標	55
15.4.1 為什麼看它而不是只看 loss	55
15.4.2 梯度裁切	55
15.4.3 尖峰偵測：一個需要修正的做法	56
15.4.4 發散的三種徵兆	57
15.5 可中斷、可續訓的訓練	58
15.5.1 續訓要還原什麼	58
15.5.2 完整實作	58
15.5.3 資料位置的還原	60
15.6 吞吐量與 MFU	61
15.6.1 三個效能指標	61
15.6.2 MFU 的合理範圍	61
15.6.3 常見的吞吐量瓶頸	61
15.6.4 activation 記憶體與 checkpointing	62
15.7 完整的訓練腳本	63
15.7.1 把前六節接起來	63
15.7.2 回滾策略	64
程式實作	66
程式 15A 完整預訓練腳本	66
程式 15B 學習率探測器	67
程式 15C 訓練哨兵與自動回滾	67
系統案例：24 小時不中斷訓練	68

事前準備清單.....	68
全程要記錄的內容.....	69
交付要求	69
評分重點	70
章末評量	71
一、概念題.....	71
二、計算題.....	71
三、除錯題.....	71
四、系統設計題.....	72
五、臺灣情境與倫理題.....	72
評量提示（給自學者）.....	72
研究延伸與版本注意事項	73
常見問題	73
教師使用建議	73
本章小結	74
成果檢核	75
訓練診斷、實驗科學與可重現報告.....	76
學習目標	76
16.1 把改動變成假設.....	76
16.1.1 假設要能被推翻.....	76
16.1.2 三種變數.....	77
16.1.3 一份實驗設計表.....	77
16.2 隨機性：為什麼單一種子不算數	78
16.2.1 一個令人不安的模擬.....	78
16.2.2 需要幾個種子.....	78
16.2.3 該報告什麼	79
16.2.4 bootstrap 信賴區間	79
16.2.5 哪些隨機源會影響結果.....	80
16.3 多重比較：跑很多組挑最好的.....	81
16.3.1 問題的規模	81
16.3.2 三種處置.....	82
16.3.3 超參數掃描的正確做法.....	82
16.4 學習曲線診斷	83
16.4.1 六種曲線形狀.....	83
16.4.2 「驗證損失低於訓練損失」的四個原因.....	83
16.4.3 用分域曲線看得更清楚.....	84
16.4.4 從曲線看資料問題	84
16.5 記憶化偵測.....	85
16.5.1 為什麼要量它	85

16.5.2	用「逐字複述長度」量化	86
16.5.3	影響記憶化的因素	87
16.5.4	可接受的範圍	88
16.6	三種自欺與它們的解藥	88
16.6.1	評測污染	88
16.6.2	選擇偏誤	88
16.6.3	cherry-picking 生成結果	89
16.6.4	一個容易忽略的偏誤：倖存者偏誤	90
16.7	可重現的實驗紀錄	90
16.7.1	從紀錄到套件	90
16.7.2	實驗套件的目錄結構	91
16.7.3	分析腳本應該做什麼	91
16.8	報告的寫法	93
16.8.1	六個段落	93
16.8.2	負面結果的價值	93
16.8.3	失敗復盤	93
16.8.4	一份實驗可信度檢查表	94
程式實作		96
程式 16A	實驗矩陣執行器	96
程式 16B	統計分析與圖表產生器	96
程式 16C	記憶化偵測	97
系統案例：詞彙表大小對繁中生成品質的完整實驗		98
為什麼選這個題目		98
實驗設計的四個難點		98
交付要求		99
評分重點		99
章末評量		100
一、概念題		100
二、計算題		100
三、除錯題		100
四、系統設計題		101
五、臺灣情境與倫理題		101
評量提示（給自學者）		101
研究延伸與版本注意事項		101
常見問題		102
教師使用建議		102
本章小結		103
成果檢核		104
尺度定律、資料模型配比與算力預算		105

學習目標.....	105
17.1 尺度定律的基本形式.....	105
17.1.1 三個變數與一條曲線.....	105
17.1.2 不可約損失：一個重要的概念.....	106
17.1.3 三條曲線與一條主線.....	106
17.1.4 為什麼冪次關係會成立.....	107
17.2 最適配比.....	107
17.2.1 直覺.....	107
17.2.2 實際算給你看.....	108
17.2.3 偏離最適配比的代價.....	108
17.2.4 這個比例不是鐵律.....	109
17.2.5 批次大小與學習率也有尺度關係.....	109
17.3 推論主導時：刻意過度訓練.....	110
17.3.1 訓練是一次性的，推論是每天的.....	110
17.3.2 一個完整的比較.....	110
17.3.3 什麼時候該過度訓練.....	111
17.3.4 推論成本的完整計算.....	111
17.4 資料受限：重複訓練的邊際效益.....	112
17.4.1 臺灣面對的實際限制.....	112
17.4.2 重複的邊際效益.....	112
17.4.3 繁中的尺度化策略.....	113
17.4.4 合成資料能否補足.....	114
17.5 從小規模外推.....	114
17.5.1 為什麼要外推.....	114
17.5.2 擬合的步驟.....	114
17.5.3 一個實測範例.....	116
17.5.4 外推的風險.....	116
17.5.5 IsoFLOP 曲線：另一種擬合方式.....	117
17.6 從 FLOPs 到新臺幣.....	118
17.6.1 完整的換算鏈.....	118
17.6.2 三種取得算力的方式.....	120
17.6.3 預算書必須包含的十項.....	120
17.6.4 一份預算書的實際範例.....	121
17.6.5 敏感度分析：哪些假設最關鍵.....	121
17.7 尺度定律的限制.....	122
17.7.1 它預測什麼、不預測什麼.....	122
17.7.2 本書的立場.....	122
17.7.3 尺度定律之外的三個變數.....	123
17.7.4 什麼時候不該用尺度定律.....	123

程式實作	125
程式 17A 尺度定律擬合器	125
程式 17B 算力預算計算器	125
程式 17C 資料受限的模擬	126
系統案例：臺灣本土 3B 模型的算力預算書	127
必須回答的五個核心問題	127
交付要求	128
評分重點	128
章末評量	129
一、概念題	129
二、計算題	129
三、除錯題	129
四、系統設計題	130
五、臺灣情境與倫理題	130
評量提示（給自學者）	130
研究延伸與版本注意事項	130
常見問題	131
教師使用建議	131
附：本章速查表	132
本章小結	133
成果檢核	134
索 引	135
英文詞條	135
中文詞條	135

全書架構總覽

全書分為十篇四十章，另有十四組附錄。本冊為第 4 篇，是第一學期的後半主線，也是從「做得出來」轉向「做得對」的轉折點。

篇次	章次	主題	核心產出
第 1 篇	1–4	大模型視野、數學與工程基礎	需求規格書、技術演進圖、數學速查表、可重現環境
第 2 篇	5–9	語言建模、臺灣語料與 Token 工程	基準評量規格、文本工程規格、自訓 tokenizer、語料清冊
第 3 篇	10–13	Transformer 從零實作	MiniFormosaGPT 與可生成繁體中文的模型
第 4 篇 (本冊)	14–17	預訓練、解碼與實驗科學	解碼工具箱、工業級訓練迴圈、實驗紀律、算力預算書
第 5 篇	18–21	算力、系統與現代架構	效能剖析、分散式決策、臺灣算力盤點
第 6 篇	22–26	本土化後訓練與對齊	繁中適配模型、LoRA、偏好對齊、邊緣小模型
第 7 篇	27–30	推理、知識更新與提示工程	推理實驗、知識更新流程、RAG 原型
第 8 篇	31–34	應用系統：進階 RAG、Agent 與產品開發	知識系統、受限 Agent、可上線的應用
第 9 篇	35–37	部署、評測、安全與治理	量化部署、臺灣題庫、紅隊與影響評估
第 10 篇	38–40	多模態、具身智慧與總整專題	VLM 應用、臺語語音、VLA、總整專題

本冊四章的能力遞進

章	回答什麼問題	你會做出什麼	被誰使用
第 14 章	輸出怎麼調？	解碼工具箱、KV cache、參數表	第 27、29、30、33、35 章

章	回答什麼問題	你會做出什麼	被誰使用
第 15 章	訓練怎麼穩？	工業級訓練迴圈、哨兵與回滾	第 22、23、25、40 章
第 16 章	結論可信嗎？	實驗矩陣、統計工具、可信度檢查表	第 17、20、22、25、36 章
第 17 章	資源夠不夠？	尺度擬合器、算力預算書	第 21、22、26、40 章

第 4 篇

預訓練、解碼與實驗科學

Pretraining, Decoding, and Experimental Science

第 14–17 章

本篇承諾：把「跑得動」升級為「說得清楚」。訓練迴圈、解碼策略、診斷方法與尺度定律，讓每個結論都有資料支撐。

第 14 章

解碼、取樣、受限生成與 KV Cache

Decoding, Sampling, Constrained Generation, and KV Cache

難度：核心 | 建議授課：第 14 週 | 硬體需求：A 級可完成全部實作；有 GPU 者可加測長序列的快取效能

叫獸開講

同一個模型，把溫度從 0.2 調到 1.2，它會從一個嚴謹的助教變成一個滿嘴胡話的詩人。權重完全沒變，變的只是你怎麼從機率裡抽籤。這一章要教你的，是模型訓練完之後那些「免費」的旋鈕——它們不花一分錢的訓練成本，卻能決定你的產品好不好用。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明 greedy 與 beam search 在開放式生成中失效的原因。
2. 推導溫度縮放對機率分布與注意力熵的影響，並用數值驗證。
3. 實作 top-k、top-p 與 min-p 三種截斷取樣，並說明各自的適用情境。
4. 診斷退化輸出（重複、無意義循環），並正確使用重複懲罰。
5. 實作 KV cache，量測它帶來的加速與記憶體代價。
6. 區分 prefill 與 decode 兩階段的效能特性，說明各自的瓶頸。
7. 用文法約束或 schema 強制模型輸出合法的結構化資料。
8. 說明推測解碼的原理與它為什麼能在不改變輸出分布的前提下加速。
9. 為臺灣的不同任務（公文摘要、創意寫作、程式生成）設計對應的解碼參數表。

本章在全書中的位置

第 13 章你的模型只會 greedy 解碼，而且每生成一個詞元就要重算整個序列。本章把這兩件事都修好：解碼策略讓輸出品質可控，KV cache 讓速度快上數十倍。這兩者都是純推論最佳化——不需要重新訓練，卻能顯著改變使用體驗。第 33、35 章的應用與部署，全部建立在本章的基礎上。

14.1 解碼是一個獨立的設計空間

14.1.1 訓練與生成的分工

一個訓練好的語言模型提供的只有一件事：給定前文，輸出詞彙表上的機率分布。至於「從這個分布中選哪一個」，模型不管——那是解碼策略的工作。

階段	模型負責	你負責	本書章節
訓練	學出好的機率分布	資料、架構、超參數	第 9、12、15 章
解碼	輸出一分布	如何選、選幾次、何時停	本章
後訓練	調整分布的形狀	偏好資料與對齊目標	第 23、25 章

這個分工有一個重要的實務意涵：解碼策略是免費的。改變溫度不需要重新訓練、不需要更多資料、不需要一張 GPU——只要改一個數字。在資源受限的臺灣情境中，這是投資報酬率最高的最佳化手段之一。

14.1.2 為什麼 greedy 不夠

第 13.5.2 節你已經親眼看過 greedy 生成的重複問題。這裡把原因講清楚。

greedy 每一步都選機率最高的詞元。這在兩種情況下有問題。第一，它是完全確定的——同樣的提示永遠得到同樣的輸出，沒有任何多樣性。第二，它是短視的——這一步的最佳選擇不保證整體最佳。更根本的問題是：人類的語言本來就不是「每一步都選最可能的詞」。真實的文本有起伏、有意外、有較低機率但更有趣的選擇。一個永遠選最高機率的模型，產生的文字會平淡、重複、缺乏資訊量——這個現象在研究上被稱為退化。

14.1.3 beam search 為什麼在這裡失效

beam search 保留 k 條最有希望的候選路徑，每一步各自擴展再篩選。它在機器翻譯上表現優異——因為翻譯有明確的目標，「最可能的譯文」通常就是最好的譯文。

但在開放式生成上，beam search 反而更糟。

問題	機制	症狀	章節
更嚴重的退化	搜尋得越好，越集中在高機率路徑	文字比 greedy 更平淡	本節
長度偏誤	每多一個詞元就乘一個小於 1 的機率	傾向產生短句	第 5.3.6 節
成本高	每一步要維護 k 條路徑	記憶體與計算都乘以 k	14.5 節

問題	機制	症狀	章節
多樣性差	k 條路徑往往高度相似	換 seed 也得到差不多的結果	本節

第一列是一個很反直覺的發現：把搜尋做得更好，結果反而更差。這說明「模型認為最可能的文字」與「人類覺得好的文字」之間存在系統性落差。所以現代的開放式生成幾乎都用取樣而非搜尋——本章的重點也在取樣。

beam search 仍有它的位置：翻譯、摘要、以及任何「有明確正確答案」的任務。第 29 章的結構化輸出與第 30 章的引用生成也可能用到它的變體。

14.2 溫度：分布形狀的旋鈕

14.2.1 定義

溫度縮放在 softmax 之前把 logits 除以一個常數 T 。

$$p_i = \text{softmax}(z_i / T)$$

T 的範圍	效果	直覺	適用
$T \rightarrow 0$	趨近 one-hot	等價於 greedy	需要確定性時
$T < 1$	分布變尖	更保守、更聚焦	事實型任務
$T = 1$	模型原始分布	不做任何調整	基準
$T > 1$	分布變平	更多樣、更冒險	創意寫作
$T \rightarrow \infty$	趨近均勻分布	純隨機	沒有實用價值

14.2.2 用數值看清楚它做了什麼

取一組 logits $[3.2, 2.8, 2.1, 1.5, 0.9, 0.4, -0.3, -1.2]$ ，掃描不同溫度：

T	最大機率	前三名機率	熵	有效候選數
0.2	0.8775	0.877 / 0.119 / 0.004	0.390	1.48
0.5	0.6218	0.622 / 0.279 / 0.069	0.967	2.63

T	最大機率	前三名機率	熵	有效候選數
0.7	0.5195	0.519 / 0.293 / 0.108	1.229	3.42
1.0	0.4185	0.419 / 0.281 / 0.139	1.501	4.49
1.5	0.3237	0.324 / 0.248 / 0.155	1.750	5.76
2.0	0.2724	0.272 / 0.223 / 0.157	1.873	6.51

「有效候選數」是熵取指數（第 11 章的計算題用過同樣的概念）。它的意義是：模型實質上在幾個選項之間猶豫。T = 0.2 時只有 1.48，幾乎等同於 greedy；T = 2.0 時有 6.51，接近八個候選的一半以上。

這張表可以直接用來選溫度。如果你希望模型在三到四個合理選項中挑一個，T = 0.7 到 1.0 是合適的範圍；如果你要它幾乎不猶豫，T = 0.2 以下。

一個常見的誤解

溫度不改變詞元的排序，只改變它們之間的相對差距。不管 T 是多少，機率最高的永遠是同一個詞元。所以「調高溫度會讓模型講出更奇怪的東西」的機制不是「排序改變」，而是「低機率選項變得更容易被抽中」。理解這一點，你就知道為什麼溫度無法讓模型產生它本來完全不會想到的內容。

14.2.3 實作

formosa/decoding/temperature.py

```
import torch
import torch.nn.functional as F

def apply_temperature(logits: torch.Tensor, temperature: float):
    """logits: (... , V) T <= 0 視為 greedy"""
    if temperature <= 0:
        # 不要真的除以 0；改為回傳 one-hot 等效的極尖分布
        out = torch.full_like(logits, float("-inf"))
        out.scatter_(-1, logits.argmax(-1, keepdim=True), 0.0)
        return out
    return logits / temperature

def distribution_stats(logits, temperature=1.0):
    """回傳可用於報告的三個數字（見 14.2.2 節）"""
    p = F.softmax(apply_temperature(logits, temperature), dim=-1)
    ent = -(p * (p + 1e-12).log()).sum(-1)
    return {
        "max_prob": float(p.max()),
        "entropy": float(ent.mean()),
        "effective_choices": float(ent.mean().exp()),
    }
```

$T \leq 0$ 的處理是一個實務細節。使用者介面上常把溫度 0 當成「完全確定」，但程式裡不能真的除以零。上面的做法是直接構造一個等效於 argmax 的分布，這樣後續的取樣程式碼不必特別處理這個情況。

14.3 截斷取樣：top-k、top-p 與 min-p

14.3.1 為什麼光有溫度不夠

溫度會等比例地提升所有低機率選項。但詞彙表有數萬個詞元，其中絕大多數在當前語境下完全不合理。即使每個的機率只有百萬分之一，數萬個加起來仍有可觀的機率質量——這叫做長尾問題。

調高溫度會讓這條長尾變得更肥，於是模型偶爾會抽到一個完全不著邊際的詞元，然後整段文字就崩壞了。截斷取樣的作用就是先把長尾砍掉，再從剩下的候選中抽樣。

14.3.2 三種截斷方式

沿用 14.2.2 節的那組機率 [0.4185, 0.2805, 0.1393, 0.0765, 0.0420, 0.0254, 0.0126, 0.0051]：

方法	規則	本例的結果	特性
top-k	只保留機率最高的 k 個	k=3 保留 3 個	候選數固定
top-p (核取樣)	保留累積機率達到 p 的最小集合	p=0.9 保留 4 個	候選數隨情境變動
min-p	保留機率不低於「最大機率 × m」者	m=0.1 保留 5 個	相對門檻，適應性強

把三者在不同參數下的行為列出來，差異就清楚了：

參數	保留數量	截斷後的分布	說明
top-k = 3	3	0.499 / 0.335 / 0.166	重新正規化
top-k = 5	5	0.437 / 0.293 / 0.146 / 0.080 / 0.044	—
top-p = 0.8	3	累積至 0.8384	剛好跨過門檻

參數	保留數量	截斷後的分布	說明
top-p = 0.9	4	累積至 0.9148	—
min-p = 0.1	5	門檻 = 0.0419	0.4185×0.1
min-p = 0.2	3	門檻 = 0.0837	門檻越高保留越少

14.3.3 為什麼 top-p 通常優於 top-k

關鍵在於「模型有多確定」是隨語境變化的。

考慮兩種情況。第一種：模型在生成「中華民國憲法第一 _」，下一個詞元幾乎確定是「條」，機率可能高達 0.99。此時 top-k = 50 會保留 49 個完全不合理的候選，雖然它們的機率很低，但仍有被抽中的風險。

第二種：模型在生成一段開放式的描述，前十個候選的機率都在 0.05 到 0.12 之間。此時 top-k = 5 會粗暴地砍掉五個同樣合理的選項，人為降低了多樣性。

top-p 自動適應這兩種情況：第一種只保留一個候選，第二種保留十幾個。這就是它被稱為「核取樣」的原因——它動態決定機率質量的核心在哪裡。

情境	模型確定度	top-k = 50	top-p = 0.9
「第一 _」→ 「條」	極高 (0.99)	保留 50 個，含 49 個雜訊	保留 1 個
開放式描述	低 (前十名相近)	保留 50 個，過多	保留約 12 個
中等確定	中	保留 50 個	保留約 5 個

14.3.4 min-p 的動機

min-p 是較新的方法，動機是 top-p 在高溫下的行為問題。當溫度調高時，分布變平，累積到 0.9 需要納入更多候選——包括那些原本很不合理的。min-p 用「相對於最大機率」的門檻，在高溫下仍能維持一定的品質下限。

實務上的選擇建議：

情境	建議	典型參數	理由
一般對話與寫作	top-p	T=0.8, p=0.9	最廣泛使用、行為可預期
需要高多樣性	min-p + 高溫	T=1.2, m=0.05	高溫下品質較穩
事實問答	低溫或 greedy	T=0.1 至 0.3	不需要多樣性
程式生成	低溫 + top-p	T=0.2, p=0.95	語法錯誤代價高

14.3.5 完整實作

formosa/decoding/sampling.py

```
import torch
import torch.nn.functional as F

def top_k_filter(logits: torch.Tensor, k: int):
    """保留最高的 k 個，其餘設為負無限大"""
    if k <= 0 or k >= logits.size(-1):
        return logits
    kth = logits.topk(k, dim=-1).values[..., -1:] # 第 k 大的值
    return logits.masked_fill(logits < kth,
                              torch.finfo(logits.dtype).min)

def top_p_filter(logits: torch.Tensor, p: float):
    """核取樣：保留累積機率達 p 的最小集合"""
    if p >= 1.0:
        return logits
    sorted_logits, sorted_idx = logits.sort(dim=-1, descending=True)
    probs = F.softmax(sorted_logits, dim=-1)
    cum = probs.cumsum(dim=-1)

    # 標記「累積已超過 p」的位置；右移一格確保至少保留一個
    remove = cum - probs > p
    remove[..., 0] = False # 永遠保留第一名

    sorted_logits = sorted_logits.masked_fill(
        remove, torch.finfo(logits.dtype).min)
    # 依原索引還原順序
    return torch.empty_like(logits).scatter_(-1, sorted_idx, sorted_logits)

def min_p_filter(logits: torch.Tensor, min_p: float):
    """相對門檻：保留機率 >= min_p * max_prob 者"""
    if min_p <= 0:
        return logits
    probs = F.softmax(logits, dim=-1)
    threshold = min_p * probs.max(dim=-1, keepdim=True).values
    return logits.masked_fill(probs < threshold,
                              torch.finfo(logits.dtype).min)
```

`top_p_filter` 中那行 `remove[..., 0] = False` 是必要的保護：如果第一名的機率就已經超過 p (例如 $p = 0.5$ 而最高機率是 0.7)，沒有這行會把所有候選都刪掉，導致 softmax 後全是 NaN。這是一個很容易漏掉、且只在特定輸入下才會爆炸的邊界情況。

過濾器的套用順序

溫度應該在截斷之前套用，因為截斷的判斷依據是機率分布，而溫度會改變分布。如果順序反過來——先截斷再調溫度——高溫就無法讓被砍掉的候選回來，等於溫度只在少數候選之間起作用。本書的標準順序是：重複懲罰 → 溫度 → top-k → top-p → min-p → 取樣。

14.4 退化輸出與重複懲罰

14.4.1 退化的三種形式

形式	樣子	成因	對策
詞元級重複	「的的的的的」	自我強化的機率迴圈	重複懲罰
短語級重複	同一句話反覆出現	進入吸引子狀態	no-repeat n-gram
主題級停滯	不斷換句話說同一件事	缺乏新資訊、模型容量不足	提高溫度或改進提示

第一與第二種在 greedy 與低溫時特別容易發生，第三種則是模型能力的問題，解碼調不出來。診斷時先區分這三者，才不會把調得動的問題與調不動的問題混在一起。

14.4.2 重複懲罰的兩種形式

乘法式： $z_i \leftarrow z_i / r$ (若 $z_i > 0$) 或 $z_i \times r$ (若 $z_i \leq 0$)，對已出現的詞元

加法式： $z_i \leftarrow z_i - \alpha$ (存在懲罰) $- \beta \times \text{count}_i$ (頻率懲罰)

方式	特性	參數範圍	注意
乘法式懲罰	對正負 logits 的處理不對稱	$r = 1.0$ 至 1.3	過大會嚴重扭曲分布
存在懲罰	出現過就扣固定值，不看次數	$\alpha = 0$ 至 1.0	鼓勵使用新詞
頻率懲罰	按出現次數累加扣分	$\beta = 0$ 至 1.0	對高頻功能詞影響大

方式	特性	參數範圍	注意
no-repeat n-gram	直接禁止重複的 n 元組	n = 3 至 6	硬性禁止，可能誤傷

對繁體中文而言有一個特別的注意事項：功能詞（的、是、在、了）本來就會高頻出現，頻率懲罰會不成比例地打擊它們，導致文字讀起來不自然。實務上建議優先使用存在懲罰或 no-repeat n-gram，並把功能詞排除在懲罰之外。

formosa/decoding/repetition.py

```
import torch

def repetition_penalty(logits, generated_ids, penalty: float = 1.1,
                       exclude: set[int] | None = None):
    """乘法式懲罰。exclude 用於排除中文功能詞"""
    if penalty == 1.0:
        return logits
    for token_id in set(generated_ids.tolist()):
        if exclude and token_id in exclude:
            continue
        z = logits[..., token_id]
        # 正的除以 penalty (變小)，負的乘以 penalty (更負)
        logits[..., token_id] = torch.where(z > 0, z / penalty, z * penalty)
    return logits

def no_repeat_ngram(logits, generated_ids, n: int = 4):
    """禁止產生會造成 n-gram 重複的詞元"""
    ids = generated_ids.tolist()
    if len(ids) < n:
        return logits
    prefix = tuple(ids[-(n - 1):])
    banned = set()
    for i in range(len(ids) - n + 1):
        if tuple(ids[i:i + n - 1]) == prefix:
            banned.add(ids[i + n - 1])
    for t in banned:
        logits[..., t] = torch.finfo(logits.dtype).min
    return logits

# 中文高頻功能詞應排除懲罰（需依你的 tokenizer 建立）
def build_function_word_exclusion(tok):
    words = ["的", "是", "在", "了", "和", "與", "有", "為", "也", "不",
            ", ", "。", "、", "；", ":", "「", "」"]
    ids = set()
    for w in words:
        enc = tok.encode(w)
        if len(enc) == 1:
            ids.add(enc[0])
    return ids
```

`build_function_word_exclusion` 是一個臺灣情境的具體實作。國際的解碼函式庫通常沒有這個機制，因為英文的功能詞（`the`、`a`、`of`）在 `subword` 切分後行為不同。你在使用現成函式庫時，可能需要自己加上這一層。

14.5 KV Cache：把平方變成線性

14.5.1 沒有快取時在浪費什麼

第 13.5.1 節的生成迴圈有一個嚴重的浪費：每產生一個新詞元，都要把整個序列重新跑一次前向傳播。但仔細想想，前面那些位置的計算結果並沒有改變——因為因果遮罩保證了它們看不到新加入的詞元（第 11.4 節）。所以第 t 步算出來的 K 與 V ，在第 $t+1$ 步完全可以重複使用。

做法	第 t 步的計算量	生成 T 個詞元的總量	複雜度
無快取	處理 t 個位置	$1 + 2 + \dots + T = T(T+1)/2$	$O(T^2)$
有快取	只處理 1 個新位置	T	$O(T)$

差距有多大？用實際數字算：

生成長度 T	無快取的總位置數	有快取	節省倍數
128	8,256	128	64.5 倍
512	131,328	512	256.5 倍
2,048	2,098,176	2,048	1,024.5 倍

生成 2,048 個詞元時，快取讓計算量降到千分之一。這不是微調級的最佳化，而是「能不能用」的差別——沒有 KV cache，任何實用的生成服務都不可能存在。

14.5.2 快取什麼、不快取什麼

項目	要快取嗎	理由	注意
K （每層每頭）	要	過去位置的 K 不會改變	主要記憶體開銷

項目	要快取嗎	理由	注意
V (每層每頭)	要	同上	同上
Q	不要	每步只需要當前位置的 Q	算完即丟
注意力權重	不要	每步都不同	—
FFN 中間值	不要	逐位置獨立，不跨步重用	—
RoPE 的 cos/sin	要 (預先算好)	與位置有關、與內容無關	第 10.4.3 節

第六列有一個關鍵細節：使用 RoPE 時，快取中存的是「已經旋轉過的」K。這表示每個位置的絕對索引必須被正確追蹤——如果你在續接生成時把位置算錯，模型的行為會突然變得很奇怪。第 10.4.6 節提過這是實作 KV cache 時最常見的 bug。

14.5.3 實作

formosa/decoding/kv_cache.py

```
import torch

class KVCache:
    """每一層一組 (K, V)，預先配置好記憶體避免反覆 realloc"""

    def __init__(self, n_layers, batch, max_len, n_kv_heads, d_head,
                 dtype=torch.bfloat16, device="cpu"):
        shape = (batch, n_kv_heads, max_len, d_head)
        self.k = [torch.zeros(shape, dtype=dtype, device=device)
                  for _ in range(n_layers)]
        self.v = [torch.zeros(shape, dtype=dtype, device=device)
                  for _ in range(n_layers)]
        self.length = 0          # 目前已填入的位置數
        self.max_len = max_len

    def update(self, layer: int, k_new, v_new):
        """k_new/v_new: (B, H_kv, T_new, d_head)
        回傳到目前為止的完整 K 與 V"""
        T_new = k_new.size(2)
        start = self.length
        end = start + T_new
        assert end <= self.max_len, \
            f"快取溢出: {end} > {self.max_len}"
        self.k[layer][:, :, start:end] = k_new
        self.v[layer][:, :, start:end] = v_new
        return self.k[layer][:, :, :end], self.v[layer][:, :, :end]

    def advance(self, n: int):
        """所有層都更新完之後才推進長度"""
        self.length += n
```

```
def reset(self):
    self.length = 0

def memory_bytes(self):
    return sum(t.numel() * t.element_size()
               for t in self.k + self.v)
```

advance 與 update 分開是刻意的設計：所有層都必須用同一個起始位置寫入，如果在每層的 update 中就推進長度，第二層開始就會寫錯位置。這是一個容易出錯的地方，把它拆成兩個方法可以讓錯誤更難發生。

注意力層的快取支援

```
def forward(self, x, cos, sin, cache=None, layer_idx=None,
            pos_offset: int = 0):
    B, T, C = x.shape
    H, dh = self.n_heads, self.d_head

    q, k, v = self.qkv(x).chunk(3, dim=-1)
    q = q.view(B, T, H, dh).transpose(1, 2)
    k = k.view(B, T, self.n_kv, dh).transpose(1, 2)
    v = v.view(B, T, self.n_kv, dh).transpose(1, 2)

    # RoPE 必須使用絕對位置：續接生成時要加上偏移
    c = cos[pos_offset:pos_offset + T]
    s = sin[pos_offset:pos_offset + T]
    q, k = apply_rope(q, c, s), apply_rope(k, c, s)

    if cache is not None:
        k, v = cache.update(layer_idx, k, v)      # 取回完整歷史

    k = self._repeat_kv(k, self.n_rep)          # GQA 展開 (第 11.6 節)
    v = self._repeat_kv(v, self.n_rep)

    # 生成單一詞元時 T=1，此時不需要因果遮罩
    causal = (cache is None) or (T > 1)
    out = F.scaled_dot_product_attention(q, k, v, is_causal=causal)

    out = out.transpose(1, 2).contiguous().view(B, T, C)
    return self.proj(out)
```

倒數第三行的判斷值得說明。當 $T = 1$ 時 (decode 階段)，這一個 Query 應該可以看到所有快取中的 Key——包括「過去的全部」，而不需要遮罩。如果錯誤地套用 `is_causal=True`，PyTorch 會以為 Q 與 K 等長而套用三角遮罩，導致模型只看得到第一個位置。這是實作 KV cache 時第二常見的 bug。

14.5.4 記憶體代價

第 11.6.1 節給過公式，這裡填入實際數字 ($L = 32$ 、 $H_{kv} = 8$ 、 $d_{head} = 128$ 、BF16)：

序列長度	B = 1	B = 16	意涵
512	0.067 GB	1.07 GB	幾乎無感
2,048	0.268 GB	4.29 GB	開始需要規劃
8,192	1.074 GB	17.18 GB	單卡吃緊
32,768	4.295 GB	68.72 GB	需要多卡或分頁

注意 B = 16 那一欄的成長。在多使用者的服務中，KV cache 會隨併發數線性增加——這往往才是限制併發量的真正瓶頸，而不是算力。第 35 章的推論服務會處理這個問題，屆時你會遇到分頁注意力（paged attention）這類專門管理 KV cache 記憶體的技术。

14.5.5 快取的三種失效情況

情況	為什麼快取失效	處置	章節
提示改變	前綴不同，快取內容無效	重新 prefill	本節
超出 max_len	快取空間用完	滑動視窗或截斷	第 20 章
批次中長度不一	對齊困難	padding 或分頁管理	第 35 章

第一列有一個重要的最佳化機會：如果多個請求共用同一個系統提示（第 29 章的對話模板），那段前綴的 KV cache 可以重複使用。這稱為前綴快取，在實務上能顯著降低成本——第 33.5 節會展開。

14.6 Prefill 與 Decode：兩種完全不同的工作

14.6.1 兩個階段

階段	做什麼	處理的詞元數	特性
prefill	把提示一次算完，填滿快取	T_0 （提示長度）	平行度高、計算受限
decode	一次產生一個詞元	每步 1 個	平行度低、頻寬受限

這兩個階段的效能特性差異極大，理解它是第 35 章部署最佳化的基礎。

14.6.2 為什麼 decode 是頻寬受限的

用第 3.2.5 節的 FLOPs 公式做一個計算。一個 7B 參數的模型，BF16 權重共 14 GB。

一次處理的詞元數	計算量	需搬移的權重	運算強度
1 (decode)	14.0 GFLOPs	14.0 GB	1.0
512 (prefill 或批次)	7,168 GFLOPs	14.0 GB	512

運算強度 (arithmetic intensity) 是「每搬移一個位元組能做多少次浮點運算」。decode 階段的運算強度只有 1——這意味著 GPU 幾乎所有時間都在等待權重從記憶體搬進來，算力完全用不上。

這解釋了一個常見的困惑：為什麼生成速度不會因為換一張更強的 GPU 而等比例變快？因為瓶頸不在算力，在記憶體頻寬。第 18 章會用 profiler 讓你親眼看到這件事。

想加速 decode	有效嗎	為什麼	章節
換算力更強的 GPU	有限	瓶頸在頻寬不在算力	第 18 章
提高批次 (多使用者)	非常有效	權重搬一次服務多個請求	第 35 章
量化權重 (4-bit)	非常有效	直接減少搬移的位元組數	第 35 章
用 GQA 縮小 KV cache	有效	減少每步要讀的快取	第 11.6 節
推測解碼	有效	一次驗證多個詞元	14.7 節

這張表傳達一個重要的實務判斷：對推論服務而言，「提高批次」與「量化」的效益遠高於「買更快的 GPU」。這對預算有限的臺灣單位是好消息——第 35 章會把它變成具體的部署決策。

14.7 推測解碼：用小模型加速大模型

14.7.1 核心想法

既然 decode 是頻寬受限的，那麼「一次處理多個詞元」就能大幅提升效率。但自回歸生成的本質是循序的——你不知道第 $t+1$ 個詞元是什麼，就無法算第 $t+2$ 個。

推測解碼的巧思是：用一個小而快的草稿模型先猜幾個詞元，再用大模型一次驗證它們。

1. 草稿模型連續生成 k 個候選詞元（很快，因為模型小）。
2. 大模型對這 k 個位置一次前向傳播，取得每個位置的真實分布（只需一次，因為可以平行）。
3. 逐一比對：如果大模型也會產生同樣的詞元，就接受；第一個不同意的地方就停止。
4. 被接受的部分直接採納，不同意的位置用大模型的分布重新取樣。

關鍵性質是：經過正確的接受與重取樣設計，最終輸出的分布與「純粹用大模型生成」完全相同。這不是近似，而是精確等價——你得到的是加速，不是品質妥協。

14.7.2 效益取決於接受率

接受率	每次驗證的平均產出	加速倍數	什麼情況
低（約 0.3）	約 1.4 個詞元	約 1.3 倍	草稿模型太差或領域不符
中（約 0.6）	約 2.4 個詞元	約 2 倍	常見情況
高（約 0.8）	約 4 個詞元	約 3 倍以上	文字可預測性高

（表中為示意量級，實際數值取決於模型對、草稿長度 k 與任務類型，必須實測。）

對繁體中文有一個值得注意的觀察：格式固定的文本（公文、法規、表格）通常接受率較高，因為下一個詞元可預測性強；開放式創作則較低。所以推測解碼在公部門文書處理這類應用上，效益可能特別好。

14.7.3 實務考量

考量	說明	影響	章節
草稿模型從哪來	小模型、蒸餾模型或同族的小版本	需額外訓練或取得	第 26 章
額外記憶體	同時載入兩個模型	小模型通常只占數百 MB	第 35 章
k 的選擇	太小效益低、太大浪費驗證	通常 3 到 8	需實測
與批次的互動	批次越大，推測解碼的相對效益越低	因為批次本身已提升運算強度	第 35 章

最後一列是一個容易被忽略的取捨：推測解碼解決的是「批次為 1 時運算強度太低」的問題。如果你的服務已經有足夠的併發量（批次夠大），這個問題本來就不存在，推測解碼的額外收益就有限。所以它最適合的場景是「單一使用者、要求低延遲」——例如本地部署的助理或編輯器外掛。

14.8 受限生成：強制輸出合法的結構

14.8.1 為什麼需要

很多應用需要模型輸出可以被程式解析的格式：JSON、特定的標籤、固定的欄位。單靠提示詞要求「請輸出 JSON」是不可靠的——模型會在某些情況下加上說明文字、漏掉引號、或產生不合法的巢狀結構。

受限生成的想法是：在每一步取樣之前，先把「會導致格式錯誤」的詞元全部遮蔽。這樣模型只能在合法的選項中選擇，輸出必然合法。

14.8.2 三種層次

層次	做法	保證	成本
提示要求	在系統提示中說明格式	不保證	零
輸出驗證與重試	解析失敗就重試	最終合法，但可能多次呼叫	延遲與費用
文法約束解碼	每一步遮蔽非法詞元	一定合法	需維護狀態機

本書的建議是分層使用：提示要求是基本盤、驗證重試是安全網、文法約束用在絕對不能失敗的場合。

第 29 章會把這套流程完整展開，並量測各層的合規率。

14.8.3 一個最小的 JSON 約束

formosa/decoding/constrained.py (概念示範)

```
class JsonStateMachine:
    """極簡示範：只處理 {"key": "value"} 這種扁平物件
    正式使用請採用成熟的文法約束函式庫（見研究延伸）"""

    def __init__(self, allowed_keys: list[str]):
        self.keys = allowed_keys
        self.state = "start"
        self.buffer = ""

    def allowed_prefixes(self) -> list[str]:
        """回傳目前狀態下合法的字串前綴"""
        if self.state == "start":
```

```

        return ["{"]
    if self.state == "expect_key":
        return [f'"{k}"' for k in self.keys]
    if self.state == "expect_colon":
        return [":"]
    if self.state == "in_value":
        return None # 值可以是任意內容，直到遇到引號
    if self.state == "expect_comma_or_end":
        return [",", "}"]
    return None

def constrain_logits(logits, tok, machine):
    """把不符合目前狀態的詞元遮蔽掉"""
    allowed = machine.allowed_prefixes()
    if allowed is None:
        return logits # 此狀態不設限
    mask = torch.full_like(logits, torch.finfo(logits.dtype).min)
    for token_id in range(logits.size(-1)):
        piece = tok.decode([token_id])
        # 只要這個詞元是某個合法前綴的一部分就允許
        if any(a.startswith(piece) or piece.startswith(a) for a in allowed):
            mask[..., token_id] = logits[..., token_id]
    return mask

```

這段程式碼刻意寫得很簡陋，目的是讓你看清楚機制：受限生成本質上就是「維護一個狀態機，每一步用它決定哪些詞元合法」。正式使用時應該採用成熟的函式庫，因為它們會預先編譯詞元到狀態的對照表，避免每一步都掃描整個詞彙表（那會非常慢）。

受限生成的代價

強制格式合法不代表內容正確。模型可能產生一個結構完美但欄位值完全錯誤的 JSON。而且過度的約束可能讓模型「說不出它想說的話」，反而降低品質。第 29 章會用實驗量測這個取舍——這是本書一貫的立場：任何約束都有代價，代價要量出來。

14.9 為臺灣任務設計解碼參數表

14.9.1 任務分類

不同任務對「確定性」與「多樣性」的需求完全不同。把它們寫成一張表，就成為系統的解碼設定依據。

任務	T	top-p	重複懲罰	最大長度	備註
公文摘要	0.3	0.9	1.05	512	需忠實、少發揮
法規問答	0.1	0.9	1.0	1024	接近確定性

任務	T	top-p	重複懲罰	最大長度	備註
教案設計	0.8	0.92	1.1	2048	需要一些創意
創意寫作	1.0	0.95	1.15	4096	多樣性優先
程式生成	0.2	0.95	1.0	2048	語法錯誤代價高
結構化輸出	0.0	—	—	512	搭配文法約束
臺語轉寫	0.3	0.9	1.05	512	低資源，宜保守

(以上為起點建議，實際參數必須在你的模型與任務上實測調整。)

幾個設計理由值得說明。法規問答用 $T = 0.1$ 而非 0，是為了保留一點點多樣性以便偵測模型的不確定 (第 27 章的多樣本一致性會用到)。臺語轉寫用較保守的參數，是因為低資源語言的模型分布本來就不夠可靠 (第 7.5.2 節)，高溫會放大錯誤。

14.9.2 把設定寫成可版本控制的檔案

configs/decoding_profiles.yaml

```
# 解碼設定檔：與提示規格 ( 第 29 章 ) 一併版本控制
version: "2026-08-20"

defaults: &defaults
  temperature: 0.7
  top_p: 0.9
  top_k: 0
  min_p: 0.0
  repetition_penalty: 1.05
  no_repeat_ngram: 0
  max_new_tokens: 1024
  stop_sequences: ["<|end|>"]

profiles:
  gov_summary:
    <<: *defaults
    temperature: 0.3
    max_new_tokens: 512
    note: "公文摘要：忠實優先，避免發揮"

  law_qa:
    <<: *defaults
    temperature: 0.1
    repetition_penalty: 1.0
    note: "法規問答：接近確定性，搭配 RAG 引用"

  creative:
    <<: *defaults
```

```
temperature: 1.0
top_p: 0.95
repetition_penalty: 1.15
max_new_tokens: 4096

code:
<<: *defaults
temperature: 0.2
top_p: 0.95
repetition_penalty: 1.0
stop_sequences: ["<|end|>", "\n\n\n"]
```

把解碼設定當成程式碼一樣版本控制，理由與第 4.7.1 節相同：如果它藏在程式裡或靠人記憶，你就無法回答「上次那個好結果是用什麼參數跑的」。第 29 章的提示規格書會與這份檔案並列，兩者一起構成系統的行為定義。

程式實作

本章三個實作都建立在第 13 章的 MiniFormosaGPT 上。完成之後，你的模型就從「能生成」進步到「生成得好且快」。

程式 14A 完整解碼工具箱

目標：實作七種解碼策略並支援批次比較，讓你能在同一個提示上一次看到所有策略的輸出。

策略	參數	必須通過的測試	節次
greedy	—	確定性：兩次結果相同	14.1
溫度取樣	T	熵隨 T 單調上升	14.2
top-k	k	恰好保留 k 個候選	14.3
top-p	p	保留數符合累積機率	14.3
min-p	m	門檻為最大機率乘以 m	14.3
重複懲罰	r	已出現詞元的機率下降	14.4
no-repeat n-gram	n	不產生重複的 n 元組	14.4

tests/test_decoding.py (節錄)

```
import math, torch, pytest
import torch.nn.functional as F
from formosa.decoding.sampling import (top_k_filter, top_p_filter,
                                       min_p_filter)
from formosa.decoding.temperature import apply_temperature

LOGITS = torch.tensor([3.2, 2.8, 2.1, 1.5, 0.9, 0.4, -0.3, -1.2])

@pytest.mark.parametrize("k,expected", [(1, 1), (3, 3), (5, 5)])
def test_top_k_count(k, expected):
    out = top_k_filter(LOGITS.clone(), k)
    assert (out > torch.finfo(out.dtype).min).sum() == expected

@pytest.mark.parametrize("p,expected", [(0.5, 2), (0.8, 3), (0.9, 4)])
def test_top_p_count(p, expected):
    out = top_p_filter(LOGITS.clone(), p)
    assert (out > torch.finfo(out.dtype).min).sum() == expected
```

```

@pytest.mark.parametrize("m,expected", [(0.1, 5), (0.2, 3)])
def test_min_p_count(m, expected):
    out = min_p_filter(LOGITS.clone(), m)
    assert (out > torch.finfo(out.dtype).min).sum() == expected

def test_temperature_monotonic_entropy():
    """熵必須隨溫度單調上升"""
    ents = []
    for T in [0.2, 0.5, 0.7, 1.0, 1.5, 2.0]:
        p = F.softmax(apply_temperature(LOGITS, T), dim=-1)
        ents.append(float(-(p * (p + 1e-12)).log()).sum()))
    assert all(ents[i] < ents[i + 1] for i in range(len(ents) - 1)), ents

def test_top_p_never_empty():
    """即使 p 很小也必須保留至少一個候選"""
    out = top_p_filter(LOGITS.clone(), 0.01)
    assert (out > torch.finfo(out.dtype).min).sum() >= 1

def test_greedy_deterministic(model, tok):
    a = generate(model, tok, "臺灣", temperature=0.0, max_new_tokens=30)
    b = generate(model, tok, "臺灣", temperature=0.0, max_new_tokens=30)
    assert a == b

```

這七個測試的數字全部來自 14.2 與 14.3 節的表格。這是本書一貫的做法：正文中的每一個數字都應該能被測試驗證，而不只是說明用的。

程式 14B KV Cache 實作與效能量測

目標：為第 13 章的模型加上 KV cache，量測加速倍數與記憶體代價，並驗證輸出與無快取版本完全一致。

ch14/lab14b_kv_bench.py (節錄)

```

import time, torch

@torch.no_grad()
def generate_with_cache(model, idx, max_new_tokens, cache):
    """先 prefill 再逐步 decode"""
    B, T0 = idx.shape
    cache.reset()

    # prefill: 一次處理整個提示
    t0 = time.perf_counter()
    logits = model(idx, cache=cache, pos_offset=0)
    cache.advance(T0)
    prefill_sec = time.perf_counter() - t0

```

```

# decode : 每次一個詞元
t1 = time.perf_counter()
out = idx
for i in range(max_new_tokens):
    nxt = logits[:, -1, :].argmax(-1, keepdim=True)
    out = torch.cat([out, nxt], dim=1)
    logits = model(nxt, cache=cache, pos_offset=T0 + i)
    cache.advance(1)
decode_sec = time.perf_counter() - t1

return out, {
    "prefill_sec": round(prefill_sec, 4),
    "decode_sec": round(decode_sec, 4),
    "tokens_per_sec": round(max_new_tokens / decode_sec, 1),
    "cache_MB": round(cache.memory_bytes() / 1e6, 1),
}

def test_cache_equivalence(model, tok):
    """最重要的驗證：有無快取的輸出必須完全相同"""
    prompt = torch.tensor([tok.encode("臺灣科技大學")])
    a = generate_greedy(model, prompt.clone(), 50)          # 無快取
    cache = KVCache(model.cfg.n_layers, 1, 256,
                    model.cfg.n_kv_heads, model.cfg.d_head)
    b, _ = generate_with_cache(model, prompt.clone(), 50, cache)
    assert torch.equal(a, b), "有無快取的輸出不一致，快取實作有誤"

```

必須產出的四項：

- 等價性驗證：有無快取的 greedy 輸出必須完全相同。這是最重要的一項——不通過就代表實作有誤。
- 加速曲線：生成長度從 32 到 512，畫出「有快取 vs 無快取」的時間曲線。無快取版本應呈平方成長。
- 記憶體量測：實際的 cache_MB 與 14.5.4 節的公式估算對照，差異應在 5% 以內。
- prefill 與 decode 的時間比：對一個 200 詞元的提示、生成 100 個詞元，兩階段各占多少時間？解讀這個比例。

程式 14C 結構化輸出的合規率量測

目標：比較三種層次（提示要求、驗證重試、文法約束）的 JSON 合規率與成本，產出可用於決策的資料。

ch14/lab14c_structured.py (節錄)

```

import json

SCHEMA_TASKS = [
    {"name": "課程資訊",

```

```

    "prompt": "從以下描述抽取課程資訊，輸出 JSON，欄位為 "
                "course_name、credits、semester：\n{text}",
    "keys": ["course_name", "credits", "semester"]},
{"name": "公文要素",
 "prompt": "抽取公文的發文機關、發文日期與主旨，輸出 JSON：\n{text}",
 "keys": ["agency", "date", "subject"]}],
]

def check_compliance(output: str, required_keys: list[str]) -> dict:
    """三層檢查：可解析、欄位齊全、型別正確"""
    result = {"parseable": False, "keys_complete": False,
              "no_extra_text": False}
    stripped = output.strip()
    result["no_extra_text"] = (stripped.startswith("{")
                              and stripped.endswith("}"))

    try:
        obj = json.loads(stripped)
        result["parseable"] = True
        result["keys_complete"] = all(k in obj for k in required_keys)
    except json.JSONDecodeError:
        pass
    return result

def compare_strategies(model, tok, tasks, n_samples=50):
    """比較三種層次，回傳合規率與平均呼叫次數"""
    rows = []
    for strategy in ["prompt_only", "retry", "constrained"]:
        ok, calls = 0, 0
        for task in tasks:
            for _ in range(n_samples):
                out, n_call = run_strategy(model, tok, task, strategy)
                calls += n_call
                c = check_compliance(out, task["keys"])
                ok += all(c.values())
            total = len(tasks) * n_samples
            rows.append({"strategy": strategy,
                        "compliance_rate": round(ok / total, 4),
                        "avg_calls": round(calls / total, 2)})
    return rows

```

這個實驗的價值在於它把「該不該用文法約束」變成一個可以用數字回答的問題。如果提示要求的合規率已經有 0.97，而文法約束要付出實作與維護成本，那可能不值得；如果只有 0.6，那就非用不可。第 29 章會把這套方法擴展到更完整的輸出契約。

系統案例：校務問答的解碼參數設計

本章的系統案例是為第 30 章即將建立的校務問答服務，設計一套完整的解碼參數方案：讓事實型問題走低溫受限生成、開放型問題走較高溫取樣，並用資料支撐每一個參數選擇。

問題分類與路由

問題類型	例子	需求	解碼設定
事實查詢	「畢業學分是幾學分？」	準確、可追溯	T=0.1, 需引用
條文引用	「學則第幾條規定？」	逐字正確	T=0.0 或受限生成
流程說明	「怎麼申請休學？」	完整、有條理	T=0.4
開放建議	「該修哪些選修課？」	多樣、有幫助	T=0.8
無法回答	「明年學費會漲嗎？」	誠實拒答	需在後訓練處理

最後一列指出了一個重要的界線：解碼參數無法讓模型學會拒答。那需要第 25 章的對齊訓練，或第 33 章的應用層防護。本章要做的是「在模型既有能力範圍內把輸出調到最好」，而不是彌補能力的缺口。誠實地標示出哪些問題是解碼調不出來的，本身就是這個案例的重要產出。

交付要求

- 解碼設定檔：至少五個 profile，每個附註使用時機與參數理由。
- 路由邏輯：如何判斷一個問題屬於哪一類（規則、分類器或提示皆可），以及誤判時的降級策略。
- 參數掃描實驗：對至少兩類問題，掃描溫度從 0 到 1.2，量測正確率、重複率與人工評分。
- 重複率量測：用 no-repeat n-gram 的命中率當作退化指標，畫出它與溫度的關係。
- KV cache 效能報告：實際的加速倍數、記憶體占用、以及在你的硬體上能支援的最大併發數估算。
- 一頁分析：哪一類問題最難用解碼參數解決？為什麼？需要在哪一章處理？

評分重點

- 參數選擇是否有資料支撐。「感覺這樣比較好」不予計分，必須有掃描結果。
- 是否誠實標示解碼調不出來的問題（例如拒答、事實錯誤）。
- KV cache 的等價性驗證是否通過。
- 路由的誤判處理：分類錯了會怎樣？有沒有安全的預設值？

章末評量

一、概念題

1. 為什麼說解碼策略是「免費的最佳化」？它與訓練、後訓練的分工是什麼？
2. 為什麼 beam search 在翻譯上有效、在開放式生成上反而更糟？
3. 溫度是否改變詞元的排序？如果不改變，它是如何影響輸出多樣性的？
4. 為什麼光有溫度不夠、還需要截斷取樣？請說明長尾問題。
5. 比較 top-k 與 top-p，說明為什麼後者通常更好。請各舉一個具體情境。
6. 為什麼頻率懲罰對繁體中文特別需要小心？應該如何處置？
7. 解釋 KV cache 為什麼能把複雜度從 $O(T^2)$ 降到 $O(T)$ 。為什麼只快取 K 與 V 而不快取 Q？
8. 為什麼 decode 階段是頻寬受限的？這對「買更快的 GPU 能不能加速生成」有什麼意涵？
9. 推測解碼為什麼能在不改變輸出分布的前提下加速？它最適合什麼場景？

二、計算題

1. 一組 logits 的 softmax 為 $[0.42, 0.28, 0.14, 0.08, 0.04, 0.03, 0.01]$ 。計算 top-p = 0.85 會保留幾個候選；min-p = 0.1 呢？
2. 某分布的熵為 1.5。計算有效候選數。若溫度調高使熵變成 1.9，有效候選數變成多少？
3. 生成 1,024 個詞元。無 KV cache 時總共處理多少個位置？有快取時呢？節省幾倍？
4. $L = 40$ 、 $H_{kv} = 8$ 、 $d_{head} = 128$ 、BF16。計算 $B = 1$ 、 $T = 4,096$ 的 KV cache 大小；若併發 32 個請求呢？
5. 一個 13B 模型（BF16 權重 26 GB）在 decode 階段一次處理 1 個詞元。計算運算強度。若批次提高到 64 呢？
6. 推測解碼的草稿長度 $k = 5$ 、接受率 0.7。若接受行為可視為獨立，計算每次驗證的期望產出詞元數。

三、除錯題

1. 加上 KV cache 後，生成的文字與無快取版本不同。請列出四個最可能的原因與檢查順序。
2. 使用 top-p = 0.3 時偶爾出現 NaN。請指出原因與修正。
3. 生成的文字在第 50 個詞元左右開始重複同一句話，調高溫度也沒用。請說明可能的原因與三種對策。
4. 同學的解碼器在溫度 0 時拋出除以零的例外。請說明正確的處理方式。

5. 受限生成的 JSON 輸出格式完全合法，但欄位內容全是空字串。請說明這代表什麼。
6. 加上 KV cache 後速度反而變慢。請列出三個可能原因（提示：其中一個與記憶體配置有關）。

四、系統設計題

1. 你要為一個部署在單張 24 GB GPU 上的服務規劃併發量。模型 7B (BF16)、 $L = 32$ 、 $H_{kv} = 8$ 、 $d_{head} = 128$ 、平均對話長度 4,096 詞元。請計算能支援多少併發請求，並提出至少三種提高併發的方法與各自的代價。
2. 你的團隊要為一個「臺灣公文寫作助理」設計完整的解碼方案。需求包括：格式必須正確、用語必須符合公文規範、不可編造條號、但主旨與說明段落需要一定的表達彈性。請設計分段的解碼策略，並說明哪些需求是解碼解決不了的。

五、臺灣情境與倫理題

1. 本章 14.7.2 節提到格式固定的文本（公文、法規）推測解碼接受率較高。請設計一個實驗驗證這個假設，並說明若成立，對臺灣公部門的應用有什麼意涵。
2. 解碼參數會顯著影響模型的輸出風格與確定性。若一個服務對外宣稱「由 AI 生成」但未說明解碼設定，使用者可能無法理解為什麼同樣的問題每次得到不同答案。請討論在什麼情況下應該向使用者揭露這些設定，以及如何用非技術語言說明。

評量提示（給自學者）

- 計算題第 1 題：累積至 0.42、0.70、0.84、0.92—— $\text{top-p}=0.85$ 保留 4 個。 $\text{min-p}=0.1$ 門檻為 0.042，保留機率 ≥ 0.042 的共 4 個。
- 計算題第 2 題： $\exp(1.5) \approx 4.48$ ； $\exp(1.9) \approx 6.69$ 。
- 計算題第 3 題：無快取為 $1024 \times 1025 \div 2 = 524,800$ ；有快取為 1,024；節省約 512.5 倍。
- 計算題第 4 題： $2 \times 1 \times 4096 \times 40 \times 8 \times 128 \times 2 = 671 \text{ MB}$ ；併發 32 時約 21.5 GB。
- 計算題第 5 題：批次 1 時 FLOPs = $2 \times 13e9 = 26 \text{ GFLOPs}$ 、搬移 26 GB，強度為 1；批次 64 時強度為 64。
- 計算題第 6 題：期望接受數為 $\sum_{i=1..5} 0.7^i = (1 - 0.7^5) \div (1 - 0.7) - 1 \approx 1.77$ ，加上被拒絕位置重取樣的 1 個，每次約產出 2.77 個詞元。
- 除錯題第 1 題順序：RoPE 的 `pos_offset` 是否正確 > decode 時是否誤用 `is_causal` > `cache.advance` 的時機 > GQA 的 `_repeat_kv` 是否在快取前展開。

研究延伸與版本注意事項

- 核取樣 (nucleus sampling) 的原始論文對「退化」現象有清楚的分析與實證，是理解 14.1.2 與 14.3 節的最佳補充。
- 推測解碼的原始提案：關鍵在於接受與重取樣的設計如何保證分布不變，那段推導值得細讀。
- 分頁注意力 (PagedAttention) 與相關的 KV cache 管理技術：第 35 章會用到，若你的專題涉及多使用者服務可提早了解。
- 文法約束解碼的實作：有數個成熟的開源函式庫提供預編譯的詞元到狀態對照，效率遠高於本章的示範程式碼。
- min-p 取樣是較新的方法，相關的比較研究仍在累積中，引用時請註明來源與時間。
- 本章的演算法穩定，但推論框架的 API 與預設參數變動較快。解碼參數的預設值在不同函式庫之間差異很大，務必明確指定而非依賴預設。

常見問題

問：溫度、top-p、top-k 可以同時用嗎？答：可以，而且常常一起用。標準順序是先套用重複懲罰、再溫度、再截斷、最後取樣（見 14.3.5 節的提醒）。但要注意過度疊加會讓行為難以預測——實務上多數情況只需要溫度加 top-p 兩個旋鈕。

問：我的模型很小，這些解碼技巧有用嗎？答：非常有用，而且相對而言更有用。小模型的機率分布通常較不可靠、長尾較肥，截斷取樣能顯著改善輸出品質。第 13 章的 tiny 模型在 greedy 下幾乎一定會重複，加上 $T = 0.9$ 與 $\text{top-p} = 0.9$ 之後會好很多——這是在本章就能親自驗證的。

問：KV cache 一定要自己實作嗎？答：正式部署時應該使用成熟的推論框架，它們的實作包含分頁管理、批次排程等本章沒有處理的複雜度。但自己寫一次的價值在於：你會知道 `pos_offset`、`is_causal` 這些細節為什麼重要，而那正是使用框架時最容易配置錯誤的地方。

問：為什麼我的 KV cache 沒有帶來預期的加速？答：先檢查三件事。第一，是否真的在使用快取（印出每步實際處理的詞元數，應該是 1）。第二，模型是否太小——小模型的單步計算本來就快，快取的相對效益較低。第三，是否在 CPU 上執行——CPU 的記憶體頻寬與快取行為與 GPU 不同，效益可能不如預期。

教師使用建議

本章排在第 14 週，是第 4 篇的第一章，建議 2 小時講授加 1 小時實驗。本章的教學特色是「立即可見的效果」——每一個參數的改變都能當場看到輸出的差異。

時段	內容	互動設計	注意事項
前 25 分鐘	14.1 至 14.2 節，解碼空間與溫度	現場用同一提示掃描溫度 0.2 到 1.5	這是全章最有感的示範
中間 30 分鐘	14.3 至 14.4 節，截斷與重複懲罰	請學生預測 top-p 會保留幾個，再驗證	用 14.3.2 節的具體數字
後 30 分鐘	14.5 至 14.6 節，KV cache 與兩階段	現場量測有無快取的速度差	加速倍數的震撼效果好
最後 30 分鐘	14.7 至 14.9 節與作業說明	各組討論自己專題的解碼參數表	連結到貫穿式案例
實驗課 1 小時	程式 14A 與 14B	兩人一組，一人做解碼一人做快取	14C 作為課後作業

一個效果極佳的開場示範：用第 13 章訓練好的 tiny 模型，同一個提示、同一個 seed，只改溫度，依序生成 $T = 0.1$ 、 0.5 、 0.9 、 1.3 的輸出並並排投影。學生會看到文字從「單調重複」到「通順」再到「開始胡言亂語」的完整光譜。這比任何公式都能說明溫度在做什麼。

另一個建議：KV cache 的等價性驗證務必在課堂上執行一次。這是本章唯一「必須完全相同」的測試，而學生第一次寫時通過率通常不到一半——原因幾乎都是 pos_offset 或 is_causal。當場示範除錯過程，比事後解答有價值得多。

高中授課的調整：14.1、14.2 完整教（溫度的示範對高中生極有吸引力）；14.3 只講 top-p 的直覺，數學可略；14.4 用「模型卡住了」的比喻講重複；14.5 講「不要重算已經算過的」的直覺與加速倍數，實作可略；14.6 至 14.8 略過；14.9 讓學生為自己的專題設計參數表。程式做 14A 的簡化版，用老師提供的模型跑溫度掃描。

本章小結

1. 解碼是獨立於訓練的設計空間，改變它不需要任何訓練成本——這是資源受限情境下投資報酬率最高的最佳化手段之一。
2. greedy 是確定且短視的，會產生退化輸出；beam search 在開放式生成上因為「搜尋得越好越平淡」而更糟。
3. 溫度縮放不改變詞元排序，只改變相對差距；用熵取指數得到的「有效候選數」是選擇溫度的實用依據。
4. 截斷取樣處理長尾問題：top-k 固定數量、top-p 適應語境、min-p 用相對門檻，三者可組合使

用。

5. top-p 通常優於 top-k，因為模型的確定度隨語境變化，動態的候選數更合理。
6. 過濾器的順序是重複懲罰、溫度、截斷、取樣；top-p 必須保護至少保留一個候選，否則會產生 NaN。
7. 退化有詞元級、短語級與主題級三種；前兩者可用重複懲罰處理，第三種是模型能力問題。
8. 繁體中文的功能詞會被頻率懲罰不成比例地打擊，應建立排除清單。
9. KV cache 把生成的複雜度從 $O(T^2)$ 降到 $O(T)$ ；生成 2,048 個詞元時節省超過一千倍。
10. 使用 RoPE 時快取的是已旋轉的 K，因此 pos_offset 必須正確；decode 階段 ($T=1$) 不應套用因果遮罩。
11. KV cache 的記憶體隨併發數線性成長，往往才是限制併發量的真正瓶頸。
12. prefill 是計算受限的、decode 是頻寬受限的；decode 的運算強度只有 1，換更強的 GPU 幫助有限。
13. 提高批次與量化權重對 decode 的效益遠高於提升算力。
14. 推測解碼用小模型草擬、大模型驗證，在正確設計下輸出分布完全等價，最適合單使用者低延遲的場景。
15. 受限生成能保證格式合法但不保證內容正確；應分層使用提示、驗證重試與文法約束。
16. 解碼參數應依任務分類設計並版本控制，與提示規格一起構成系統的行為定義。

成果檢核

完成本章後你必須繳交：

- 解碼工具箱（程式 14A）：七種策略，通過全部測試，數字與正文表格一致。
- 溫度掃描報告：至少六個溫度值的熵、有效候選數與實際生成樣本。
- KV cache 實作（程式 14B）：等價性驗證通過，並附加速曲線與記憶體量測。
- prefill 與 decode 的時間拆解：對一個典型的提示與生成長度，兩階段各占多少。
- 結構化輸出報告（程式 14C）：三種層次的合規率與平均呼叫次數對照。
- 解碼參數設定檔：至少五個 profile，每個附使用時機與參數理由。
- 一頁分析：你的模型在哪些任務上「解碼調不出來」？那些問題該在哪一章處理？

本章的解碼工具箱與 KV cache 會被第 27 章（多樣本推理）、第 29 章（結構化輸出）、第 30 章（RAG 的生成端）、第 33 章（應用開發）與第 35 章（推論服務）直接使用。從這一章開始，你的

模型不只是「能跑」，而是「能用」——而那個差距，往往就是一個原型與一個產品的距離。

第 15 章

預訓練迴圈、最佳化與訓練穩定性

Pretraining Loops, Optimization, and Training Stability

難度：核心 / 進階 | 建議授課：第 15 週 | 硬體需求：A 級可完成大部分實作；24 小時訓練建議 B 級以上

叫獸開講

訓練跑到第 8000 步突然 loss 變成 NaN，這是每個做大模型的人都會遇到的成年禮。但真正的專業不在於「遇到之後怎麼救」，而在於「在它發生前三百步就看見徵兆」。這一章教你的，是怎麼把訓練從一場賭博變成一件可以被監控、被預測、被回滾的工程。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明批次大小、梯度累積與有效批次的等價關係，並正確實作累積的損失平均。
2. 設計 warmup、cosine 與 WSD 三種學習率排程，並判斷各自的適用情境。
3. 用短跑實驗探測可用的學習率區間，而非憑經驗猜測。
4. 正確實作混合精度訓練，說明 loss scaling 在 FP16 與 BF16 下的差異。
5. 實作梯度裁切並用梯度範數作為訓練健康度的主要指標。
6. 辨識訓練發散的三種徵兆，並在損失爆炸前採取行動。
7. 設計可中斷、可續訓、且資料位置能正確還原的訓練流程。
8. 計算 tokens/sec 與 MFU，並用它們定位效能瓶頸。
9. 完成一次 24 小時不中斷的訓練，全程記錄資源、成本與異常事件。

本章與第 13 章的關係

第 13 章的訓練迴圈是「最小可行」版本——能跑，但沒有排程、沒有混合精度、沒有監控、中斷後只能大致續上。本章把它升級為工業級：跑得久、跑得穩、出事時知道發生了什麼。如果你打算做第 22 章的持續預訓練或第 40 章的總整專題，本章的每一項都會用到。

15.1 批次大小與梯度累積

15.1.1 三種批次

這是一組容易混淆的名詞，先定義清楚。

名稱	定義	受什麼限制	影響什麼
微批次	一次前向傳播處理的樣本數	GPU 記憶體	activation 記憶體
有效批次	一次參數更新所依據的樣本數	無硬性限制	梯度雜訊、收斂行為
全域批次	多卡時所有卡合計的有效批次	卡數 × 有效批次	第 19 章的分散式

$$\text{有效批次} = \text{微批次} \times \text{梯度累積步數} \times \text{資料平行的卡數}$$

關鍵洞見是：有效批次才是影響訓練行為的量，而微批次只是一個記憶體上的妥協。這給了你一個重要的自由度——當記憶體放不下大批次時，可以用累積換取相同的訓練行為。

微批次	累積步數	有效批次	梯度是否等價
32	1	32	基準
8	4	32	幾乎完全等價
4	8	32	同上
2	16	32	同上，但速度較慢

「幾乎完全等價」的保留來自兩個地方：一是跨樣本的統計層（BatchNorm）會有差異，但 Transformer 不用它（第 12.1.2 節）；二是浮點加法的順序不同會有微小的數值差異（第 3.6.1 節）。實務上兩者的訓練曲線幾乎重合。

15.1.2 累積時的損失平均：最常見的錯誤

第 5.1.5 節與第 13.2.1 節都提過這件事，這裡給出完整的實作，因為它在梯度累積下特別容易錯。

錯誤的做法是把每個微批次的平均損失相加再除以累積步數。這只在「每個微批次的有效詞元數相同」時才正確——而在有 padding 或指令微調（第 23 章）時並不成立。

formosa/training/accumulation.py

```
import torch

def train_step_with_accumulation(model, batches, optimizer, scaler=None,
                                grad_clip=1.0):
```

```

"""正確的梯度累積：累加損失總和與詞元數，最後才相除"""
optimizer.zero_grad(set_to_none=True)

total_loss_sum = 0.0
total_tokens = 0

# 先算出這一輪的總詞元數，才能正確縮放每個微批次的梯度
token_counts = [int((y != -100).sum()) for _, y in batches]
denom = sum(token_counts)
assert denom > 0, "整輪沒有任何有效詞元"

for (x, y), n_tok in zip(batches, token_counts):
    logits, loss_sum, _ = model(x, targets=y, reduction="sum")
    # 除以「整輪」的總詞元數，而非這個微批次的
    loss = loss_sum / denom
    if scaler is not None:
        scaler.scale(loss).backward()
    else:
        loss.backward()
    total_loss_sum += float(loss_sum)
    total_tokens += n_tok

if scaler is not None:
    scaler.unscale_(optimizer)          # 裁切前必須先反縮放
    gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)

if scaler is not None:
    scaler.step(optimizer); scaler.update()
else:
    optimizer.step()

return {"loss": total_loss_sum / total_tokens,
        "grad_norm": float(gnorm),
        "tokens": total_tokens}

```

注意 `denom` 是先算好的。這是因為 `backward` 會立刻累加梯度，所以每個微批次在 `backward` 之前就必須知道正確的縮放係數。先掃一遍算 `token` 數的成本很小（只是比較與加總），但能保證梯度精確等價於大批次。

15.1.3 批次大小該設多少

第 3.5.8 節說過梯度雜訊與批次大小的平方根關係。實務上的選擇邏輯是這樣：

考量	偏好大批次	偏好小批次	本書建議
梯度品質	雜訊小、曲線平滑	—	至少讓曲線可讀
硬體效率	矩陣乘法更飽和	—	微批次盡量填滿記憶體
收斂速度	每步進步多但步數少	每步進步少但步數多	有臨界點，超過就浪費

考量	偏好大批次	偏好小批次	本書建議
記憶體	—	放得下	用累積解耦
正則效果	—	雜訊有正則作用	小資料時勿過大

一個實用的起點：微批次設到記憶體剛好放得下（留 10% 餘裕），有效批次設在 0.25 M 到 2 M 個詞元之間。以序列長度 1024 為例，那對應到有效批次 256 到 2048。小模型可以用更小的值。

15.2 學習率排程

15.2.1 為什麼不能用固定學習率

階段	模型狀態	需要的學習率	不調整的後果
初期	參數隨機、梯度方向不可靠	小 (warmup)	容易發散
中期	方向穩定、需要快速前進	大	進步緩慢
後期	接近極小值、需要精修	小	在極小值附近震盪

第一列值得展開。訓練剛開始時，Adam 的二階動量估計（第 3.5.3 節的 v ）還很不準——它只看過幾個批次。此時如果用完整的學習率，更新步伐可能大得離譜。warmup 讓模型有時間累積可靠的動量估計。

Post-LN 架構對此特別敏感（第 12.2.2 節），沒有 warmup 幾乎一定發散。Pre-LN 較寬容，但仍然建議使用。

15.2.2 三種排程

formosa/training/schedule.py

```
import math

def cosine_schedule(step, warmup_steps, total_steps, peak_lr,
                    min_ratio=0.1):
    """最常用：warmup 後餘弦下降到 peak_lr * min_ratio"""
    if step < warmup_steps:
        return peak_lr * step / max(1, warmup_steps)
    progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    progress = min(1.0, progress)
    cos = 0.5 * (1.0 + math.cos(math.pi * progress))
    return peak_lr * (min_ratio + (1 - min_ratio) * cos)
```

```

def wsd_schedule(step, warmup_steps, total_steps, peak_lr,
                 decay_fraction=0.2, min_ratio=0.0):
    """Warmup-Stable-Decay: 中段維持恆定，最後線性衰減
    優點是可以在任何時候延長訓練而不必重新規劃排程"""
    decay_start = int(total_steps * (1 - decay_fraction))
    if step < warmup_steps:
        return peak_lr * step / max(1, warmup_steps)
    if step < decay_start:
        return peak_lr
    progress = (step - decay_start) / max(1, total_steps - decay_start)
    return peak_lr * ((1 - progress) * (1 - min_ratio) + min_ratio)

def constant_schedule(step, warmup_steps, total_steps, peak_lr, **kw):
    """僅 warmup 後保持恆定，用於對照實驗與短期微調"""
    return peak_lr * min(1.0, step / max(1, warmup_steps))

SCHEDULES = {"cosine": cosine_schedule, "wsd": wsd_schedule,
             "constant": constant_schedule}

def apply_lr(optimizer, lr):
    for group in optimizer.param_groups:
        group["lr"] = lr

```

用實際數字看三種排程的差異 (peak = $3e-4$ 、總步數 20,000、warmup 500) :

步數	cosine	WSD	說明
0	0	0	warmup 起點
250	$1.50e-4$	$1.50e-4$	warmup 中
500	$3.00e-4$	$3.00e-4$	達到峰值
2,000	$2.96e-4$	$3.00e-4$	cosine 已開始下降
10,000	$1.70e-4$	$3.00e-4$	差距最大
16,000	$5.71e-5$	$3.00e-4$	WSD 尚未開始衰減
18,000	$3.70e-5$	$1.50e-4$	WSD 衰減中
20,000	$3.00e-5$	0	終點

15.2.3 WSD 的實務優勢

cosine 有一個結構性的問題：它需要在訓練開始前就知道總步數。如果你跑到一半想延長訓練，整條曲線都要重新規劃，而已經跑過的部分無法補救。

WSD 把訓練分成三段：warmup、穩定、衰減。穩定段的學習率恆定，所以可以隨時決定「再跑久一點」，只要在最後保留衰減段即可。這對算力不確定的環境特別有價值——臺灣的實驗室常常是「先申請到 48 小時，看情況再申請」，WSD 讓這種模式可行。

面向	cosine	WSD	判斷
需預知總步數	是	否	WSD 較彈性
可延長訓練	困難	容易	WSD 勝
中途取用檢查點	學習率已降，效果尚可	學習率仍高，需另行衰減	cosine 略優
最終效果	基準	相當或略優	差異不大
與退火資料配合	需對齊	天然對應衰減段	WSD 較自然

最後一列連結到第 9.5.4 節：把最珍貴的高品質本土語料留到訓練最後階段使用。WSD 的衰減段剛好是這個「退火階段」的天然位置——低學習率加高品質資料，兩者一起收尾。

15.2.4 學習率探測

該用多大的峰值學習率？與其猜測或抄別人的，不如用短跑實驗量出來。

scripts/lr_finder.py

```
import math, torch

def lr_range_test(model_fn, data_iter, min_lr=1e-6, max_lr=1e-1,
                  n_steps=200, diverge_factor=4.0):
    """指數掃描學習率，記錄 loss；發散即停"""
    model = model_fn()
    opt = torch.optim.AdamW(model.parameters(), lr=min_lr, betas=(0.9, 0.95))
    mult = (max_lr / min_lr) ** (1 / n_steps)

    lr, best, history = min_lr, float("inf"), []
    for step in range(n_steps):
        for g in opt.param_groups:
            g["lr"] = lr
        x, y = next(data_iter)
        _, loss = model(x, targets=y)
        opt.zero_grad(set_to_none=True)
        loss.backward()
```

```

gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1e9)
opt.step()

l = loss.item()
history.append({"lr": lr, "loss": l, "grad_norm": float(gnorm)})
best = min(best, l)
if math.isnan(l) or l > best * diverge_factor:
    print(f"在 lr={lr:.2e} 發散，停止")
    break
lr *= mult
return history

def suggest_peak_lr(history, safety=0.3):
    """建議值：loss 下降最快處的學習率，再乘一個安全係數"""
    valid = [h for h in history if not math.isnan(h["loss"])]
    slopes = [(valid[i + 1]["loss"] - valid[i]["loss"], valid[i]["lr"])
               for i in range(len(valid) - 1)]
    steepest_lr = min(slopes)[1]
    return steepest_lr * safety

```

判讀方式：把 loss 對 lr (對數座標) 畫出來，你會看到一條「先平坦、再下降、然後暴增」的曲線。最陡的下降處是理論上最有效的學習率，但它太接近發散邊緣，實務上取它的三分之一左右作為峰值。

一個重要的限制

LR range test 是在訓練初期做的，而模型在訓練後期能承受的學習率可能不同。它給的是一個合理的起點，不是最佳解。如果你的訓練在中期發散，即使當初的探測顯示這個學習率安全，仍然應該降低。這也是為什麼梯度範數監控 (15.4 節) 不能省略。

15.3 混合精度訓練

15.3.1 為什麼要混合

第 3.6.5 節講過原則：矩陣乘法與 activation 用低精度 (快、省記憶體)，參數更新用高精度 (避免小更新被吃掉)。這一節處理實作。

項目	FP16 方案	BF16 方案	差異
數值範圍	窄 (易溢位)	同 FP32	BF16 安全得多
尾數精度	10 位	7 位	FP16 較精確
需要 loss scaling	是	否	BF16 實作簡單
硬體支援	較舊的 GPU 也有	需較新的硬體	FP16 相容性廣
本書建議	硬體不支援 BF16 時	首選	—

15.3.2 loss scaling : 只有 FP16 需要

FP16 的數值範圍窄，梯度中的小數值可能下溢成零。loss scaling 的做法是：反向傳播前把損失乘上一個大常數，讓梯度整體放大到 FP16 能表示的範圍；更新前再除回來。

BF16 的指數位元與 FP32 相同，範圍一樣大，不會有這個問題——所以不需要 loss scaling。這是 BF16 在實作上簡單得多的主要原因。

formosa/training/precision.py

```
import torch
from contextlib import nullcontext

def setup_precision(device, precision: str):
    """回傳 (autocast context, scaler)"""
    if device.type != "cuda" or precision == "fp32":
        return nullcontext(), None

    if precision == "bf16":
        assert torch.cuda.is_bf16_supported(), \
            "此硬體不支援 BF16，請改用 fp16 或 fp32"
        return torch.autocast("cuda", dtype=torch.bfloat16), None

    if precision == "fp16":
        # FP16 需要 GradScaler 處理下溢
        return (torch.autocast("cuda", dtype=torch.float16),
                torch.amp.GradScaler("cuda"))

    raise ValueError(f"未知的精度設定：{precision}")

def training_step(model, x, y, optimizer, ctx, scaler, grad_clip=1.0):
    with ctx:
        # autocast 只包住前向
        _, loss = model(x, targets=y)

    if scaler is not None:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        # 裁切前必須反縮放
        gnrm = torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
        scaler.step(optimizer)
        scaler.update()
    else:
        loss.backward()
        gnrm = torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
        optimizer.step()

    optimizer.zero_grad(set_to_none=True)
    return loss.item(), float(gnrm)
```

兩個關鍵細節。第一，autocast 只包住前向傳播與損失計算，不包 backward——PyTorch 會自動讓

反向使用對應的精度。第二，使用 `scaler` 時必須先 `unscale_` 再裁切，否則你裁切的是被放大過的梯度，閾值完全失去意義。這是混合精度最常見的實作錯誤。

15.3.3 哪些運算不該用低精度

運算	應使用的精度	理由	章節
正規化的統計量	FP32	BF16 尾數不足以算平方和	第 12.1.3 節
softmax 與交叉熵	FP32	涉及指數與對數，範圍大	第 3.3.4 節
最佳化器狀態	FP32	累積量，誤差會放大	第 3.5.5 節
損失的加總	FP32	大量小數相加	第 3.6.1 節
矩陣乘法	BF16	這裡最耗時，收益最大	第 18 章

PyTorch 的 `autocast` 對這些運算大多已經有正確的預設處理（例如 `softmax` 會自動提升到 FP32）。但如果你自訂了 `kernel` 或用了非標準的實作，就必須自己確認。本書的 `RMSNorm`（第 12.1.3 節）就是明確地把統計量轉成 `float32` 的例子。

15.4 梯度範數：訓練健康度的主要指標

15.4.1 為什麼看它而不是只看 `loss`

`loss` 是結果，梯度範數是過程。而過程的異常通常比結果的異常早出現數十到數百步。

現象	loss 的表現	梯度範數的表現	提前量
即將發散	仍在正常下降	開始出現尖峰	數十至數百步
資料異常	偶爾小幅上升	單步暴增	立即
學習率過大	震盪或緩慢下降	持續偏高	立即
梯度消失	停滯不動	接近零	立即
正常收斂	穩定下降	緩慢下降並趨於平穩	—

15.4.2 梯度裁切

$$\text{if } \|g\| > c \quad \text{then} \quad g \leftarrow g \times c / \|g\|$$

第 3.6.4 節講過原理。這裡補充幾個實務要點。

- 按整體範數縮放，不是逐元素截斷。前者保持梯度方向，後者會扭曲方向。
- c 通常取 1.0。這個值在多數語言模型訓練中都適用，不太需要調。
- 即使裁切很少觸發，也應該保留它——它的價值在於處理罕見的異常樣本。
- 記錄裁切前的範數，而非裁切後的。後者永遠不會超過 c ，看不出任何資訊。
- 在混合精度下必須先 `unscale_` 再裁切（15.3.2 節）。

第四點是一個常見的錯誤。`clip_grad_norm_` 回傳的是「裁切前」的總範數，所以直接記錄它的回傳值是對的；但如果你在裁切之後才自己計算範數，那個數字永遠是 $\min(\text{原範數}, c)$ ，完全失去監控價值。

15.4.3 尖峰偵測：一個需要修正的做法

第 13 章的訓練哨兵用了「loss 超過近期中位數的三倍」作為尖峰判準。這個做法在實務上不夠敏感，值得修正。

原因是：一段穩定訓練的 loss 標準差可能只有 0.05 左右（在 3.0 附近波動）。要達到中位數的三倍就是 9.0——那已經是完全發散了，不是「尖峰」。等到那時候才告警，來不及做任何事。

判準	在 $\text{loss} \approx 3.0$ 、 $\sigma \approx 0.05$ 時	敏感度	建議用途
中位數 $\times 3$	門檻 9.0	極不敏感	不建議用於 loss
絕對增量 > 0.5	門檻 3.5	合理	loss 的主要判準
IQR 標準化 $z > 5$	依實際分布	自適應	較穩健
梯度範數 $\times 3$	對梯度合理	敏感	梯度的主要判準

結論是：loss 與梯度範數應該用不同的判準。loss 適合用絕對增量或穩健的 z 分數；梯度範數因為本身變異大、且尖峰是乘性的，倍數判準才合適。

formosa/training/sentinel.py — 修正版哨兵

```
import math
from collections import deque
```

```

class TrainingSentinel:
    """修正第 13 章的版本：loss 與梯度用不同判準"""

    def __init__(self, vocab_size, window=100,
                  loss_jump=0.5, gnorm_factor=3.0, z_threshold=5.0):
        self.expected_init = math.log(vocab_size)
        self.losses = deque(maxlen=window)
        self.gnorms = deque(maxlen=window)
        self.loss_jump = loss_jump
        self.gnorm_factor = gnorm_factor
        self.z_threshold = z_threshold
        self.events = []

    @staticmethod
    def _robust_z(x, values):
        s = sorted(values)
        n = len(s)
        med = s[n // 2]
        iqr = s[int(n * 0.75)] - s[int(n * 0.25)]
        return abs(x - med) / max(iqr, 1e-8)

    def step(self, step, loss, grad_norm, lr=None):
        msgs = []

        if math.isnan(loss) or math.isinf(loss):
            msgs.append(("critical", "loss 為 NaN/Inf，應立即回滾"))
        elif len(self.losses) >= 20:
            med = sorted(self.losses)[len(self.losses) // 2]
            if loss - med > self.loss_jump:
                msgs.append(("warn",
                             f"loss 尖峰：{loss:.4f} 高於中位數 {med:.4f}"))
            if self._robust_z(loss, self.losses) > self.z_threshold:
                msgs.append(("warn", "loss 偏離近期分布（穩健 z 分數）"))

        if len(self.gnorms) >= 20:
            gmed = sorted(self.gnorms)[len(self.gnorms) // 2]
            if grad_norm > gmed * self.gnorm_factor:
                msgs.append(("warn",
                             f"梯度尖峰：{grad_norm:.2f} 為中位數 "
                             f"{gmed:.2f} 的 {grad_norm/gmed:.1f} 倍"))
        if grad_norm == 0.0:
            msgs.append(("critical", "梯度範數為 0：參數未被最佳化器追蹤"))

        self.losses.append(loss)
        self.gnorms.append(grad_norm)
        for level, m in msgs:
            self.events.append({"step": step, "level": level, "msg": m,
                               "loss": loss, "grad_norm": grad_norm, "lr": lr})
            print(f"[哨兵 {level}] step {step}] {m}")
        return msgs

```

15.4.4 發散的三種徵兆

徵兆	觀察到什麼	通常的成因	處置
梯度範數階梯式上升	每隔一段時間跳上一階	學習率偏高、累積的不穩	降學習率、延長 warmup
梯度範數單點暴增	單一步驟數十倍	異常樣本（超長、亂碼、重複）	檢查該批次、加強資料過濾
activation 統計漂移	深層輸出的標準差持續放大	初始化或正規化問題	檢查殘差縮放、final norm

第二列的處置值得強調：發現梯度暴增時，把那個批次的實際內容印出來解碼看看。第 9 章的清理管線再嚴謹，仍可能漏掉一些極端樣本，而它們往往就是尖峰的來源。這是一個「訓練告訴你資料有問題」的例子——監控不只保護訓練，也在回頭檢驗資料。

15.5 可中斷、可續訓的訓練

15.5.1 續訓要還原什麼

第 13.4.2 節的 checkpoint 存了模型與最佳化器狀態，那還不夠。要讓續訓的結果與「從未中斷」完全一致，必須還原六樣東西。

項目	不還原的後果	如何還原	難度
模型參數	完全失去進度	state_dict	低
最佳化器狀態	動量歸零，loss 會跳升	optimizer state_dict	低
步數	學習率排程錯位	存入 step	低
隨機狀態	資料順序與 dropout 不同	存 RNG state	中
資料位置	重複看過的資料或跳過部分	存 sampler 狀態	中高
哨兵歷史	尖峰偵測的基準遺失	存 sentinel 的 deque	低

第五列是最容易被忽略、也最容易造成隱形問題的一項。如果續訓後資料從頭開始，模型會重複看一遍已經看過的內容——這相當於局部的多輪訓練，會增加記憶化風險（第 8.4.5 節）而你不會察覺。

15.5.2 完整實作

formosa/training/checkpoint.py — 完整續訓支援

```

import random
import numpy as np
import torch
from dataclasses import asdict

FORMAT_VERSION = 2          # 第 13 章為 1，本章擴充後升版

def capture_rng_state(device):
    state = {
        "python": random.getstate(),
        "numpy": np.random.get_state(),
        "torch": torch.get_rng_state(),
    }
    if device.type == "cuda":
        state["cuda"] = torch.cuda.get_rng_state_all()
    return state

def restore_rng_state(state, device):
    random.setstate(state["python"])
    np.random.set_state(state["numpy"])
    torch.set_rng_state(state["torch"])
    if device.type == "cuda" and "cuda" in state:
        torch.cuda.set_rng_state_all(state["cuda"])

def save_full(path, model, optimizer, step, mcfg, tcfg,
              data_position, sentinel, device, fingerprint=None):
    torch.save({
        "format_version": FORMAT_VERSION,
        "model_state": model.state_dict(),
        "optim_state": optimizer.state_dict(),
        "step": step,
        "model_config": asdict(mcfg),
        "train_config": asdict(tcfg),
        "rng_state": capture_rng_state(device),
        "data_position": data_position,      # 已消耗的樣本索引
        "sentinel_history": {
            "losses": list(sentinel.losses),
            "gnorms": list(sentinel.gnorms),
            "events": sentinel.events[-100:],
        },
        "fingerprint": fingerprint,
        "torch_version": torch.__version__,
    }, path)

def load_full(path, model, optimizer, sentinel, device):
    ck = torch.load(path, map_location=device, weights_only=False)
    assert ck["format_version"] >= 2, "舊格式不含續訓所需的完整狀態"

    # 設定一致性檢查（延續第 13.4.2 節）
    saved = ck["model_config"]
    for key in ("d_model", "n_layers", "n_heads", "vocab_size",
               "activation", "norm_type", "pos_encoding", "tie_weights"):
        if saved.get(key) != getattr(model.cfg, key):
            raise ValueError(
                f"設定不符: {key} ck={saved.get(key)} 目前={getattr(model.cfg, key)}")

```

```

model.load_state_dict(ck["model_state"])
optimizer.load_state_dict(ck["optim_state"])
restore_rng_state(ck["rng_state"], device)
hist = ck.get("sentinel_history", {})
sentinel.losses.extend(hist.get("losses", []))
sentinel.gnorms.extend(hist.get("gnorms", []))
return ck["step"], ck.get("data_position", 0)

```

15.5.3 資料位置的還原

要讓資料從正確的位置繼續，最簡單可靠的做法是使用確定性的取樣順序，並記錄已消耗的樣本數。

formosa/data/resumable.py

```

import torch
from torch.utils.data import Sampler

class ResumableSampler(Sampler):
    """可續訓的取樣器：順序由 (seed, epoch) 決定，可跳過前 N 個"""

    def __init__(self, dataset_size: int, seed: int = 42,
                 shuffle: bool = True):
        self.n = dataset_size
        self.seed = seed
        self.shuffle = shuffle
        self.consumed = 0

    def set_position(self, consumed: int):
        self.consumed = consumed

    def __iter__(self):
        epoch = self.consumed // self.n
        offset = self.consumed % self.n
        while True:
            g = torch.Generator().manual_seed(self.seed + epoch)
            order = (torch.randperm(self.n, generator=g) if self.shuffle
                    else torch.arange(self.n))
            for idx in order[offset:].tolist():
                yield idx
            epoch += 1
            offset = 0

    def __len__(self):
        return self.n

```

這個設計的關鍵是：第 k 個 epoch 的順序完全由 $\text{seed} + k$ 決定，與先前的執行歷史無關。所以只要知道「已經消耗了幾個樣本」，就能算出應該從哪個 epoch 的哪個位置繼續。這比儲存整個排列順序簡單得多，也不會讓 checkpoint 膨脹。

一個必做的驗證

續訓的正確性測試：跑 100 步存檔、再跑到 200 步記下 loss 序列；然後從 100 步的 checkpoint 續訓到 200

步，兩次的 loss 序列應該完全相同（CPU 上）或差異在浮點誤差內（GPU 上）。這是程式 15A 的第一項驗收，也是第 13 章程式 13A 那項驗收的嚴格版本。

15.6 吞吐量與 MFU

15.6.1 三個效能指標

指標	定義	用途	章節
tokens/sec	每秒處理的詞元數	最直接的進度指標	本節
MFU	實際 FLOPs ÷ 硬體峰值 FLOPs	衡量硬體利用效率	本節
step time	每步的牆鐘時間	定位瓶頸	第 18 章

$$MFU = 6 \times \text{參數量} \times (\text{tokens/sec}) \div \text{硬體峰值 FLOPs}$$

這個公式來自第 3.2.5 節的 6ND 估算。分子是「你實際做了多少有用的計算」，分母是「硬體理論上能做多少」。

15.6.2 MFU 的合理範圍

情況	典型 MFU	判讀	該做什麼
大模型、長序列、最佳化良好	0.4 至 0.55	相當好	維持
中型模型、一般設定	0.25 至 0.4	正常	可再最佳化
小模型或短序列	0.1 至 0.25	常見	未必值得最佳化
明顯偏低	低於 0.1	有瓶頸	用 profiler 找（第 18 章）

為什麼小模型的 MFU 天然較低？因為每一步的計算量小，而固定開銷（kernel launch、資料搬移、Python 迴圈）的占比就相對高。這是第 4 章程式 4B 就觀察到的現象——矩陣越大 TFLOPS 越高。

15.6.3 常見的吞吐量瓶頸

瓶頸	徵兆	診斷方式	處置
資料載入	GPU 使用率忽高忽低	看 DataLoader 的等待時間	增加 num_workers、預先編碼
CPU 前處理	同上	把前處理搬到離線	第 13.3.1 節的 .bin
批次太小	GPU 使用率穩定但偏低	加大微批次看是否改善	調整批次或用累積
未使用融合核心	注意力特別慢	確認 scaled_dot_product_attention 生效	第 11.5.4 節
頻繁存檔	週期性的停頓	看 save_every 的間隔	降低頻率或非同步存檔
同步點過多	每步都有等待	減少 .item() 呼叫	批次記錄而非每步

最後一列是一個容易忽略的問題：每次呼叫 `loss.item()` 都會強制 GPU 與 CPU 同步，打斷執行管線。如果你在每一步都記錄好幾個指標，累積起來的開銷可觀。實務上的做法是：把要記錄的量先累積在 GPU 上的 tensor 中，每 N 步才搬回 CPU 一次。

15.6.4 activation 記憶體與 checkpointing

訓練的記憶體由四部分組成，第 3.5.5 節處理了前三項（參數、梯度、最佳化器狀態），第四項是 activation——反向傳播所需的中間值。

設定 (base 模型)	約略 activation	相對	說明
B=8, T=512	1.66 GB	1	基準
B=8, T=1024	4.53 GB	2.7	超線性 (注意力矩陣)
B=4, T=2048	6.95 GB	4.2	同上
B=2, T=4096	11.78 GB	7.1	同上

(粗估值，含注意力矩陣與各層中間值，BF16，未使用 checkpointing。實際數字請用第 18 章的方法量測。)

activation checkpointing 的做法是：不保留所有中間值，只保留每隔幾層的檢查點，反向傳播時重新計算中間的部分。

策略	記憶體	計算成本	適用
不使用	$O(L)$	基準	記憶體充足時
每層都存	$O(L)$	基準	同上
每 $\sqrt{L}$ 層存一個	$O(\sqrt{L})$	約 +33%	常用的折衷
完全不存	$O(1)$	約 +33% 至更多	極度受限時

「約 +33%」的來源是：反向傳播原本的成本約為前向的兩倍，加上重新計算一次前向，總成本從 3 變成 4，增加三分之一。用三分之一的時間換取數倍的記憶體，在多數情況下是划算的——特別是當它讓你能用更大的批次時，整體吞吐量甚至可能提升。第 18 章會完整處理。

15.7 完整的訓練腳本

15.7.1 把前六節接起來

scripts/train_full.py (主迴圈)

```
def train(mcfg, tcfg, paths, resume=None):
    set_seed(tcfg.seed)
    device = get_device()
    log = setup_logging(paths.out, tcfg.run_name)
    metrics = JsonlMetricLogger(paths.out / "metrics.jsonl")

    model = MiniFormosaGPT(mcfg).to(device)
    opt = torch.optim.AdamW(build_param_groups(model, tcfg.weight_decay),
                             lr=tcfg.peak_lr, betas=(0.9, 0.95), eps=1e-8)
    ctx, scaler = setup_precision(device, tcfg.precision)
    sentinel = TrainingSentinel(mcfg.vocab_size)
    sched = SCHEDULES[tcfg.schedule]

    sampler = ResumableSampler(len(train_ds), seed=tcfg.seed)
    step, consumed = 0, 0
    if resume:
        step, consumed = load_full(resume, model, opt, sentinel, device)
        sampler.set_position(consumed)
        log.info(f"從第 {step} 步、已消耗 {consumed:,} 個樣本續訓")

    loader = DataLoader(train_ds, batch_size=tcfg.micro_batch,
                        sampler=sampler, drop_last=True,
                        num_workers=tcfg.num_workers,
                        pin_memory=(device.type == "cuda"))
    it = iter(loader)

    t0, tokens_seen = time.perf_counter(), 0
    model.train()

    while step < tcfg.max_steps:
        lr = sched(step, tcfg.warmup_steps, tcfg.max_steps, tcfg.peak_lr)
```

```

apply_lr(opt, lr)

# 收集一輪梯度累積所需的微批次
batches = []
for _ in range(tcfg.grad_accum):
    x, y = next(it)
    batches.append((x.to(device, non_blocking=True),
                    y.to(device, non_blocking=True)))
    consumed += x.size(0)

stats = train_step_with_accumulation(
    model, batches, opt, scaler, tcfg.grad_clip)
step += 1
tokens_seen += stats["tokens"]

events = sentinel.step(step, stats["loss"], stats["grad_norm"], lr)
if any(lv == "critical" for lv, _ in events):
    log.error("偵測到嚴重異常，嘗試回滾至前一個檢查點")
    if rollback(paths.out, model, opt, sentinel, device):
        step, consumed = _last_ckpt_state(paths.out)
        sampler.set_position(consumed)
        it = iter(loader)
        apply_lr(opt, lr * tcfg.rollback_lr_factor)  # 降學習率重試
        continue
    raise RuntimeError("回滾失敗，訓練中止")

if step % tcfg.log_every == 0:
    elapsed = time.perf_counter() - t0
    tps = tokens_seen / elapsed
    mfu = 6 * n_params * tps / (tcfg.peak_tflops * 1e12)
    metrics.log(step, loss=stats["loss"], lr=lr,
                grad_norm=stats["grad_norm"],
                tokens_per_sec=round(tps), mfu=round(mfu, 4))

if step % tcfg.save_every == 0:
    save_full(paths.out / f"step{step:07d}.pt", model, opt, step,
              mcfg, tcfg, consumed, sentinel, device)

```

注意 critical 事件的處理：偵測到 NaN 時自動回滾到前一個 checkpoint，並降低學習率重試。這比讓訓練直接崩潰要好得多——特別是在無人值守的長時間訓練中，你可能要八小時後才會發現它掛了。

15.7.2 回滾策略

觸發條件	動作	重試次數上限	仍失敗時
loss 為 NaN	回滾 + 學習率 $\times 0.5$	3	中止並告警
梯度範數暴增 10 倍以上	跳過該步（不更新）	不限	記錄該批次內容
連續 N 步 loss 上升	回滾 + 降學習率	2	中止並人工檢視

第二列的「跳過該步」是一個溫和的處置：既然梯度暴增很可能來自單一異常樣本，那就不要用這一步的梯度更新，直接繼續。這比回滾便宜得多，而且往往就足夠了。同時把該批次的內容記錄下來，事後可以回頭檢查資料。

程式實作

本章三個實作把第 13 章的訓練迴圈升級為可長時間執行的工業版本。完成之後，你就有能力進行第 22 章的持續預訓練。

程式 15A 完整預訓練腳本

目標：整合排程、梯度累積、混合精度、裁切、續訓與監控，並通過六項驗收。

驗收項	測試方式	通過標準	節次
續訓一致性	中斷後續訓，比對 loss 序列	CPU 上完全相同	15.5.3
累積等價性	B=32 vs B=8×accum4	梯度差異小於 1e-5	15.1.2
排程正確	印出關鍵步數的學習率	與 15.2.2 節的表格一致	15.2.2
混合精度	BF16 與 FP32 的 loss 曲線	差異在合理範圍內	15.3
裁切生效	人為製造大梯度	裁切後範數等於 c	15.4.2
哨兵告警	注入 NaN	立即偵測並回滾	15.4.3

tests/test_training.py (節錄)

```
import torch, math, copy

def test_accumulation_equivalence(model_fn, batch_data):
    """B=32 一次 vs B=8 累積 4 次，梯度必須幾乎相同"""
    torch.manual_seed(0)
    m1 = model_fn()
    m2 = copy.deepcopy(m1)

    x, y = batch_data # (32, T)
    # 方案一：一次大批次
    _, loss = m1(x, targets=y)
    loss.backward()
    g1 = [p.grad.clone() for p in m1.parameters() if p.grad is not None]

    # 方案二：4 個微批次累積
    opt2 = torch.optim.SGD(m2.parameters(), lr=0.0)
    batches = [(x[i*8:(i+1)*8], y[i*8:(i+1)*8]) for i in range(4)]
    train_step_with_accumulation(m2, batches, opt2, None, grad_clip=1e9)
    g2 = [p.grad.clone() for p in m2.parameters() if p.grad is not None]

    for a, b in zip(g1, g2):
        assert torch.allclose(a, b, atol=1e-5), (a - b).abs().max()
```

```
def test_resume_exactness(tmp_path, model_fn, tcfg):
    """續訓後的 loss 序列必須與未中斷時相同"""
    ref = run_training(model_fn, tcfg, steps=200, out=tmp_path / "ref")
    run_training(model_fn, tcfg, steps=100, out=tmp_path / "a")
    resumed = run_training(model_fn, tcfg, steps=200,
                           out=tmp_path / "a",
                           resume=tmp_path / "a" / "step0000100.pt")
    for s in range(100, 200):
        assert abs(ref[s] - resumed[s]) < 1e-6, (s, ref[s], resumed[s])

def test_grad_clip_effective(model_fn):
    m = model_fn()
    for p in m.parameters():
        p.grad = torch.ones_like(p) * 100.0      # 人為製造大梯度
    gnorm = torch.nn.utils.clip_grad_norm_(m.parameters(), 1.0)
    after = math.sqrt(sum(float((p.grad ** 2).sum()) for p in m.parameters()))
    assert gnorm > 1.0, "回傳值應為裁切前的範數"
    assert abs(after - 1.0) < 1e-4, "裁切後範數應等於閾值"
```

第三個測試同時驗證了 15.4.2 節的兩個要點：clip_grad_norm_ 回傳的是裁切前的範數（所以記錄它是對的），而裁切後的實際範數等於閾值。

程式 15B 學習率探測器

目標：實作 LR range test，在你自己的模型與資料上找出可用的學習率區間，而非抄別人的數字。

必須產出的四項：

- loss 對 lr 的曲線（對數座標），標出下降最陡處與發散點。
- 梯度範數對 lr 的曲線。它通常在發散前就開始異常，可以作為更早的訊號。
- 建議的峰值學習率，以及你採用的安全係數與理由。
- 對三種模型尺寸（tiny、small、base）各做一次，比較建議值。模型越大，通常可用的學習率越小——把這個趨勢畫出來。

一個值得驗證的經驗法則

文獻中常提到「學習率應隨模型寬度縮小」的關係。用你的三種尺寸實測一次，看看是否符合。即使結果與文獻不同也值得報告——那可能反映了你的資料或設定的特性。第 17 章的尺度定律會用到類似的外推思路。

程式 15C 訓練哨兵與自動回滾

目標：把 15.4.3 節的哨兵接進訓練迴圈，並實作自動回滾。這是第 13 章程式 13C 的完整版本。

formosa/training/rollback.py

```
from pathlib import Path
import torch
```

```

def find_last_checkpoint(out_dir: Path, before_step: int | None = None):
    cks = sorted(out_dir.glob("step*.pt"))
    if before_step is not None:
        cks = [c for c in cks
                if int(c.stem.replace("step", "")) < before_step]
    return cks[-1] if cks else None

def rollback(out_dir, model, optimizer, sentinel, device,
             before_step=None):
    """回滾到前一個檢查點，回傳是否成功"""
    ck_path = find_last_checkpoint(out_dir, before_step)
    if ck_path is None:
        return False
    step, consumed = load_full(ck_path, model, optimizer, sentinel, device)
    sentinel.events.append({
        "step": step, "level": "info",
        "msg": f"已回滾至 {ck_path.name} (step={step}) ",
    })
    print(f"[回滾] 已還原至 {ck_path.name}")
    return True

def dump_bad_batch(out_dir: Path, step: int, x, y, tok):
    """把造成異常的批次解碼存檔，供事後檢查資料（15.4.4 節）"""
    path = out_dir / f"bad_batch_step{step}.txt"
    with path.open("w", encoding="utf-8") as f:
        for i in range(min(4, x.size(0))):
            f.write(f"--- sample {i} ---\n")
            f.write(tok.decode(x[i].tolist())[:1000] + "\n\n")
    print(f"[哨兵] 異常批次已存至 {path}")

```

`dump_bad_batch` 是一個常被忽略但極有價值的工具。當梯度暴增時，把那個批次解碼存下來，你往往會看到一段亂碼、一段極長的重複、或某種第 9 章的清理管線漏掉的異常內容。訓練異常於是變成了資料品質的回饋。

系統案例：24 小時不中斷訓練

本章的系統案例是完成一次 24 小時（或你的硬體條件允許的最長時間）的連續訓練，全程記錄資源、成本與異常事件，並產出一份可供他人重現的報告。

事前準備清單

項次	準備什麼	為什麼	節次
1	用 LR range test 決定峰值學習率	避免用猜的	15.2.4
2	確認記憶體餘裕至少 10%	避免中途 OOM	15.6.4

項次	準備什麼	為什麼	節次
3	設定 <code>save_every</code> 使存檔間隔約 30 分鐘	中斷時損失可控	15.5
4	確認磁碟空間足夠存放所有 checkpoint	第 4.9.1 節的常見問題	第 4 章
5	測試回滾機制確實可用	出事時才發現壞掉最糟	15.7.2
6	記錄環境指紋並確認 git 非 dirty	正式實驗的前提	第 4.7.3 節
7	規劃電力與散熱（若為個人設備）	長時間滿載的實際問題	第 21 章

全程要記錄的內容

類別	內容	頻率	用途
訓練指標	loss、grad_norm、lr、tokens/sec、MFU	每 50 步	曲線與診斷
評估	驗證 loss、分域 PPL	每 1,000 步	泛化追蹤
生成樣本	固定探針提示的輸出	每 2,000 步	質性進展
資源	GPU 使用率、記憶體、溫度、功耗	每 5 分鐘	效能與成本
事件	哨兵告警、回滾、OOM、中斷	發生時	異常紀錄

第四列的功耗記錄連結到第 21 章：把訓練的 GPU 小時換算成電力與碳排，是臺灣算力議題的一部分。即使你的訓練規模不大，養成記錄的習慣，之後做算力預算（第 17 章）時就有真實的參考基準。

交付要求

- 完整的訓練報告：含所有指標曲線、事件時間軸、以及每一個異常的處置說明。
- 資源與成本表：總 GPU 小時、平均功耗、估計電費、以及若租用雲端的等價成本。
- MFU 分析：實測值、與 15.6.2 節的合理範圍對照、以及你認為的主要瓶頸。
- 至少一次的中斷與續訓紀錄：證明你的續訓機制在真實情況下有效。

- 異常事件復盤：如果完全沒有異常，請說明你採取了哪些預防措施；如果有，說明如何發現與處置。
- 一頁反思：如果重跑一次，你會改變哪三個設定？為什麼？

評分重點

- 續訓一致性是否真的驗證過（不是「應該可以」而是「實測過」）。
- 異常處置的紀錄品質。完全沒有異常的報告會被特別檢視監控是否確實運作。
- MFU 是否被計算並解讀，而非只報告 tokens/sec。
- 資源記錄是否完整，特別是功耗與成本的部分。

章末評量

一、概念題

1. 說明微批次、有效批次與全域批次的差異，以及為什麼「有效批次」才是影響訓練行為的量。
2. 梯度累積時為什麼不能把各微批次的平均損失再平均？什麼情況下這個錯誤特別嚴重？
3. 為什麼訓練初期需要 warmup？Adam 的哪個部分與此有關？
4. 比較 cosine 與 WSD 排程，說明 WSD 在算力不確定的環境下有什麼優勢。
5. 為什麼 BF16 不需要 loss scaling 而 FP16 需要？
6. 使用 GradScaler 時，為什麼必須先 unscale_ 再做梯度裁切？
7. 為什麼梯度範數比 loss 更適合作為訓練健康度的早期指標？
8. 為什麼「loss 超過中位數三倍」不是好的尖峰判準？應該用什麼替代？
9. 續訓需要還原哪六樣東西？漏掉「資料位置」會造成什麼隱形問題？

二、計算題

1. 微批次 4、累積 8 步、4 張卡做資料平行。有效批次與全域批次各是多少？若序列長度 1024，每次更新看過多少詞元？
2. 峰值學習率 $3e-4$ 、warmup 500 步、總步數 20,000。用 cosine 排程 ($\text{min_ratio}=0.1$) 計算第 250、500、10,000、20,000 步的學習率。
3. 一個 110 M 參數的模型在峰值 30 TFLOPS 的 GPU 上達到 15,000 tokens/sec。計算 MFU。若最佳化後達到 45,000 tokens/sec 呢？
4. 訓練 42 M 參數的模型、842 M 詞元。若 MFU 為 0.30、GPU 峰值 30 TFLOPS，需要多少小時？若 MFU 提升到 0.45 呢？
5. 一段穩定訓練的 loss 在 3.0 附近波動，標準差 0.05。用「中位數三倍」與「絕對增量 0.5」兩種判準，各自的告警門檻是多少？哪一個合理？
6. 每 30 分鐘存一次 checkpoint，模型 110 M 參數，checkpoint 含參數、梯度與最佳化器狀態 (FP32)。跑 24 小時共需多少磁碟空間？

三、除錯題

1. 續訓後 loss 從 3.2 跳到 4.1，然後慢慢降回來。請列出三個可能原因與檢查順序。
2. 訓練在第 3,000 步 loss 變成 NaN。回滾後降學習率重試，又在第 3,000 步 NaN。請說明這代表什麼，以及該檢查什麼。

3. 使用 FP16 混合精度時，梯度裁切似乎沒有作用——範數仍然很大。請指出原因。
4. 梯度累積實作後，訓練曲線與大批次版本明顯不同。請列出三個可能原因。
5. MFU 只有 0.05，但 GPU 使用率顯示 95%。請說明這兩者為何不矛盾，以及可能的瓶頸。
6. 訓練跑得很順，但每 2,000 步會有一次明顯的停頓。請指出最可能的原因與處置。

四、系統設計題

1. 你申請到國網中心 48 小時的單卡使用權，但不確定之後能否續申請。請設計你的訓練方案：排程選擇、存檔策略、以及如何確保「48 小時後就算不能續跑，也有一個可用的模型」。
2. 你的實驗室要建立一套「無人值守訓練」的標準流程，讓學生可以在週五下班前啟動、週一回來收成果。請設計這套流程：需要哪些自動化、哪些告警、失敗時如何降級、以及如何避免浪費整個週末的算力。

五、臺灣情境與倫理題

1. 第 21 章會處理臺灣的算力議題。請以本章的 24 小時訓練為基礎，估算它的電力消耗與碳排放，並討論：在什麼規模以上，一個研究團隊應該把環境成本納入實驗規劃？
2. 共用 GPU 的環境中，一個學生的訓練崩潰但沒有自動釋放資源，可能占用資源數小時。請設計一套機制（技術與制度並用）來降低這種浪費，並說明哪些部分需要制度而非技術解決。

評量提示（給自學者）

- 計算題第 1 題：有效批次 32、全域批次 128；每次更新看過 $128 \times 1024 = 131,072$ 個詞元。
- 計算題第 2 題：250 步為 $1.50e-4$ ；500 步為 $3.00e-4$ ；10,000 步約 $1.70e-4$ ；20,000 步為 $3.00e-5$ 。
- 計算題第 3 題： $6 \times 1.1e8 \times 15,000 \div 30e12 \approx 0.33$ ；45,000 tokens/sec 時約 0.99（已接近理論上限，實務上不可能，代表峰值或參數估計有誤，值得回頭檢查）。
- 計算題第 4 題： $6 \times 42e6 \times 842e6 \div (30e12 \times 0.30) \div 3600 \approx 6.55$ 小時；MFU 0.45 時約 4.37 小時。
- 計算題第 5 題：中位數三倍為 9.0（等於已完全發散，來不及）；絕對增量 0.5 為 3.5（約十個標準差，合理）。
- 計算題第 6 題：每份約 $110e6 \times 12 \text{ bytes} \approx 1.32 \text{ GB}$ （參數 $4 + m^4 + v^4$ ）；24 小時存 48 份約 63.4 GB。
- 除錯題第 3 題：忘記在 clip_grad_norm_ 之前呼叫 scaler.unscale_()，所以裁切的是被放大過的

梯度。

研究延伸與版本注意事項

- WSD 排程的相關研究：它在近年的開放模型中越來越常見，論證與實證都值得一讀，與第 9.5.4 節的退火資料策略直接相關。
- 大批次訓練與臨界批次大小：關於「批次多大之後就不再有效益」有明確的理論與實證，第 17 章的算力規劃會用到。
- 訓練不穩定性的實證研究：關於 loss spike 的成因（資料異常、注意力 logit 爆炸、深度相關的不穩）有多篇分析，對 15.4.4 節有直接補充。
- 各大開放模型的技術報告中，訓練細節（排程、批次、裁切、發散處置）通常是最有價值的部分，比架構描述更值得細讀。
- PyTorch 的 AMP 與 GradScaler 文件：本章 15.3 節的實作以它為準，API 在版本間有變動，請以課程倉庫的版本為準。

常見問題

問：我的模型很小，需要這麼複雜的訓練基礎設施嗎？答：續訓、監控與資料位置還原這三項無論規模都建議做——它們保護的是你的時間與結論的可信度。混合精度與 checkpointing 在小模型上效益有限，可以省略。判準是：如果一次訓練超過一小時，就值得把這些做好。

問：學習率該用文獻上的數字還是自己探測？答：自己探測。文獻的數字是在他們的模型、資料與批次大小下得到的，直接套用未必適合。LR range test 只需要幾分鐘，而錯誤的學習率可能浪費你好幾小時。

問：訓練發散一定要回滾嗎？答：不一定。如果是單步的梯度暴增，跳過那一步通常就夠了（15.7.2 節）。回滾的成本較高（損失了一段進度），應該保留給真正的發散——loss 已經變成 NaN 或連續多步惡化。

問：MFU 太低該優先處理什麼？答：先確認不是資料載入的瓶頸（最常見也最好修）。用一個簡單的方法判斷：把資料換成隨機產生的假 tensor 再跑一次，如果速度大幅提升，瓶頸就在資料管線。第 18 章會教你用 profiler 做更精確的診斷。

教師使用建議

本章排在第 15 週，建議 2 小時講授加 1 小時實驗。本章的內容較為工程導向，教學重點是「讓學生

相信這些基礎設施值得投資」。

時段	內容	互動設計	注意事項
前 25 分鐘	15.1 至 15.2 節，批次與排程	現場畫出三種排程的曲線	WSD 的彈性優勢要強調
中間 25 分鐘	15.3 節，混合精度	示範忘記 <code>unscale_</code> 的後果	這個錯誤很常見
後 30 分鐘	15.4 節，梯度範數與哨兵	展示一次真實的 <code>loss spike</code> 紀錄	梯度先於 <code>loss</code> 異常要看到
最後 35 分鐘	15.5 至 15.7 節與作業說明	現場示範中斷後續訓的一致性驗證	這是本章最實用的一段
實驗課 1 小時	程式 15A 與 15B	兩人一組，互相中斷對方的訓練	互相中斷的設計很有效

一個效果很好的示範：事先跑一次會發散的訓練（把學習率設高），把 `loss` 與梯度範數兩條曲線疊在同一張圖上。學生會看到梯度範數在 `loss` 還很正常時就開始出現尖峰，而且提前了數百步。這比任何說明都能讓他們理解為什麼要監控梯度。

另一個建議：實驗課讓兩人一組互相中斷對方的訓練（直接 `Ctrl+C`），然後續訓並驗證 `loss` 序列一致。這個設計有兩個好處：一是模擬真實的意外中斷，二是讓學生無法作弊（不能挑一個「好中斷」的時機）。通過率通常不高，而失敗的原因幾乎都是資料位置或 RNG 狀態沒還原。

高中授課的調整：15.1 只講「記憶體放不下就分幾次算」的直覺；15.2 用圖解講三種排程，公式可略；15.3 略過（除非學生有 GPU）；15.4 完整教，梯度範數的監控概念對高中生完全可懂，而且那個「先於 `loss` 異常」的示範很有說服力；15.5 講「存檔與續訓」的必要性，實作可用老師提供的版本；15.6、15.7 挑重點。

本章小結

1. 有效批次等於微批次乘以累積步數乘以卡數；微批次只是記憶體妥協，有效批次才影響訓練行為。
2. 梯度累積必須以整輪的總詞元數為分母，先掃一遍計數再逐批 `backward`，否則梯度不等價。
3. `warmup` 讓 Adam 的二階動量累積可靠的估計，Post-LN 架構沒有它幾乎一定發散。
4. `cosine` 需預知總步數；WSD 的穩定段可隨時延長，且衰減段天然對應退火資料策略。
5. 學習率應以 LR range test 探測，取最陡下降處乘以約三分之一的安全係數。
6. BF16 因指數範圍與 FP32 相同而不需要 `loss scaling`；FP16 需要 `GradScaler`。
7. 使用 `GradScaler` 時必須先 `unscale_` 再裁切，否則裁切的是被放大過的梯度。

8. 正規化統計量、softmax、最佳化器狀態與損失加總都應保持 FP32。
9. 梯度範數比 loss 更早反映異常，提前量可達數十至數百步；應記錄裁切前的範數。
10. 「中位數三倍」對 loss 過於不敏感（3.0 附近的門檻會是 9.0）；loss 宜用絕對增量或穩健 z 分數，梯度才適合倍數判準。
11. 續訓需還原參數、最佳化器狀態、步數、隨機狀態、資料位置與哨兵歷史六項；資料位置漏掉會造成隱形的重複訓練。
12. 可續訓取樣器的順序應由 seed 與 epoch 決定，只需記錄已消耗的樣本數即可還原。
13. MFU 等於 6ND 除以硬體峰值；小模型天然較低，低於 0.1 才需要用 profiler 找瓶頸。
14. activation 記憶體隨序列長度超線性成長；checkpointing 用約三分之一的計算換取數倍記憶體。
15. 自動回滾應區分處置強度：單步梯度暴增用跳過、NaN 用回滾加降學習率、連續惡化才中止。
16. 梯度暴增時應把該批次解碼存檔——訓練異常常常是資料品質的回饋。

成果檢核

完成本章後你必須繳交：

- 完整訓練腳本（程式 15A）：通過六項驗收，特別是續訓一致性與累積等價性。
- 學習率探測報告（程式 15B）：三種模型尺寸的曲線與建議值，含你採用的安全係數理由。
- 哨兵與回滾實作（程式 15C）：注入 NaN 後能正確偵測、回滾並降學習率重試。
- 24 小時訓練報告：指標曲線、事件時間軸、資源與成本表、MFU 分析。
- 中斷續訓的實測紀錄：至少一次真實中斷，並驗證 loss 序列的一致性。
- 異常批次紀錄：若有梯度尖峰，附上 dump_bad_batch 的輸出與你的分析。
- 一頁反思：三個你會改變的設定與理由。

本章的訓練基礎設施會被第 22 章（持續預訓練）、第 23 章（指令微調）、第 25 章（偏好對齊）與第 40 章（總整專題）直接沿用。從這一章開始，「訓練一個模型」對你而言不再是一件需要運氣的事——它是一個有監控、有備援、可被複製的工程流程。

第 16 章

訓練診斷、實驗科學與可重現報告

Training Diagnostics, Experimental Science, and Reproducible Reports

難度：核心 | 建議授課：第 16 週 | 硬體需求：A 級可完成全部實作；系統案例的九次訓練建議用 tiny 尺寸

叫獸開講

「我改了這個之後分數變高了」不是結論，那是巧合的候選人。我做過一個模擬：在一個完全沒有效果的改動上，只跑一個隨機種子，有 50.6% 的機率會看到「變好了」。跑三個種子，你才開始知道自己是發現了什麼，還是只是運氣好。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 把模型改動設計成可檢驗的假設，明確區分自變數、控制變數與應變數。
2. 說明為什麼單一種子的結果不能作為結論，並用模擬證明它的誤導性。
3. 計算標準誤與信賴區間，判斷你的實驗能偵測到多小的效果。
4. 辨識多重比較問題，並說明「跑很多組挑最好的」為什麼會產生假發現。
5. 從學習曲線診斷過擬合、欠擬合與資料問題。
6. 實作記憶化偵測，量化模型逐字複述訓練資料的程度。
7. 辨識評測污染、選擇偏誤與 cherry-picking，並建立自我檢查機制。
8. 撰寫符合開放科學要求的實驗紀錄與可重現套件。
9. 寫出誠實的失敗復盤報告，並說明負面結果的價值。

本章的性質

這是全書唯一一章不教任何新技術的章節。它教的是「怎麼知道自己有沒有做對」——而這件事在大模型領域特別重要，因為訓練成本高、隨機性大、而評測指標又容易被操弄。第 5 章建立了量測的規格，第 15 章建立了訓練的監控，本章建立的是**下結論的紀律**。這三者加起來，才讓你的實驗結果值得別人相信。

16.1 把改動變成假設

16.1.1 假設要能被推翻

一個好的實驗假設必須滿足三個條件：具體、可量測、可能為假。

寫法	問題	改寫成	評價
「用 SwiGLU 效果會更好」	「更好」是什麼？多好？	「在相同參數量下，SwiGLU 的驗證損失比 GELU 低至少 0.05」	可檢驗
「加入公文語料有幫助」	對什麼有幫助？	「加入公文語料後，公文分域驗證集的 BPC 下降至少 0.1，且其他四域不退步超過 0.02」	可檢驗
「這個模型很不錯」	完全無法檢驗	需拆成多個具體指標	不是假設

第二列的寫法特別值得學習：它同時規定了「要進步多少」與「不能退步多少」。這防止了一個常見的自欺——某個改動讓目標領域大幅改善，卻讓其他領域嚴重退步，而你只報告了前者。

16.1.2 三種變數

類型	定義	本章的要求	違反的後果
自變數	你刻意改變的東西	一次只改一個	無法歸因
控制變數	必須保持不變的東西	明確列出並驗證	結論不成立
應變數	你量測的結果	事前決定，不可事後挑選	cherry-picking

第三列的「事前決定」是本章最重要的紀律之一。如果你跑完實驗才決定要看哪個指標，你幾乎一定會找到一個對你有利的——因為指標夠多的話，總有幾個碰巧變好了。第 5.5.4 節的實驗紀錄九項就是為此設計的。

16.1.3 一份實驗設計表

experiments/exp_042_swiglu.yaml

```

experiment_id: exp_042
date: "2026-08-20"
owner: 李天宇

hypothesis: >
  在參數量對齊的前提下（d_ff 依 8d/3 調整），SwiGLU 的
  驗證損失比 GELU 低至少 0.03，且五個分域無任何一域
  退步超過 0.02。

independent_variable:
  name: activation
  values: [gelu, swiglu]
```

```

controlled:
- {name: params, value: "對齊至 ±1% 以內", verified_by: "程式 12B"}
- {name: train_tokens, value: 500_000_000}
- {name: peak_lr, value: 3.0e-4}
- {name: schedule, value: wsd}
- {name: data_version, value: "2026-08-15-a"}
- {name: tokenizer, value: "formosa-32k-v1"}

dependent_variables:          # 事前決定，不可事後增減
  primary: valid_loss_overall
  secondary: [bpc_gov, bpc_edu, bpc_social, bpc_industry, bpc_local]

seeds: [0, 1, 2]

decision_rule: >
  以三個種子的中位數比較。primary 改善  $\geq 0.03$  且所有
  secondary 退步  $< 0.02$  才判定假設成立。否則記為「未支持」。

preregistered: true          # 執行前已寫定，不可事後修改

```

preregistered 這個欄位是一個簡單但有力的機制：在跑實驗之前就把假設與判準寫下來並提交進版控。這樣事後就無法辯稱「我本來就想看這個指標」。這是從臨床試驗借來的做法，在機器學習領域越來越受重視。

16.2 隨機性：為什麼單一種子不算數

16.2.1 一個令人不安的模擬

假設你做了一個改動，而它實際上完全沒有效果。你跑一個種子，比較改動前後的驗證損失。

我做了一千次這樣的模擬（假設訓練的種子間標準差為 0.08，這在小模型上是合理的量級）。結果是：

****有 50.6% 的機率你會看到「改動後比較好」**。**

這個數字不令人意外——沒有效果的改動當然有一半機率看起來變好。但令人不安的是它的推論：如果你跑了十個不同的改動，其中五個會「看起來有效」，而你很可能會把它們寫進報告。

16.2.2 需要幾個種子

標準誤隨種子數的平方根下降。用同樣的雜訊量級（ $\sigma = 0.08$ ）計算：

種子數	差異的標準誤	可偵測的效果（2 SE）	實務判讀
1	0.113	0.226	只能偵測極大的效果

種子數	差異的標準誤	可偵測的效果 (2 SE)	實務判讀
3	0.065	0.131	本書的最低要求
5	0.051	0.101	較可靠
10	0.036	0.072	能偵測中等效果
20	0.025	0.051	成本通常不划算

這張表給了一個實用的判斷方法：****先看你想偵測多大的效果，再決定種子數****。如果你預期的改善只有 0.02，那麼三個種子根本測不出來——與其跑了三次得到不確定的結果，不如一開始就承認這個實驗設計不足以回答問題。

一個現實的妥協

大模型訓練昂貴，跑三個種子意味著三倍成本。實務上的折衷是：在小規模上跑多個種子確認方向，只在最終的關鍵實驗上放大規模。但無論如何，****單一種子的結果只能當作「值得進一步驗證的線索」，不能當作結論****——這是本章不打折扣的底線。

16.2.3 該報告什麼

報告方式	傳達什麼	問題	建議
只報最佳值	「我們達到了 2.85」	無法判斷穩定性	不可接受
只報平均	「平均 2.91」	看不出變異	不足
平均 ± 標準差	「2.91 ± 0.08」	假設常態分布	可接受
中位數與全距	「2.90 (2.83–2.99)」	穩健，不假設分布	本書建議
全部列出	「2.83, 2.90, 2.99」	最透明	種子少時最佳

本書的要求是：種子數少於五個時列出全部數值，五個以上用中位數與四分位距。無論哪一種，都必須說明種子數——一個沒有註明 n 的平均值是沒有意義的。

16.2.4 bootstrap 信賴區間

當種子數少、無法假設常態分布時，bootstrap 是一個穩健的替代方案。

[formosa/analysis/stats.py](#)

```

import numpy as np

def bootstrap_ci(values, n_boot=10000, ci=95, seed=0):
    """對小樣本估計信賴區間，不假設分布形狀"""
    rng = np.random.default_rng(seed)
    values = np.asarray(values, dtype=float)
    n = len(values)
    assert n >= 2, "至少需要兩個樣本"
    means = np.array([rng.choice(values, n, replace=True).mean()
                      for _ in range(n_boot)])
    lo, hi = np.percentile(means, [(100 - ci) / 2, 100 - (100 - ci) / 2])
    return {"mean": float(values.mean()),
            "median": float(np.median(values)),
            "ci_low": float(lo), "ci_high": float(hi),
            "ci_width": float(hi - lo), "n": n}

def compare_groups(a, b, n_boot=10000, seed=0):
    """比較兩組，回傳差異的信賴區間與『b 較優』的比例"""
    rng = np.random.default_rng(seed)
    a, b = np.asarray(a, float), np.asarray(b, float)
    diffs = np.array([
        rng.choice(a, len(a), replace=True).mean()
        - rng.choice(b, len(b), replace=True).mean()
        for _ in range(n_boot)])
    lo, hi = np.percentile(diffs, [2.5, 97.5])
    return {
        "observed_diff": float(a.mean() - b.mean()),
        "ci_low": float(lo), "ci_high": float(hi),
        # 區間是否跨越 0：跨越代表無法斷言有差異
        "crosses_zero": bool(lo < 0 < hi),
        "prob_b_better": float((diffs > 0).mean()),
    }

```

`crosses_zero` 這個欄位是判讀的關鍵。如果差異的 95% 信賴區間包含 0，那你就不能說「A 比 B 好」——只能說「在這個實驗規模下無法區分」。這句話聽起來很弱，但它是誠實的，而且它明確指出了下一步該做什麼（加大樣本或放大效果）。

一個實測參考：五個種子、 $\sigma \approx 0.08$ 時，bootstrap 的 95% 信賴區間寬度約 0.106。這意味著小於 0.1 的差異在五個種子下無法可靠區分——與 16.2.2 節的表格一致。

16.2.5 哪些隨機源會影響結果

第 4.6.1 節列過七個隨機來源。這裡從實驗設計的角度重新看：哪些是你該固定的、哪些是你該刻意變動的。

隨機源	該固定嗎	理由	做法
參數初始化	刻意變動	這是「種子間變異」的主要來源	每個種子不同
資料順序	刻意變動	同上	隨種子改變
dropout 遮罩	刻意變動	同上	隨種子改變
資料切分	必須固定	切分改變會讓比較失效	用固定的鹽值 (第 5.4.5 節)
tokenizer	必須固定	影響指標可比性	同一份檔案
評測題目順序	必須固定	避免引入額外變異	固定順序或多次平均

這張表的邏輯是：****你想量測的變異必須被允許發生，你不想要的變異必須被消除****。種子間的變異正是你要估計的東西 (16.2.2 節的 σ)，所以初始化與資料順序必須隨種子改變；而資料切分若也隨種子改變，那你量到的就會是「切分變異加上訓練變異」的混合，無法歸因。

一個常見的錯誤是把「seed」這個參數同時傳給切分與訓練。結果是每個種子看到不同的驗證集，而驗證集的難度差異可能大於你想偵測的效果——整個實驗就白做了。

16.3 多重比較：跑很多組挑最好的

16.3.1 問題的規模

假設你有一個完全無效的改動，用 5% 的顯著水準檢定。跑一組實驗，有 5% 機率誤判為有效。但如果你跑很多組呢？

跑的組數	至少一組「顯著」的機率	判讀	實務情境
1	5.0%	可接受	單一假設
5	22.6%	開始有問題	掃描五個超參數值
10	40.1%	很可能有假發現	中等規模的網格搜尋
20	64.2%	幾乎一定有假發現	常見的調參規模
50	92.3%	幾乎必然	大規模掃描

這張表解釋了一個常見的現象：為什麼很多論文報告的改進在別人重現時消失了。不是作者造假，而是他們跑了很多組設定，挑了最好的那一組報告——而那一組很可能只是運氣好。

16.3.2 三種處置

做法	說明	代價	適用
預先註冊	事前寫定要檢驗的假設	失去探索彈性	關鍵結論
調整顯著水準	如 Bonferroni : $\alpha \div$ 比較次數	變得非常保守	正式的統計檢定
分成探索與確認	一半資料探索、另一半確認	需要更多資料	本書建議
誠實報告全部	列出所有跑過的設定	報告變長	一律要求

本書採用第三與第四種的組合。探索階段可以自由地掃描、嘗試、挑選——那是研究的正常過程；但一旦選定了某個設定，必須在一組**從未被用於選擇的保留資料**上重新驗證。這與第 5.4.7 節的私有保留集是同一個邏輯。

第四種是無論如何都要做的。你的報告可以說「我們掃描了 20 組設定，其中最好的是 X」——這是誠實的；但不能只說「X 的表現是 2.85」而不提你試過另外 19 組。差別在於讀者能不能自行判斷這個數字有多可信。

16.3.3 超參數掃描的正確做法

scripts/sweep.py (節錄)

```
import itertools, json
from pathlib import Path

def run_sweep(base_cfg, grid: dict, seeds, out_dir: Path,
              exploration_split="valid_explore",
              confirmation_split="valid_confirm"):
    """掃描時只看探索集，最終確認才用確認集"""
    keys = list(grid)
    combos = list(itertools.product(*(grid[k] for k in keys)))
    print(f"共 {len(combos)} 組設定 × {len(seeds)} 個種子 ")
    f"= {len(combos) * len(seeds)} 次訓練")

    records = []
    for combo in combos:
        cfg = replace_many(base_cfg, dict(zip(keys, combo)))
        for seed in seeds:
            r = train_and_eval(cfg, seed, splits=[exploration_split])
            records.append(**dict(zip(keys, combo)), "seed": seed,
```

```

        "explore_loss": r[exploration_split])

# 用探索集選出最佳設定
best = select_best(records, keys, metric="explore_loss")
print(f"探索集選出: {best}")

# 只有這一組才在確認集上跑（且用新的種子）
confirm_seeds = [s + 100 for s in seeds]
confirm = [train_and_eval(replace_many(base_cfg, best), s,
                          splits=[confirmation_split])
            for s in confirm_seeds]

(out_dir / "sweep_full.json").write_text(
    json.dumps(records, ensure_ascii=False, indent=2), encoding="utf-8")
return {"n_configs_tried": len(combos),
        "best_config": best,
        "explore_best": min(r["explore_loss"] for r in records),
        "confirm_values": confirm,
        "confirm_median": sorted(confirm)[len(confirm) // 2]}

```

注意 `confirm_seeds` 用的是不同的種子。這是刻意的：如果用同一組種子，你等於是在同樣的隨機條件下重測，無法排除「這組種子剛好適合這個設定」的可能。

報告時應該同時列出 `explore_best` 與 `confirm_median`。前者通常比後者好——那個差距就是「挑選帶來的樂觀偏誤」的大小，而把它明確寫出來本身就是誠實的表現。

16.4 學習曲線診斷

16.4.1 六種曲線形狀

形狀	訓練與驗證的關係	診斷	處置
兩者同步下降	差距小且穩定	正常	繼續
訓練降、驗證升	差距持續擴大	過擬合	加資料、加正則、早停
兩者都停在高位	差距小但都不理想	欠擬合或容量不足	加大模型、調學習率
驗證低於訓練	驗證持續較低	dropout 或切分問題	見 16.4.2
驗證劇烈震盪	訓練平穩但驗證跳動	驗證集太小	加大驗證集
兩者都極低	接近 0	資料洩漏	查污染與去重

16.4.2 「驗證損失低於訓練損失」的四個原因

這個現象看起來違反直覺，但其實相當常見，而且有多個可能的成因。

原因	機制	如何確認	是否需處置
dropout	訓練時開啟、驗證時關閉	把 dropout 設為 0 重測	正常，不需處置
訓練損失是移動平均	包含了早期較高的值	比較同一步的瞬時值	報告方式問題
驗證集較簡單	切分時未隨機或分布不同	檢查兩者的長度與領域分布	需重切
驗證集與訓練集重疊	去重不完整	跑第 5.6 節的污染偵測	必須處置

前兩項是正常的、不需要處理的；後兩項是真正的問題。診斷順序應該是：先關掉 dropout 重測（幾分鐘），再比較瞬時值，最後才去查切分與污染（較耗時）。

16.4.3 用分域曲線看得更清楚

總體的驗證損失會把不同領域的表現平均掉。第 5.4.4 節建立的五個分域驗證集在這裡發揮作用。

觀察	可能的意義	進一步檢查	章節
某域下降特別慢	該域的語料比例不足	看語料統計報表	第 9.7.3 節
某域在中期開始上升	該域被其他域擠壓	調整配比或做退火	第 9.5 節
某域曲線異常平滑	該域樣本數太少	檢查驗證集規模	第 5.4.3 節
本土語言域幾乎不動	模型容量或 tokenizer 限制	看 fertility 與 BPC	第 7.5.2 節

最後一列指出一個重要的診斷原則：當某個領域完全不進步時，先確認問題出在哪一層。如果是 tokenizer 把臺羅切得極碎（第 7.5.2 節），那再多訓練也沒用——必須回頭改詞彙表。這是本書一貫的做法：把「效果不好」拆解成可定位的成因鏈。

16.4.4 從曲線看資料問題

學習曲線不只反映模型，也反映資料。以下四種形狀是資料出問題的訊號。

形狀	可能的資料問題	如何確認	章節
週期性的規律起伏	資料未打亂，同類內容聚集	檢查取樣器與資料排列	第 15.5.3 節
某個 epoch 邊界明顯轉折	資料量太小，開始第二輪	算詞元數除以語料大小	第 9 章
loss 階梯式下降	語料由差異極大的來源拼接	看分域曲線與配比	第 9.5 節
初期下降極快後停滯	語料重複度高	跑去重統計	第 9.3 節

第二列值得展開。如果你的訓練詞元數是語料大小的三倍，那就是跑了三個 epoch——而第二輪之後模型是在重看已經看過的內容。這在小語料的情境（臺灣的多數專案）中很常見，且會直接提高記憶化風險（16.5 節）。

判斷方式很簡單：把 epoch 邊界標在曲線上。如果每次跨越邊界時 loss 都有一個明顯的下墜，那就是模型在「認出」它看過的內容——那不是學習，是記憶。

把 epoch 邊界標在曲線上

```
def annotate_epochs(ax, tokens_per_step, corpus_tokens, max_step):
    """在訓練曲線上標出每個 epoch 的邊界"""
    steps_per_epoch = corpus_tokens / tokens_per_step
    e = 1
    while e * steps_per_epoch < max_step:
        x = e * steps_per_epoch
        ax.axvline(x, linestyle=":", linewidth=1, color="gray")
        ax.text(x, ax.get_ylim()[1], f"epoch {e+1}",
               fontsize=8, va="top", ha="left", color="gray")
        e += 1
    return int(max_step / steps_per_epoch) + 1    # 總 epoch 數
```

16.5 記憶化偵測

16.5.1 為什麼要量它

第 8.4.5 節從個資的角度談過記憶化，這裡從實驗科學的角度再看一次。記憶化會同時造成三個問題。

問題	機制	後果	章節
評測虛高	模型背過測試題	所有結論不可信	第 5.6 節

問題	機制	後果	章節
隱私風險	可能吐出訓練資料中的個資	法遵與倫理問題	第 8.4 節
泛化假象	看起來學會了，其實是記住了	換個場景就失效	本節

16.5.2 用「逐字複述長度」量化

最直接的方法是：給模型訓練資料中某段文本的前綴，看它能不能逐字接出後續。

為什麼「逐字」有意義？因為隨機產生一段特定文字的機率極低。以 32,000 的詞彙表為例：

連續正確的詞元數	隨機產生的機率	判讀	嚴重度
5	約 3×10^{-23}	可能是常見片語	低
10	約 9×10^{-46}	幾乎確定是記住的	中
20	約 8×10^{-91}	確定是記住的	高
50	約 6×10^{-226}	嚴重的逐字記憶	極高

(機率為 $(1/V)^n$ 的直接計算，實際上模型並非均勻取樣，但這個數量級足以說明問題。)

要注意的是：短片語的逐字重現未必是記憶化。「臺灣科技大學」這六個字任何學過中文的模型都會接，那是語言知識而非記憶。所以判準通常設在 10 個詞元以上，並排除高頻的固定搭配。

formosa/analysis/memorization.py

```
import torch

@torch.no_grad()
def verbatim_length(model, tok, text, prefix_tokens=32,
                    max_check=200, device="cpu"):
    """給定前綴，量測模型 greedy 續寫與原文逐字相符的長度"""
    ids = tok.encode(text)
    if len(ids) < prefix_tokens + 10:
        return None # 太短，無法判斷

    prefix = torch.tensor([ids[:prefix_tokens]], device=device)
    target = ids[prefix_tokens:prefix_tokens + max_check]

    model.eval()
    cur = prefix
    matched = 0
    for t in target:
```

```

logits = model(cur[:, -model.cfg.max_seq_len:])
nxt = int(logits[0, -1].argmax())
if nxt != t:
    break
matched += 1
cur = torch.cat([cur, torch.tensor([[nxt]], device=device)], dim=1)
return matched

def memorization_report(model, tok, samples, threshold=10, device="cpu"):
    """samples 應取自訓練集；回傳分布與高風險樣本"""
    lengths, flagged = [], []
    for s in samples:
        n = verbatim_length(model, tok, s["text"], device=device)
        if n is None:
            continue
        lengths.append(n)
        if n >= threshold:
            flagged.append({"doc_id": s.get("doc_id"), "matched": n,
                           "preview": s["text"][:80]})

    lengths.sort()
    n = len(lengths)
    return {
        "n_samples": n,
        "median": lengths[n // 2] if n else 0,
        "p95": lengths[int(n * 0.95)] if n else 0,
        "max": lengths[-1] if n else 0,
        "flagged_rate": round(len(flagged) / max(n, 1), 4),
        "flagged": sorted(flagged, key=lambda d: -d["matched"])[ :20],
    }

```

16.5.3 影響記憶化的因素

因素	增加記憶化	降低記憶化	你能控制嗎
重複次數	同一內容出現多次	徹底去重	能 (第 9.3 節)
訓練輪數	多輪重複訓練	單輪或少輪	能 (第 15 章)
模型大小	參數越多記得越牢	小模型	能 (第 12 章)
內容獨特性	罕見字串 (號碼、專有名詞)	常見表述	部分 (第 8.4.5 節)
資料總量	總量少時每筆權重高	大量資料稀釋	受限於取得管道
上取樣	刻意提高比例的資料	—	能 (第 9.5.2 節)

最後一列是一個需要特別留意的權衡：第 9.5.2 節建議把本土語言上取樣三倍，以免模型完全學不到；但上取樣本質上就是重複訓練，會提高那批資料的記憶化風險。所以本書要求：任何被上取樣的資料，

都必須在本章做一次記憶化檢查。

16.5.4 可接受的範圍

記憶化無法完全消除——某種程度的記憶其實是必要的（模型要記得「臺灣」的首都相關事實）。問題在於界線畫在哪裡。

資料類型	可接受的逐字長度	理由	超標時
公開的常識與百科	較寬鬆	本來就是公開知識	通常不需處理
法規條文	較寬鬆	逐字正確反而是優點	注意版本時效
一般網頁文本	10 詞元以下	避免著作權疑慮	加強去重
含個資的內容	零容忍	第 8.4 節的要求	必須重新處理資料
評測集內容	零容忍	污染	重建語料（第 9.4 節）

16.6 三種自欺與它們的解藥

16.6.1 評測污染

第 5.6 節已經完整處理，這裡只補充實驗科學的視角：污染的可怕之處在於它讓所有後續的比較都失效。如果 A 與 B 都在受污染的測試集上評估，你甚至無法判斷哪一個比較好——因為污染的影響程度可能不同。

本章的要求是：每一次正式的實驗報告，都必須附上污染檢查的結果，即使你認為不可能有污染。這與第 8 章的資料清冊是同一個邏輯——證明你檢查過，比宣稱沒問題有價值。

16.6.2 選擇偏誤

形式	例子	為什麼有問題	對策
挑種子	「這次跑得特別好」	樣本不具代表性	固定種子集合並全部報告
挑檢查點	「第 8,347 步的結果最好」	在驗證集上過擬合	事前決定選點規則
挑指標	事後選對自己有利的	多重比較	事前註冊（16.1.3 節）
挑樣本	只展示好的生成結果	cherry-picking	隨機抽樣展示

第二列的「挑檢查點」值得展開，因為它很隱蔽。如果你在驗證集上每 500 步評估一次、跑 20,000 步，那就是 40 次比較——你選出的「最佳檢查點」在驗證集上必然是樂觀的。正確做法是事前定好規則（例如「最後一個檢查點」或「驗證損失連續三次未改善時停止」），並在獨立的測試集上報告最終數字。

16.6.3 cherry-picking 生成結果

這是展示模型時最常見的自欺。你跑了三十次生成，挑出最好的三段放進簡報——讀者看到的是模型的最佳表現，卻誤以為那是典型表現。

formosa/analysis/sampling_display.py

```
import random, json

def honest_sample_display(model, tok, prompts, n_per_prompt=5,
                          seed=42, out_path=None):
    """誠實的展示方式：固定種子、全部保留、不事後挑選"""
    rng = random.Random(seed)
    records = []
    for p in prompts:
        for i in range(n_per_prompt):
            gen_seed = rng.randint(0, 2 ** 31)
            text = generate(model, tok, p, seed=gen_seed)
            records.append({"prompt": p, "index": i,
                           "gen_seed": gen_seed, "text": text})
    if out_path:
        out_path.write_text(json.dumps(records, ensure_ascii=False,
                                         indent=2), encoding="utf-8")
    return records

def pick_for_display(records, mode="random", k=3, seed=0):
    """展示時的抽樣規則，必須在報告中說明用了哪一種"""
    if mode == "random":
        return random.Random(seed).sample(records, k)
    if mode == "first":
        return records[:k]
    if mode == "spread":
        # 依長度排序後均勻取樣，涵蓋不同情況
        s = sorted(records, key=lambda r: len(r["text"]))
        idx = [int(i * (len(s) - 1) / (k - 1)) for i in range(k)]
        return [s[i] for i in idx]
    raise ValueError(mode)
```

本書的要求很簡單：**展示的樣本必須說明是怎麼選出來的**。「隨機抽取三段」「前三段」「涵蓋長中短各一段」都是誠實的；「精選三段」不是。而完整的生成紀錄應該附在附錄或倉庫中，讓讀者能自行檢視。

一個實務建議

在報告中同時放一段「代表性的失敗案例」。這聽起來違反直覺——為什麼要展示自己的模型做不好的地方？但它有兩個效果：一是讓讀者更相信你的其他數字，二是它往往是最有資訊量的部分。第 40 章的專題評分中，誠實的失敗分析是加分項而非扣分項。

16.6.4 一個容易忽略的偏誤：倖存者偏誤

你的實驗紀錄中只有「跑完的」實驗。那些因為 OOM、發散、或你中途覺得「這樣不對」而手動停掉的呢？

情況	為什麼被排除	造成的偏誤	處置
訓練發散	沒有最終數字	掩蓋了設定的不穩定性	記錄發散率
中途手動停止	「看起來不對」	主觀判斷混入結果	事前定好停止規則
OOM	技術問題	排除了大批次設定	報告記憶體限制
忘記存檔	人為疏失	隨機遺失	自動化存檔

第一列在比較架構時特別重要。假設 Post-LN 在十次執行中有四次發散、成功的六次平均為 3.05；Pre-LN 十次都成功、平均 3.09。如果你只報告成功的執行，Post-LN 看起來比較好——但它實際上有 40% 的機率浪費你整次訓練。

正確的報告方式是把發散率當成一個獨立的指標。第 12 章程式 12C 的消融實驗就明確要求統計「發散的執行次數 ÷ 總次數」，理由就在這裡。

第二列的處置值得強調：如果你會在訓練中途根據觀察決定要不要繼續，那個決定規則必須事前寫下來（例如「若 2,000 步後 loss 仍高於 5.0 則停止」）。憑感覺停掉的實驗等於把你的主觀判斷混進了結果。

16.7 可重現的實驗紀錄

16.7.1 從紀錄到套件

第 4.7 節建立了環境指紋，第 5.5.4 節列了實驗紀錄九項。本節把它們整合成一個可以直接交付的套件。

層次	內容	回答什麼問題	章節
實驗設計	假設、變數、判準（預先註冊）	你想證明什麼？	16.1.3
環境指紋	commit、版本、硬體、資料雜湊	在什麼條件下跑的？	第 4.7.3 節
執行紀錄	metrics.jsonl、日誌、事件	過程發生了什麼？	第 15 章
分析	統計、信賴區間、圖表	結果代表什麼？	16.2
產出物	checkpoint、生成樣本、評測結果	可以驗證嗎？	第 13 章
重現腳本	一鍵從零重跑	別人能重做嗎？	第 4 章

16.7.2 實驗套件的目錄結構

experiments/exp_042_swiglu/

```

exp_042_swiglu/
├── DESIGN.yaml          # 預先註冊的假設與判準（16.1.3 節）
├── README.md           # 三行說明 + 一行重現指令
├── reproduce.sh        # 一鍵重跑
├── configs/
│   ├── gelu.yaml
│   └── swiglu.yaml
├── runs/
│   ├── gelu_seed0/
│   │   ├── fingerprint.json
│   │   ├── metrics.jsonl
│   │   ├── train.log
│   │   └── final.pt
│   ├── gelu_seed1/ ...
│   └── swiglu_seed2/ ...
├── analysis/
│   ├── analyze.py      # 從 runs/ 產生下面的結果
│   ├── results.json    # 統計摘要與信賴區間
│   ├── curves.png
│   └── by_domain.png
├── samples/
│   ├── all_generations.jsonl # 完整紀錄，不挑選
│   └── display_selection.md  # 展示用，附選取規則
└── REPORT.md           # 最終報告（16.8 節的格式）

```

`reproduce.sh` 是這個套件的核心。它應該做到：在一台乾淨的機器上，執行一個指令就能從原始資料重建所有結果。如果因為算力限制無法完整重跑，至少要能重跑分析階段（從 `metrics.jsonl` 產生所有圖表與統計）。

16.7.3 分析腳本應該做什麼

analysis/analyze.py (節錄)

```

import json
from pathlib import Path
from formosa.analysis.stats import bootstrap_ci, compare_groups

def load_runs(runs_dir: Path):
    """從各次執行讀取最終指標，並驗證環境一致性"""
    runs, fingerprints = [], []
    for d in sorted(runs_dir.iterdir()):
        if not d.is_dir():
            continue
        fp = json.loads((d / "fingerprint.json").read_text(encoding="utf-8"))
        fingerprints.append(fp)
        lines = [json.loads(l) for l in
                  (d / "metrics.jsonl").read_text(encoding="utf-8").splitlines()]
        final = [l for l in lines if "valid_loss" in l][-1]
        arm, seed = d.name.rsplit("_seed", 1)
        runs.append({"arm": arm, "seed": int(seed), **final})

    # 環境一致性檢查：所有執行必須在相同條件下
    for key in ("torch", "cuda", "git"):
        vals = {json.dumps(f.get(key), sort_keys=True) for f in fingerprints}
        if len(vals) > 1:
            print(f"[警告] 各次執行的 {key} 不一致，比較可能無效")
    if any(f.get("git", {}).get("dirty") for f in fingerprints):
        print("[警告] 有執行在 git dirty 狀態下進行，不可作為正式結果")
    return runs

def analyze(runs, design):
    arms = sorted({r["arm"] for r in runs})
    assert len(arms) == 2, "本分析僅處理兩組比較"
    a = [r[design["primary"]] for r in runs if r["arm"] == arms[0]]
    b = [r[design["primary"]] for r in runs if r["arm"] == arms[1]]

    result = {
        arms[0]: bootstrap_ci(a),
        arms[1]: bootstrap_ci(b),
        "comparison": compare_groups(a, b),
        "n_seeds": len(a),
    }
    # 依預先註冊的判準自動判定
    diff = result["comparison"]["observed_diff"]
    crosses = result["comparison"]["crosses_zero"]
    result["verdict"] = (
        "未支持 (信賴區間跨越 0)" if crosses else
        "支持" if abs(diff) >= design["min_effect"] else
        f"效果存在但小於預設門檻 {design['min_effect']}"
    )
    return result

```

那段環境一致性檢查值得注意：它會警告「各次執行的 torch 版本不一致」或「有執行在 git dirty 狀態下進行」。這種檢查在多人協作或跨越數週的實驗中特別重要——你可能中途更新了套件而不自知，而那足以讓比較失效。

16.8 報告的寫法

16.8.1 六個段落

段落	內容	常見錯誤	篇幅
假設與動機	你想檢驗什麼、為什麼重要	寫成願景而非假設	半頁
方法	變數、控制、種子、指標定義	沒寫控制了什麼	一頁
結果	表格與圖，含信賴區間與 n	只報平均或最佳值	一至二頁
分析	結果代表什麼、為什麼	直接跳到結論	一頁
限制	哪些結論不成立、沒測到什麼	整段省略	半頁
結論與下一步	判定與後續實驗	過度推廣	半頁

第五段是本書一貫的要求（第 5 章的系統案例也有同樣的規定）。一份沒有限制段落的報告，讀者無法判斷你的結論適用到哪裡——而那往往比結論本身更重要。

16.8.2 負面結果的價值

如果你的假設沒有被支持，那份報告仍然有價值——而且往往比正面結果更有價值。

負面結果的類型	傳達什麼	對誰有用	該怎麼寫
改動確實無效	這條路不通	所有想試同樣改動的人	明確說明實驗規模與檢定力
效果小於門檻	有效但不值得	成本敏感的決策者	報告效果大小與成本
無法區分	實驗規模不足	規劃後續實驗的人	說明需要多大樣本
與文獻不符	可能有條件限制	重現該文獻的人	仔細比對設定差異

第三列與第一列的區別非常重要：「證明無效」與「無法證明有效」是兩件不同的事。前者需要足夠的檢定力（第 16.2.2 節的表格），後者只代表你的實驗不夠靈敏。誠實的報告必須說清楚自己屬於哪一種。

16.8.3 失敗復盤

除了實驗的負面結果，工程上的失敗也值得記錄。本書建議每個專案維護一份復盤紀錄。

POSTMORTEM.md 的格式

```
## 事件：訓練在第 12,400 步發散

**發生時間**：2026-08-18 03:20
**發現方式**：哨兵自動告警（loss NaN）
**影響**：損失約 40 分鐘的訓練進度

### 時間軸
- 11,800 步：梯度範數開始出現週期性尖峰（事後回看才發現）
- 12,380 步：梯度範數達到平時的 12 倍
- 12,400 步：loss 變成 NaN，哨兵觸發回滾
- 12,400 步：自動回滾至 12,000 步的檢查點，學習率降半
- 之後訓練正常完成

### 根本原因
dump_bad_batch 顯示第 12,395 步的批次含一份約 8,000 字的
重複內容（同一段文字重複約 60 次），來自某個未被第 9 章
line_repetition 過濾器捕捉的文件。

### 為什麼過濾器沒抓到
該文件的重複發生在段落之間而非行之間，而過濾器只檢查行級重複。

### 處置
1. 立即：回滾已自動完成，訓練得以繼續
2. 短期：在過濾器加入段落級重複檢查
3. 長期：把「梯度範數週期性尖峰」加入哨兵的告警條件

### 學到什麼
哨兵的告警設在 NaN 太晚了。梯度範數在 600 步前就有徵兆，
若當時告警，可以在完全發散前介入。
```

這份格式的重點在最後兩節：處置分成立即、短期、長期三層，而「學到什麼」要具體到可以改進系統。一份寫「以後要小心」的復盤沒有價值；寫「哨兵的告警條件應該加上 X」才有。

16.8.4 一份實驗可信度檢查表

把本章的所有紀律濃縮成一份可以在十分鐘內完成的檢查表。建議印出來，在提交任何實驗結論前逐項確認。

項次	檢查什麼	通過標準	節次
1	假設是否事前寫定並可被推翻	DESIGN.yaml 存在且已提交	16.1.3

項次	檢查什麼	通過標準	節次
2	自變數是否只有一個	其餘皆在 controlled 清單中	16.1.2
3	種子數是否足以偵測預期效果	n 滿足 $2\sigma\sqrt{(2/n)} \leq \text{效果}$	16.2.2
4	是否報告了變異而非只有平均	中位數與四分位距或全列	16.2.3
5	信賴區間是否跨越 0	若跨越則不可宣稱有差異	16.2.4
6	跑過的設定總數是否誠實報告	含未採用的組合	16.3.2
7	最終數字是否來自未被用於選擇的資料	確認集或保留集	16.3.2
8	是否做過污染檢查	有報告，即使結果為零	第 5.6 節
9	是否做過記憶化檢查	訓練集與驗證集對照	16.5
10	展示樣本的選取規則是否說明	不可為「精選」	16.6.3
11	發散或中止的執行是否納入報告	報告發散率	16.6.4
12	環境指紋是否一致且非 dirty	分析腳本無警告	16.7.3
13	限制段落是否具體到可偵測效果	有數字而非「可能不足」	16.8.1
14	是否能一鍵重現	reproduce.sh 在乾淨環境可執行	16.7.2

這十四項不是為了刁難，而是為了保護你。一份通過全部檢查的實驗，即使結論是「無法區分」，也是可以引用、被延續、被信任的。而一份沒通過的實驗，即使數字漂亮，也只是一個待驗證的線索。

第 40 章的總整專題評分中，這份檢查表會被直接使用。建議你從現在開始，每一次實驗都跑一遍——三十次之後它會變成反射，那時候你就真正養成了做實驗的紀律。

程式實作

本章三個實作構成你的實驗分析工具鏈。它們會被第 17、20、22、25 章的所有實驗直接使用。

程式 16A 實驗矩陣執行器

目標：自動執行超參數格點與多種子實驗，正確分離探索集與確認集，並彙整成統計表。

功能	說明	驗收標準	節次
格點展開	從 YAML 讀取並展開所有組合	組合數正確	16.3.3
多種子	每組設定跑指定的種子	種子集合可重現	16.2
探索 / 確認分離	選擇只看探索集	確認集僅用一次	16.3.2
續跑	中斷後跳過已完成的組合	不重跑	第 15.5 節
彙整	產出含信賴區間的統計表	與手算一致	16.2.4
一致性檢查	警告環境不一致的執行	能偵測 dirty 與版本差異	16.7.3

延伸挑戰：加入一個「所需種子數估算器」。輸入你觀察到的種子間標準差與想偵測的效果大小，輸出建議的種子數（用 16.2.2 節的公式）。這會讓你在規劃實驗時就知道它夠不夠靈敏，而不是跑完才發現測不出來。

程式 16B 統計分析與圖表產生器

目標：從 metrics.jsonl 自動產生含誤差範圍的比較圖與統計摘要。

analysis/plots.py (節錄)

```
import json
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = ["Noto Sans CJK TC", "Microsoft JhengHei"]
plt.rcParams["axes.unicode_minus"] = False

def plot_curves_with_band(runs_by_arm, metric="valid_loss",
                          out_path=None):
    """多種子的曲線：畫中位數與四分位帶，而非單一條線"""
    fig, ax = plt.subplots(figsize=(7, 4.5))
    for arm, runs in runs_by_arm.items():
```

```

steps = sorted([p["step"] for r in runs for p in r])
med, lo, hi = [], [], []
for s in steps:
    vals = [p[metric] for r in runs for p in r
             if p["step"] == s and metric in p]
    if len(vals) < len(runs):
        continue # 只畫所有種子都有的步數
    med.append(np.median(vals))
    lo.append(np.percentile(vals, 25))
    hi.append(np.percentile(vals, 75))
xs = steps[:len(med)]
line, = ax.plot(xs, med, label=f"{arm} (n={len(runs)})")
ax.fill_between(xs, lo, hi, alpha=0.2, color=line.get_color())

ax.set_xlabel("訓練步數"); ax.set_ylabel(metric)
ax.set_title(f"{metric}：中位數與四分位距")
ax.legend(); fig.tight_layout()
if out_path:
    fig.savefig(out_path, dpi=150)
return fig

def plot_effect_size(comparison, out_path=None):
    """把差異與信賴區間畫成森林圖，一眼看出是否跨越 0"""
    fig, ax = plt.subplots(figsize=(6, 2.5))
    d = comparison["observed_diff"]
    lo, hi = comparison["ci_low"], comparison["ci_high"]
    ax.errorbar([d], [0], xerr=[[d - lo], [hi - d]], fmt="o", capsize=6)
    ax.axvline(0, linestyle="--", linewidth=1, color="gray")
    ax.set_yticks([]); ax.set_xlabel("差異（負值代表改動較優）")
    verdict = "無法區分" if comparison["crosses_zero"] else "有差異"
    ax.set_title(f"效果大小與 95% 信賴區間 結論：{verdict}")
    fig.tight_layout()
    if out_path:
        fig.savefig(out_path, dpi=150)
    return fig

```

plot_curves_with_band 刻意不畫單一條線。多種子的曲線應該畫成「中位數 + 帶狀範圍」，這樣讀者一眼就能看出兩組是否真的分開，還是只是在彼此的變異範圍內晃動。這個視覺化本身就是一種誠實。

程式 16C 記憶化偵測

目標：量測你在第 13 或 15 章訓練的模型的記憶化程度，並找出高風險樣本。

必須完成的五項：

- 從訓練集隨機抽取至少 500 段文本，量測逐字複述長度的分布（中位數、p95、最大值）。
- 從驗證集抽同樣數量作為對照。訓練集的複述長度應該明顯高於驗證集——若沒有差異，可能代表你的切分有問題。
- 對被上取樣的資料（例如本土語言）單獨統計，與一般資料比較（16.5.3 節的要求）。
- 找出複述長度最長的 20 個樣本，人工檢視它們的性質（是常見片語？重複內容？還是真正的記

憶化？）。

- 若有任何含個資或評測內容的樣本被複述超過 10 個詞元，依第 8.4 節與第 9.4 節處置並記錄。

一個預期中的觀察

在小模型與大語料的情況下，記憶化通常很低——你可能發現多數樣本的複述長度只有兩三個詞元。這是好消息，但也要注意：模型太小時「記不住」與「學會了不需要記」是兩回事。真正需要警惕的是第 22 章之後——當你在較大的底模上做適配時，記憶化風險會顯著上升。

系統案例：詞彙表大小對繁中生成品質的完整實驗

本章的系統案例是執行一次完整的、可投稿等級的實驗：檢驗詞彙表大小（8K / 16K / 32K）對繁體中文生成品質的影響。這會整合第 7 章的 tokenizer、第 13 章的模型、第 15 章的訓練與本章的分析。

為什麼選這個題目

這個題目有三個教學上的優點。第一，它有真實的爭議性——第 7 章告訴你大詞彙表能降低 fertility，第 12 章告訴你它會吃掉小模型的參數，兩者互相拉扯，答案並不顯然。第二，它涉及一個棘手的比較問題：不同 tokenizer 的困惑度不可比（第 5.3.4 節），必須用 BPC。第三，它的結論對臺灣有實際意義。

實驗設計的四個難點

難點	問題	處置	章節
指標不可比	不同 tokenizer 的 PPL 無法比較	主指標用 BPC	第 5.3.5 節
參數量不同	詞彙表大小直接改變參數量	兩種對照：固定總參數與固定層數	第 12.5 節
訓練預算	同樣的語料會產生不同的詞元數	固定「看過的字元數」而非詞元數	第 5.5.3 節
序列長度	同樣的 T 涵蓋的內容不同	報告時註明每種設定的字元數	第 7.7.3 節

第三列是這個實驗最巧妙的地方。如果你固定訓練詞元數，8K 詞彙表的模型會看過較少的實際文字（因為 fertility 高）；如果固定字元數，它會走較多的訓練步數。兩種都可以，但必須明確說明並解釋你為什麼選那一種。本書建議固定字元數，因為那才是「模型讀了多少內容」的真實度量。

交付要求

- 預先註冊的實驗設計 (DESIGN.yaml)，含假設、變數、判準與種子集合。
- 九次訓練 (三種詞彙表 × 三個種子) 的完整紀錄與環境指紋。
- 主結果：BPC 的中位數、四分位距與 bootstrap 信賴區間，並依判準給出判定。
- 分域結果：五個領域各自的 BPC，特別注意本土語言域。
- 參數量與 fertility 的對照，說明每種設定的取舍。
- 誠實的生成樣本：每種設定隨機抽取三段，附選取規則。
- 限制段落：你的實驗規模能偵測多大的效果？哪些結論不成立？
- 完整的實驗套件，含 reproduce.sh。

評分重點

- 是否用 BPC 而非 PPL 作為主指標。用 PPL 比較不同 tokenizer 者，本案例不予計分。
- 訓練預算的固定方式是否明確說明並有理由。
- 信賴區間是否被計算並用於判定。只報平均值者扣分。
- 若結論是「無法區分」，是否誠實報告並說明所需的實驗規模。
- 限制段落的品質。寫「樣本數可能不足」不算，要具體到「 $n=3$ 時只能偵測 0.13 以上的差異」。

最後一項連結到 16.2.2 節的表格：那張表讓「樣本數不足」從一句籠統的免責聲明，變成一個具體的數字。這正是本章想教會你的——把模糊的擔憂變成可計算的量。

章末評量

一、概念題

1. 一個好的實驗假設要滿足哪三個條件？請把「用 SwiGLU 效果會更好」改寫成可檢驗的形式。
2. 為什麼應變數必須事前決定？事後挑選指標會造成什麼問題？
3. 為什麼單一種子的結果不能作為結論？請用本章的模擬數字說明。
4. 解釋「證明無效」與「無法證明有效」的差別，以及各自需要什麼條件。
5. 什麼是多重比較問題？跑 20 組設定挑最好的，會有多高的機率得到假發現？
6. 「驗證損失低於訓練損失」有哪四種可能原因？診斷順序應該如何安排？
7. 為什麼「逐字複述 10 個詞元」幾乎可以確定是記憶化？請用機率說明。
8. 上取樣與記憶化的關係是什麼？本書對此有什麼要求？
9. 為什麼負面結果有價值？請說明三種不同類型的負面結果各自對誰有用。

二、計算題

1. 種子間標準差為 0.08。計算 $n = 3, 5, 10$ 時，兩組比較的差異標準誤，以及能偵測的效果大小（2 SE）。
2. 若你預期的改善只有 0.04，用上題的公式計算需要多少種子才能可靠偵測。
3. 跑 10 組設定，每組在 5% 顯著水準下檢定。若實際上全部無效，至少一組被誤判為有效的機率是多少？
4. 詞彙表 32,000。計算模型「隨機」產生連續 10 個特定詞元的機率。這說明了什麼？
5. 五個種子的驗證損失為 [2.88, 2.91, 2.95, 2.97, 3.04]。計算中位數、四分位距與全距。
6. 兩組各三個種子：A = [3.10, 3.14, 3.19]、B = [3.02, 3.09, 3.16]。觀察到的平均差異是多少？以標準差估計，這個差異可信嗎？

三、除錯題

1. 同學報告「新方法把 loss 從 3.15 降到 3.08，改善 2.2%」。請列出你會提出的五個追問。
2. 同學的實驗中，A 組跑了 5 個種子、B 組跑了 2 個種子。請說明為什麼這個比較有問題。
3. 同學發現訓練集與驗證集的記憶化程度相同。請列出兩個可能原因。
4. 同學的分析腳本顯示「各次執行的 torch 版本不一致」的警告，但他認為小版本差異應該沒關係。請說明風險。
5. 同學掃描了 30 組超參數，在驗證集上選出最佳的一組，然後報告它在同一個驗證集上的分數。

請指出問題與正確做法。

6. 同學展示了五段很好的生成結果，但助教實際操作時品質明顯較差。請說明可能的原因與應有的做法。

四、系統設計題

1. 你的實驗室要建立一套「實驗可信度檢查表」，讓學生在提交任何實驗結論前必須通過。請設計至少十項檢查、各自的判準、以及若無法通過時的處置。這份檢查表要能在十分鐘內完成。
2. 你只有足夠跑 12 次訓練的算力，要比較三種架構改動（各有兩個選項，共八種組合）。請設計實驗方案：如何分配這 12 次、能得到什麼結論、以及有哪些問題無法回答。

五、臺灣情境與倫理題

1. 本章的系統案例會產出「詞彙表大小對繁中的影響」的實證結果。假設你的結論與國際文獻不符，請說明你會如何處理：如何檢查自己的實驗、如何比對設定差異、以及如何在報告中誠實表述。
2. 學術界對「只發表正面結果」的偏誤有長期批評。請討論：在臺灣的研究環境（升等壓力、計畫結案要求）下，如何鼓勵誠實報告負面結果？你認為指導老師與學生各有什麼責任？

評量提示（給自學者）

- 計算題第 1 題： $SE = \sigma\sqrt{(2/n)}$ 。n=3 時 0.065、n=5 時 0.051、n=10 時 0.036；可偵測效果分別約 0.131、0.101、0.072。
- 計算題第 2 題：要求 $2\sigma\sqrt{(2/n)} \leq 0.04$ ，解得 $n \geq 32$ 。這說明偵測小效果的成本極高。
- 計算題第 3 題： $1 - 0.95^{10} \approx 40.1\%$ 。
- 計算題第 4 題： $(1/32000)^{10} \approx 8.9 \times 10^{-46}$ 。這個機率小到不可能是巧合，因此逐字複述必然來自記憶。
- 計算題第 5 題：中位數 2.95、四分位距（2.91 至 2.97）= 0.06、全距 0.16。
- 計算題第 6 題：A 平均 3.143、B 平均 3.090，差異 0.053。兩組的標準差約 0.045 與 0.070，n=3 時標準誤約 0.048——差異與標準誤同量級，無法斷言有效。
- 除錯題第 1 題的五個追問：跑了幾個種子？信賴區間多少？跑了幾組設定才選出這個？測試集有沒有污染？其他分域有沒有退步？

研究延伸與版本注意事項

- 機器學習領域的重現危機：關於「報告的改進在重現時消失」有多篇系統性的調查，本章 16.2 與 16.3 節的論證多來自這些研究。

- 各大會議的 reproducibility checklist：它們是本章 16.7 節的直接參考，也是你投稿時會被要求填寫的東西。
- 預先註冊 (preregistration) 的實踐：源自臨床試驗與心理學，近年在機器學習領域逐漸被採用，值得了解其動機與限制。
- 記憶化的實證研究：關於模型規模、資料重複次數與逐字複述的關係，有多篇量化分析，對 16.5 節有直接補充。
- 超參數搜尋的統計方法：關於如何在有限預算下做出可靠的比較，有專門的研究方向值得參考。
- 本章的統計方法都是標準的、不會過時的。但工具鏈 (實驗追蹤平台、繪圖函式庫) 變動較快，程式碼以課程倉庫為準。

常見問題

問：跑三個種子的成本是三倍，真的值得嗎？答：換個角度想：跑一個種子的結論有 50% 機率是錯的，那個「成本」其實是零效益。如果算力真的不足，正確的做法是縮小實驗規模 (更小的模型、更短的訓練) 以換取多個種子，而不是放大單次實驗。可靠性優先於規模。

問：如果信賴區間跨越 0，是不是就代表沒有效果？答：不是。它代表「在這個實驗規模下無法區分」。可能真的沒效果，也可能效果太小或樣本太少。誠實的表述是後者，而且應該一併說明你的實驗能偵測多大的效果 (16.2.2 節)。

問：我的專題不做訓練，只做 RAG 或應用，需要這一章嗎？答：需要，而且同樣適用。你比較兩種檢索策略、兩種提示、兩個模型時，面對的是完全一樣的統計問題——多重比較、選擇偏誤、cherry-picking。第 30、36 章的評測會直接沿用本章的方法。

問：預先註冊會不會太僵化？如果實驗中途發現更好的想法呢？答：預先註冊不禁止探索，它只是要求你區分「事前假設」與「事後發現」。中途發現的東西可以寫成新的假設，用新的資料再驗證一次。這與 16.3.2 節的探索 / 確認分離是同一個精神。

教師使用建議

本章排在第 16 週，建議 2 小時講授加 1 小時實驗。本章沒有新技術，但它的觀念會改變學生做研究的方式，值得認真對待。

時段	內容	互動設計	注意事項
前 25 分鐘	16.1 至 16.2 節，假設與隨機性	現場跑 50.6% 的模擬	這個數字的震撼效果極強
中間 25 分鐘	16.3 節，多重比較	請學生猜跑 20 組的假發現機率	多數人會低估
後 30 分鐘	16.4 至 16.5 節，曲線診斷與記憶化	展示一組真實的異常曲線	讓學生先診斷再公布答案
最後 35 分鐘	16.6 至 16.8 節，自欺與報告	拿一篇論文的實驗段落找缺漏	找缺漏比講原則有效
實驗課 1 小時	程式 16A 與 16B	兩人一組互相審查對方的實驗設計	互審是最好的訓練

一個效果極佳的開場：課前準備兩份實驗報告，一份跑了一個種子並宣稱有 2.3% 的改進，另一份跑了三個種子並報告「信賴區間跨越 0，無法區分」。問學生哪一份比較好。多數人會選第一份——因為它有「結果」。然後公布 50.6% 的模擬，並說明第二份才是誠實的科學。

另一個建議：實驗課讓學生互相審查實驗設計（而非結果）。給每組一份別組的 DESIGN.yaml，要求他們找出三個問題。學生審別人時特別敏銳，而那些敏銳會回饋到自己的設計上。這比老師逐一批改有效得多。

高中授課的調整：16.1、16.2 完整教（假設與隨機性的觀念高中生完全可懂，而且對科展極有幫助）；16.3 只講「試很多次總會有一次成功」的直覺與那張機率表；16.4 挑兩三種曲線講；16.5 用「背答案」的比喻講記憶化；16.6 完整教（cherry-picking 的觀念對所有科學展示都適用）；16.7、16.8 簡化成「報告要寫限制」一句話。程式做 16B 的簡化版即可。

本章小結

1. 好的假設必須具體、可量測、可能為假；同時規定「要進步多少」與「不能退步多少」。
2. 應變數必須事前決定並預先註冊，否則事後挑選指標必然導致樂觀偏誤。
3. 在一個完全無效的改動上，單一種子有 50.6% 的機率顯示「變好了」。
4. 差異的標準誤為 $\sigma\sqrt{2/n}$ ；三個種子只能偵測 0.13 以上的效果，這是本書的最低要求。
5. 報告時應列出全部數值（ $n < 5$ ）或中位數與四分位距（ $n \geq 5$ ），並必須註明種子數。
6. bootstrap 信賴區間適合小樣本；區間跨越 0 時只能說「無法區分」，不能說「沒有差異」。
7. 跑 10 組設定有 40.1% 機率出現假發現，跑 20 組是 64.2%；應分離探索集與確認集，且確認時換新種子。

8. 無論如何都要誠實報告跑過的全部設定數量，讓讀者能自行判斷可信度。
9. 「驗證損失低於訓練損失」有四種原因，前兩種（dropout、移動平均）正常，後兩種（切分、污染）必須處置。
10. 分域曲線比總體曲線提供更多診斷資訊；某域完全不進步時應先確認問題在哪一層。
11. 逐字複述 10 個詞元的隨機機率約 10^{-46} ，因此幾乎可確定是記憶化；判準通常設在 10 個詞元以上。
12. 上取樣本質上是重複訓練，任何被上取樣的資料都必須做記憶化檢查。
13. 三種自欺是污染、選擇偏誤與 cherry-picking；展示樣本必須說明選取規則，並附完整紀錄。
14. 實驗套件應包含設計、指紋、執行紀錄、分析、產出物與重現腳本六層。
15. 報告的六個段落中，「限制」不可省略；負面結果有價值，但要區分「證明無效」與「無法證明有效」。
16. 失敗復盤的處置應分立即、短期、長期三層，且「學到什麼」要具體到能改進系統。

成果檢核

完成本章後你必須繳交：

- 實驗矩陣執行器（程式 16A）：通過六項驗收，含探索 / 確認分離與環境一致性檢查。
- 統計分析工具（程式 16B）：bootstrap 信賴區間、帶狀曲線圖、森林圖，數值與手算一致。
- 記憶化報告（程式 16C）：訓練集與驗證集的對照、上取樣資料的單獨統計、高風險樣本清單。
- 完整實驗套件：詞彙表大小實驗的九次訓練、統計分析、以及可執行的 reproduce.sh。
- 實驗報告：依 16.8.1 節的六段格式，限制段落必須具體到可偵測的效果大小。
- 一份失敗復盤：從你的訓練經驗中挑一次異常，依 16.8.3 節的格式撰寫。
- 實驗可信度檢查表：至少十項，可供你的專題與後續章節使用。

本章的工具與紀律會被第 17 章（尺度定律的擬合與外推）、第 20 章（架構選型的比較）、第 22 章（適配前後的回歸測試）、第 25 章（對齊稅的量測）與第 36 章（完整評測體系）直接使用。從這一章開始，你的每一個結論都應該能回答同一個問題：****如果別人重做一次，會得到同樣的答案嗎？****

第 17 章

尺度定律、資料模型配比與算力預算

Scaling Laws, Data-Model Balance, and Compute Budgets

難度：核心 / 進階 | 建議授課：第 17 週（第一學期末） | 硬體需求：A 級可完成擬合與預算計算；多尺寸實驗建議 B 級

叫獸開講

如果你手上只有 4 張 GPU 和三個月，你該訓練一個大模型配少量資料，還是小模型配大量資料？這個問題有數學答案，而且答案跟直覺相反——多數人第一次算完都會發現：自己原本打算訓練的模型太大了，而餵給它的資料太少。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明損失與參數量、資料量、算力之間的冪次關係，並解釋不可約損失的意義。
2. 推導給定算力預算下的最適參數量與資料量配比。
3. 說明為什麼推論成本主導時應該刻意「過度訓練」較小的模型。
4. 從小規模實驗擬合冪次曲線並外推，同時說明外推的風險與限制。
5. 在資料受限的情境下，判斷重複訓練的邊際效益與飽和點。
6. 把 FLOPs 換算成 GPU 小時、電費與新臺幣，做出可稽核的估算。
7. 為繁體中文語料稀缺的情境設計臺灣專屬的尺度化策略。
8. 撰寫一份含假設、誤差範圍與決策建議的算力預算書。
9. 比較自建叢集、國網中心與雲端租用三種方案的總持有成本。

本章的定位

這一章把前面十六章的所有量化工具接起來：第 3 章的 FLOPs 估算、第 7 章的 token 稅、第 12 章的參數量計算、第 15 章的 MFU、第 16 章的外推與誤差估計。它的產出是一份能拿去申請計畫、說服主管、或決定專題規模的預算書。如果你只能從本書帶走一份文件去說服別人，通常就是這一份。

17.1 尺度定律的基本形式

17.1.1 三個變數與一條曲線

尺度定律的核心觀察是：模型的損失與參數量、資料量、算力之間存在穩定的冪次關係，而且這個關

係跨越好幾個數量級都成立。

$$L(N) = E + A \cdot N^{(-\alpha)}$$

符號	意義	典型量級	說明
L	損失 (每詞元交叉熵)	1.5 至 4	第 5 章的指標
N	參數量 (不含嵌入層)	10^6 至 10^{11}	第 12.5 節的計算
E	不可約損失	約 1.5 至 2	語言本身的熵，見下文
A	尺度係數	依設定而異	需擬合
α	冪次指數	約 0.07 至 0.35	越大代表加參數的效益越高

17.1.2 不可約損失：一個重要的概念

E 是這條曲線的水平漸近線——無論你把模型放大到多少，損失都不會低於它。

第 3.3.6 節已經解釋過為什麼：語言本身有不確定性。「今天天氣很 _ 」後面接「好」「熱」「糟」「悶」都是合理的，即使一個完美的模型也無法確定是哪一個。那份不確定性就是 E。

這個概念有兩個實務意涵。第一，你不該期待 loss 趨近於零——第 13.7.3 節說過，驗證損失降到 1.5 以下應該先懷疑資料洩漏。第二，當你的損失已經接近 E 時，繼續加大模型的邊際效益極低，那個投資應該轉向別的地方（例如後訓練或檢索）。

對繁體中文的一個推測

E 的大小與語言、語域、以及 tokenizer 都有關。理論上，同樣的內容用效率較低的 tokenizer 切分後，每詞元的資訊量較少，E 也會較低——但那不代表模型更好（第 5.3.4 節的陷阱）。這也是為什麼跨 tokenizer 的尺度比較必須用 BPC 而非 PPL。本章的所有擬合都建議同時用兩種指標各做一次。

17.1.3 三條曲線與一條主線

除了對 N 的曲線，還有對資料量 D 與算力 C 的曲線，形式相同：

$$L(D) = E + B \cdot D^{(-\beta)} \quad ; \quad L(C) = E + K \cdot C^{(-\gamma)}$$

但真正有用的是把它們合起來看。因為在實務上你不會單獨增加 N 或 D——它們共同受算力預算 C 限制：

$$C \approx 6ND \quad (\text{第 3.2.5 節的訓練 FLOPs 公式})$$

於是問題變成一個最佳化問題：****給定 C，怎麼分配 N 與 D 才能讓 L 最小****？這就是下一節的主題，也是本章最實用的部分。

17.1.4 為什麼冪次關係會成立

尺度定律是一個實證發現，不是從第一原理推導出來的定理。但有幾個直覺可以幫助理解它為什麼合理。

角度	直覺	對應到哪個參數	限制
資料的結構	語言中的模式有各種頻率，越罕見的越難學	α 反映模式頻率的分布	不同語言可能不同
容量的邊際效益	前幾個參數學最常見的規律，後續逐步處理長尾	A 反映初始的學習效率	架構會影響
不可壓縮的部分	有些不確定性無法從上下文消除	E 是這部分的下限	與 tokenizer 有關

第一列給了一個有用的思考角度：如果語言中的模式頻率遵循類似 Zipf 的長尾分布，那麼「多花一倍的容量能多學到多少」就會呈現遞減的規律——而那正是冪次律的來源。

這也解釋了為什麼 α 在不同語言或語域上可能不同。一個結構高度重複的語域（例如公文）可能有較大的 α （加參數的效益高，因為模式好學），而一個高度多樣的語域（例如社群口語）則相反。這是一個值得在你自己的分域資料上驗證的假設——程式 17A 的延伸挑戰就是做這件事。

一個必須說清楚的界線

本節的直覺是為了幫助理解，不是嚴謹的推導。尺度定律至今仍是一個實證規律，它為什麼成立、在什麼條件下會失效，都還是開放的研究問題。所以本書的態度是：把它當成一個「在觀測範圍內非常可靠的經驗公式」來使用，而不是一個物理定律。這個區別在 17.5.4 節談外推風險時會變得重要。

17.2 最適配比

17.2.1 直覺

想像你有固定的預算。全部拿去買一個很大的模型，但只餵它一點點資料——它會欠訓練，很多參數學不到東西。反過來，用一個很小的模型吃海量資料——它的容量不足，資料再多也裝不進去。

最佳解在中間。而重要的實證發現是：****這個最佳解可以用一個簡單的比例來近似****。

$$D^* \approx 20 \times N^* \quad (\text{每個參數約配 20 個訓練詞元})$$

把它代入 $C = 6ND$ ，可以解出給定算力下的最適配置：

$$N^* = \sqrt{(C/120)} \quad ; \quad D^* = 20 N^*$$

17.2.2 實際算給你看

算力預算	總 FLOPs	最適參數量	最適訓練詞元
1 張 GPU × 1 天	9.07×10^{17}	約 87 M	約 1.74 B
1 張 GPU × 1 週	6.35×10^{18}	約 230 M	約 4.60 B
8 張 GPU × 1 月	2.18×10^{20}	約 1.35 B	約 26.9 B
1×10^{21} FLOPs	1.00×10^{21}	約 2.89 B	約 57.7 B

(假設單卡 30 TFLOPS、MFU 0.35。你自己的數字請用第 4 章程式 4B 與第 15.6 節的實測值。)

第一列給了一個很有用的參照：**一張消費級 GPU 跑一天，最適合訓練的是一個約 87 M 參數的模型，配 1.74 B 詞元**。這比多數學生的直覺小得多——他們往往想訓練 1 B 參數的模型，然後在資料不足的情況下訓練幾千步就停了。

17.2.3 偏離最適配比的代價

如果你堅持要訓練比最適值更大的模型，會發生什麼？

選擇	參數量	可用詞元	後果
最適	87 M	1.74 B (20×)	基準
過大 4 倍	348 M	0.43 B (1.25×)	嚴重欠訓練，多數參數無效
過小 4 倍	22 M	6.96 B (320×)	容量不足，資料浪費

第二列是實務上更常見的錯誤，因為「大模型」聽起來比較厲害。但一個訓練詞元數只有參數量 1.25 倍的模型，多數參數根本沒被有效訓練——你付了大模型的推論成本，卻只得到小模型的能力。

對臺灣的實務判斷是：**在算力受限時，寧可訓練一個較小但充分訓練的模型**。這與第 26 章的邊緣小模型策略是一致的，也與第 1.6 節的整體定位相符。

17.2.4 這個比例不是鐵律

20:1 是一個廣泛引用的近似值，但必須知道它的限制。

因素	如何影響最適比例	方向	章節
資料品質	高品質資料的每個詞元價值更高	可降低 D	第 9 章
資料重複	重複的詞元效益遞減	需提高有效 D	17.4
tokenizer 效率	影響同樣內容的詞元數	間接影響	第 7 章
架構差異	MoE 等稀疏架構有不同的比例	需另行擬合	第 20 章
推論成本	若推論主導，應刻意偏離	大幅提高 D	17.3
後續的後訓練	會做微調的模型可略小	略降 N	第 22–25 章

最後一列對本書的讀者特別重要：如果你的模型之後會經過持續預訓練與指令微調（第六篇的主線），預訓練階段的最適配比可能與「訓練完就直接用」的情況不同。目前這方面的實證還不夠充分，本書的建議是：先按 20:1 規劃，並在預算中保留餘裕給後訓練。

17.2.5 批次大小與學習率也有尺度關係

第 15 章講過批次與學習率的選擇，但沒說它們與模型規模的關係。實務上這兩者也需要隨規模調整。

參數	隨模型放大的趨勢	直覺	章節
批次大小	增大	大模型的梯度雜訊相對較小，可用更大批次	第 15.1.3 節
峰值學習率	減小	參數多時每步的更新應更謹慎	第 15.2.4 節
warmup 步數	大致固定或略增	主要取決於最佳化器的動量累積	第 15.2.1 節
梯度裁切閾值	通常不變	1.0 在各種規模下都堪用	第 15.4.2 節

第二列有一個常被引用的經驗關係：學習率大致與模型寬度成反比。這意味著你不能把小模型調好的

學習率直接套用到大模型——那幾乎一定會發散。

實務上的做法是：在做尺度擬合的多個尺寸上，同時記錄各自的最適學習率，然後也對它擬合一條曲線。這樣外推到目標規模時，你不只知道「預期損失是多少」，也知道「該用什麼學習率」。程式 15B 的延伸挑戰就是在做這件事，本章的程式 17A 可以直接沿用那些結果。

一個容易忽略的實驗設計問題

如果你在做尺度擬合時，每個尺寸都用同一個學習率，那麼你量到的就不是「純粹的規模效應」，而是「規模效應加上學習率不匹配的懲罰」——而後者對大模型的傷害更大，會讓你的 α 被低估。正確做法是每個尺寸都用它自己的最適學習率。這是第 16.1.2 節「控制變數」原則的一個微妙應用：這裡要控制的不是「學習率數值相同」，而是「學習率同樣接近最適」。

17.3 推論主導時：刻意過度訓練

17.3.1 訓練是一次性的，推論是每天的

17.2 節的最適配比只考慮訓練成本。但一個要長期服務的模型，總成本包含推論——而推論成本正比於參數量。

$$\text{訓練 FLOPs} \approx 6ND \quad ; \quad \text{推論 FLOPs} \approx 2N \times \text{服務的總詞元數}$$

所以如果你預期會服務大量請求，用一個較小但過度訓練的模型可能總成本更低——它的訓練較貴（因為餵了更多資料），但每一次推論都便宜。

17.3.2 一個完整的比較

比較兩個選擇：一個 7 B 參數配 140 B 詞元（接近 20:1），與一個 3 B 參數配 600 B 詞元（200:1，刻意過度訓練）。

方案	訓練 FLOPs	服務 10^{13} 詞元的推論	總計
7 B $\times$ 140 B	5.88×10^{21}	1.40×10^{23}	1.46×10^{23}
3 B $\times$ 600 B	1.08×10^{22}	6.00×10^{22}	7.08×10^{22}

在服務 10 兆個詞元的情境下，過度訓練的 3 B 模型總成本只有 7 B 的一半不到——即使它的訓練成本是後者的近兩倍。

損益兩平點可以直接解出來：

$$\text{服務量 } T^* = (\text{訓練}_2 - \text{訓練}_1) / (2 \times (N_1 - N_2))$$

代入數字得到 $T^* \approx 6.15 \times 10^{11}$ ，也就是**約 6,150 億個詞元**。超過這個服務量，過度訓練的小模型就比較划算。

17.3.3 什麼時候該過度訓練

情境	推論量	建議	理由
研究或一次性實驗	極低	按 20:1	訓練成本主導
內部工具、少數使用者	低至中	略微過度訓練 (40:1)	折衷
對外服務、長期營運	高	大幅過度訓練	推論主導
邊緣部署	不限但硬體受限	盡量小 + 大量訓練	第 26 章

最後一列連結到第 26 章：邊緣裝置的記憶體與功耗是硬限制，所以模型大小不是選擇而是約束。在這種情況下，「用足夠的資料把一個小模型訓練到極致」幾乎是唯一的路——而尺度定律告訴你這條路能走多遠。

對臺灣而言，第三與第四列是主要的情境。我們的優勢在應用與邊緣（第 1.7 節），而那些場景幾乎都是推論主導的。這意味著臺灣的模型訓練策略應該系統性地偏向「小而充分訓練」，而不是追求參數量的數字。

17.3.4 推論成本的完整計算

17.3.2 節只算了 FLOPs，但實際的推論成本還有兩個成分：記憶體與延遲。

成分	與參數量的關係	實際限制	章節
計算量	正比於 N	通常不是瓶頸	第 14.6.2 節
權重記憶體	正比於 N	決定能不能放進單卡	第 35 章
KV cache	正比於層數與頭數	決定併發數	第 14.5.4 節
延遲	受頻寬主導，正比於 N	決定使用體驗	第 14.6.2 節

第二列往往才是真正的硬限制。一個 7 B 模型用 BF16 需要 14 GB 權重記憶體，在一張 16 GB 的卡上幾乎放不下（還要留給 KV cache 與 activation）；而一個 3 B 模型只要 6 GB，輕鬆容納。這個差別不是「比較貴」，而是「能不能部署」。

所以過度訓練小模型的好處不只是省 FLOPs，還包括：能部署在更便宜的硬體上、能支援更多併發、延遲更低。把這些一起算進去，17.3.2 節那個 6,150 億詞元的損益兩平點其實會更早到來。

$$\text{總擁有成本} \approx \text{訓練成本} + \text{服務量} \times \text{每詞元推論成本} + \text{硬體門檻的機會成本}$$

最後一項難以量化，但它常常是決定性的。如果選擇 3 B 讓你能用現有的硬體部署，而 7 B 需要採購新設備，那個差額可能遠超過所有的 FLOPs 計算。第 26 章談邊緣部署時會把這個考量完整展開。

17.4 資料受限：重複訓練的邊際效益

17.4.1 臺灣面對的實際限制

前面的討論假設資料是無限的——你需要多少就有多少。但第 1.6.3 節與第 9 章已經說明，繁體中文的高品質語料規模遠小於英文。

這帶來一個具體的問題：如果最適配比要求 20 B 詞元，而你只有 5 B，該怎麼辦？

選項	做法	代價	章節
重複訓練	同樣的資料跑多輪	效益遞減、記憶化	本節
縮小模型	按可用資料量反推 N	能力上限降低	17.2
擴充資料	取得更多語料	時間與授權成本	第 8 章
混入其他語言	用英文或程式碼補足	繁中比例下降	第 9.5 節
合成資料	用模型生成	品質與崩壞風險	第 9.6 節

17.4.2 重複的邊際效益

實證研究顯示：重複訓練的資料，其效益隨重複次數遞減，但在前幾輪仍相當可觀。

重複輪數	相對於新資料的效益	記憶化風險	本書建議
1 輪 (全新)	100%	低	基準
2 至 4 輪	約 70% 至 90%	略升	可接受
5 至 15 輪	逐步下降	明顯上升	需監控
超過 15 輪	接近零	高	不建議

(數值為文獻中報告的量級，實際依資料品質與模型規模而異，應在你的設定上實測。)

一個實用的判斷方法是把重複的資料折算成「有效詞元數」，再代回 17.2 節的公式。例如你有 5 B 詞元、打算跑 4 輪，有效詞元數大約是 $5 \times (1 + 0.9 + 0.8 + 0.7) \approx 17 \text{ B}$ ，而非 20 B。

17.4.3 繁中的尺度化策略

把前面的分析整合成一套適用於臺灣情境的建議。

步驟	做什麼	依據	章節
1	盤點可用的高品質繁中語料	第 8 章的清冊	第 8、9 章
2	扣除去重與去污染的損失	通常剩 50% 至 70%	第 9.7.3 節
3	決定可接受的重複輪數	2 至 4 輪，配合記憶化監控	17.4.2、第 16.5 節
4	算出有效詞元數	折算後的總量	17.4.2
5	用 $D^* = 20 N^*$ 反推模型大小	這是資料決定的上限	17.2.1
6	與算力預算取交集	取兩者的較小值	17.5
7	若考慮推論成本則再縮小	過度訓練策略	17.3

第五步是關鍵的心態轉換：****在臺灣，模型大小往往是被資料決定的，而不是被算力決定的****。這與國際大廠的情況相反——他們資料充足，受限於算力。認清這個差異，你的規劃才會務實。

一個具體的例子：如果你有 8 B 詞元的高品質繁中語料，去重後剩 5 B，跑 3 輪折算成約 13 B 有效詞元，那麼由資料決定的最適模型大小約為 650 M 參數。如果你的算力允許訓練 2 B 的模型，那多出來的算力應該用在別的地方——例如更多的後訓練、更好的評測、或訓練多個小模型做比較。

17.4.4 合成資料能否補足

第 9.6 節談過合成資料的三個限制。在尺度定律的框架下，還有一個更根本的問題值得說明。

尺度定律中的 D 假設的是**獨立的、帶有新資訊的資料**。用模型生成的資料，其資訊量上限受限於生成它的模型——它無法憑空創造出教師模型不知道的知識。所以在預訓練階段，合成資料的「有效詞元數」遠低於它的名目數量。

階段	合成資料的效果	為什麼	章節
預訓練	效益有限，建議上限 10%	無法帶來新的世界知識	第 9.6.1 節
指令微調	效益高	學的是格式與行為而非知識	第 23 章
偏好對齊	效益高	同上	第 25 章
蒸餾小模型	效益高	目標本來就是複製教師	第 26 章

這張表給出一個清楚的實務判斷：**合成資料不能解決預訓練的資料短缺，但能大幅降低後訓練的資料成本**。對臺灣而言，這意味著繁中預訓練語料的不足無法靠生成來繞過，必須真的去建置（第 8 章）——但指令與偏好資料的建置成本可以顯著降低。

這也解釋了為什麼本書的第六篇（後訓練）比第二篇（語料）樂觀：前者的資料瓶頸有技術解，後者沒有。

17.5 從小規模外推

17.5.1 為什麼要外推

你不可能為了決定「該訓練多大的模型」而先訓練好幾個大模型——那就本末倒置了。實務上的做法是：在小規模上跑幾組實驗、擬合冪次曲線、再外推到目標規模。

這正是尺度定律最有價值的地方：它讓你能用便宜的實驗預測昂貴的結果。

17.5.2 擬合的步驟

1. 選擇至少四到五個模型尺寸，最大與最小之間跨越至少一個數量級。
2. 每個尺寸都按最適配比訓練（用 17.2 節的公式決定訓練詞元數）。
3. 固定所有其他變因：tokenizer、資料版本、架構元件、學習率排程規則。
4. 每個尺寸至少跑兩到三個種子（第 16.2 節的要求）。

5. 在對數空間擬合 $L - E$ 對 N 的直線，得到 α 與 A 。
6. 外推到目標規模，並報告誤差範圍。

第五步的細節：因為 $L = E + A \cdot N^{(-\alpha)}$ ，兩邊減 E 再取對數得到 $\log(L - E) = \log A - \alpha \cdot \log N$ ，這是一條直線。問題是 E 未知——實務上有兩種做法：三參數同時擬合（較準但需要更多資料點），或先假設一個 E 再檢查殘差（較簡單）。

formosa/analysis/scaling.py

```
import numpy as np
from scipy.optimize import curve_fit

def power_law(N, E, A, alpha):
    return E + A * np.power(N, -alpha)

def fit_scaling(Ns, losses, p0=(1.8, 100.0, 0.3)):
    """三參數擬合。Ns 與 losses 應為同尺寸的陣列"""
    Ns = np.asarray(Ns, float)
    losses = np.asarray(losses, float)
    assert len(Ns) >= 4, "至少需要四個資料點才值得擬合"

    popt, pcov = curve_fit(power_law, Ns, losses, p0=p0, maxfev=20000)
    E, A, alpha = popt
    perr = np.sqrt(np.diag(pcov))          # 參數的標準誤

    pred = power_law(Ns, *popt)
    resid = losses - pred
    ss_res = float((resid ** 2).sum())
    ss_tot = float(((losses - losses.mean()) ** 2).sum())

    return {
        "E": float(E), "A": float(A), "alpha": float(alpha),
        "E_err": float(perr[0]), "alpha_err": float(perr[2]),
        "r_squared": 1 - ss_res / ss_tot,
        "max_residual": float(np.abs(resid).max()),
        "n_points": len(Ns),
        "fit_range": (float(Ns.min()), float(Ns.max())),
    }

def extrapolate(fit, N_target, warn_factor=10.0):
    """外推並附上風險提示"""
    lo, hi = fit["fit_range"]
    ratio = N_target / hi
    pred = power_law(N_target, fit["E"], fit["A"], fit["alpha"])

    # 用 alpha 的標準誤估計外推的不確定性
    hi_a = power_law(N_target, fit["E"], fit["A"],
                     fit["alpha"] - fit["alpha_err"])
    lo_a = power_law(N_target, fit["E"], fit["A"],
                     fit["alpha"] + fit["alpha_err"])
```

```

return {
    "predicted_loss": float(pred),
    "range": (float(min(lo_a, hi_a)), float(max(lo_a, hi_a))),
    "extrapolation_factor": float(ratio),
    "warning": (f"外推倍數 {ratio:.1f}× 超過建議上限 {warn_factor}×，"
               f"預測可信度低" if ratio > warn_factor else None),
}

```

17.5.3 一個實測範例

用一組模擬資料驗證這套流程（真值 $E = 1.8$ 、 $A = 500$ 、 $\alpha = 0.35$ ，加入 1% 雜訊）：

參數	真值	擬合值	評價
α	0.35	0.354	誤差 1.1%
A	500	531	誤差 6.2%

再用擬合結果外推到訓練範圍之外：

外推目標	預測損失	真值	絕對誤差
N = 300 M	2.3331	2.3395	0.0064
N = 1,000 M	2.1482	2.1540	0.0057

（擬合資料的最大規模為 80 M，所以這是 3.75 倍與 12.5 倍的外推。）

誤差不到 0.007，看起來非常準——但這是**在模擬資料上**。真實情況會差得多，因為真實的損失曲線未必嚴格遵守冪次律，而且訓練還有其他變因。這帶我們到下一節。

17.5.4 外推的風險

風險	為什麼發生	徵兆	處置
曲線在大規模轉折	冪次律未必無限適用	無法事前得知	限制外推倍數
超參數需隨規模調整	學習率、批次都有尺度關係	大模型訓練不穩	同步調整並記錄

風險	為什麼發生	徵兆	處置
資料不足以支撐	大模型需要更多資料	欠訓練	先檢查 D 是否跟上
擬合點太少或太集中	曲線被少數點主導	R^2 高但殘差有系統性	看殘差圖而非只看 R^2
E 估計不準	三參數擬合對 E 敏感	不同 E 假設給出不同 α	報告多組 E 的結果

本書的建議是：****外推不超過擬合範圍的十倍****，並且一律報告不確定範圍而非單一數字。程式碼中的 `warn_factor` 就是為此設計的。

第四列的「看殘差圖而非只看 R^2 」值得強調。冪次律的 R^2 通常都很高（0.99 以上），因為資料本來就跨越好幾個數量級。真正有診斷價值的是殘差有沒有系統性的形狀——如果殘差呈現曲線而非隨機散布，那代表你的模型形式不對。

17.5.5 IsoFLOP 曲線：另一種擬合方式

17.5.2 節的做法是固定配比、變動規模。另一種常見的做法是反過來：****固定算力預算，變動配比****，看哪個配比最好。

具體做法是：選定幾個算力預算（例如 10^{17} 、 10^{18} 、 10^{19} FLOPs），在每個預算下訓練多個不同 N 與 D 的組合（但 $6ND$ 相同），畫出「N 對最終損失」的曲線。這條曲線通常是 U 形，最低點就是該預算下的最適 N。

方法	做法	優點	缺點
固定配比法	每個尺寸都用 20:1，變動規模	實驗數少、實作簡單	假設了配比是對的
IsoFLOP 法	固定算力，掃描配比	不預設配比，直接量出最適值	實驗數多好幾倍
兩者並用	先 IsoFLOP 確認配比，再擬合規模	最可靠	成本最高

第二列是 Chinchilla 那類研究採用的方法，也是「20:1」這個數字的來源——它不是推導出來的，而是從 U 形曲線的最低點量出來的。

對本書的讀者而言，實務建議是：如果你的算力只夠做一種，用固定配比法（並接受 20:1 的假設）；

如果你懷疑自己的情境（例如繁中語料、特殊架構）不適用標準配比，那就值得花成本做一組小規模的 IsoFLOP 掃描來驗證。

IsoFLOP 掃描的設定產生器

```
def isoflop_configs(budget_flops, n_points=5,
                   ratio_range=(5, 80)):
    """固定算力預算，產生不同配比的設定"""
    import numpy as np
    ratios = np.geomspace(ratio_range[0], ratio_range[1], n_points)
    configs = []
    for r in ratios:
        # C = 6ND 且 D = r*N => N = sqrt(C/(6r))
        N = (budget_flops / (6 * r)) ** 0.5
        configs.append({
            "tokens_per_param": round(float(r), 1),
            "n_params": round(N),
            "n_tokens": round(N * r),
            "flops": budget_flops,
        })
    return configs

def find_optimal_ratio(results):
    """從 U 形曲線找最低點；若最低點在端點則警告範圍不足"""
    best = min(results, key=lambda r: r["final_loss"])
    idx = results.index(best)
    warn = None
    if idx in (0, len(results) - 1):
        warn = "最佳值落在掃描範圍的端點，應擴大 ratio_range 重測"
    return {"optimal_ratio": best["tokens_per_param"],
            "optimal_n": best["n_params"], "warning": warn}
```

find_optimal_ratio 中的端點警告很重要：如果 U 形曲線的最低點落在你掃描範圍的邊緣，那你根本沒找到真正的最低點——只是找到了範圍內最好的。這是一個容易被忽略的實驗設計錯誤。

17.6 從 FLOPs 到新臺幣

17.6.1 完整的換算鏈

前面的計算都以 FLOPs 為單位，但你要拿去申請計畫或說服主管時，需要的是時間與金錢。

步驟	公式	需要的輸入	章節
1	$\text{FLOPs} = 6 \times N \times D$	模型大小、訓練詞元數	第 3.2.5 節
2	有效算力 = 峰值 $\times$ MFU	硬體規格、實測 MFU	第 15.6 節
3	GPU 秒 = FLOPs $\div$ 有效算力	—	—
4	GPU 小時 = GPU 秒 $\div$ 3600	—	—

步驟	公式	需要的輸入	章節
5	電費 = GPU 小時 × 功耗 × 電價	實測功耗、電價	第 21 章
6	雲端費用 = GPU 小時 × 時價	報價	第 21 章
7	總持有成本 = 硬體攤提 + 電費 + 維運	三年期估算	第 1.4.4 節

formosa/analysis/budget.py

```

from dataclasses import dataclass

@dataclass
class ComputeBudget:
    peak_tflops: float          # 單卡峰值 (BF16)
    mfu: float                  # 實測值，見第 15.6 節
    n_gpus: int = 1
    power_watts: float = 350.0  # 單卡實測功耗
    pue: float = 1.4            # 資料中心的電力使用效率
    price_per_kwh: float = 3.0  # 新臺幣
    cloud_price_per_gpu_hour: float | None = None

    def effective_flops_per_sec(self) -> float:
        return self.peak_tflops * 1e12 * self.mfu * self.n_gpus

    def train_flops(self, n_params: int, n_tokens: int) -> float:
        return 6.0 * n_params * n_tokens

    def estimate(self, n_params: int, n_tokens: int) -> dict:
        flops = self.train_flops(n_params, n_tokens)
        seconds = flops / self.effective_flops_per_sec()
        wall_hours = seconds / 3600
        gpu_hours = wall_hours * self.n_gpus

        kwh = gpu_hours * self.power_watts / 1000 * self.pue
        electricity = kwh * self.price_per_kwh
        cloud = (gpu_hours * self.cloud_price_per_gpu_hour
                 if self.cloud_price_per_gpu_hour else None)

        return {
            "flops": flops,
            "petaflops": round(flops / 1e15, 2),
            "wall_hours": round(wall_hours, 2),
            "wall_days": round(wall_hours / 24, 2),
            "gpu_hours": round(gpu_hours, 1),
            "kwh": round(kwh, 1),
            "electricity_twd": round(electricity),
            "cloud_twd": round(cloud) if cloud else None,
        }

    def max_model_for_time(self, wall_hours: float,
                           tokens_per_param: float = 20.0) -> dict:

```

```

"""反向計算：給定時間，最適的模型大小是多少"""
C = self.effective_flops_per_sec() * wall_hours * 3600
n = (C / (6 * tokens_per_param)) ** 0.5
return {"budget_flops": C, "n_params": round(n),
        "n_tokens": round(n * tokens_per_param)}

```

17.6.2 三種取得算力的方式

方式	優點	缺點	適用
自建	長期成本低、資料不外流、隨時可用	前期投入大、需維運人力	長期且持續使用
國家級算力中心	單位成本低、規模大	需申請、排隊、有使用規範	大型專案與研究
雲端租用	彈性、免維運、可隨時擴縮	長期昂貴、資料須評估外流	短期實驗與尖峰

第 21 章會完整處理臺灣的算力生態，包括國網中心的申請條件與實務限制。本章只提供計算工具——但有一件事值得先說：**三種方式的損益兩平點通常在數個月的量級**（第 1.4.4 節算過一個例子）。如果你的專案會持續超過半年且用量穩定，自建通常較划算；短期或用量不確定，雲端較安全。

17.6.3 預算書必須包含的十項

1. 目標：要訓練什麼模型、達到什麼能力、如何驗收。
2. 模型規格：參數量、架構、tokenizer，以及它們的依據（第 12、17.2 節）。
3. 資料規格：可用詞元數、去重後的有效量、重複輪數（第 9、17.4 節）。
4. 算力估算：FLOPs、MFU 依據、GPU 小時、牆鐘時間。
5. 硬體方案：自建、國網或雲端，含比較與選擇理由。
6. 成本明細：硬體、電力、雲端費用、人力，以及三年期的總持有成本。
7. 時程：含實驗、正式訓練、後訓練、評測各階段。
8. 風險與餘裕：發散重跑、硬體故障、資料延誤，建議保留 20% 至 30% 的餘裕。
9. 假設清單：每一個估算所依據的假設，以及它們的不確定範圍。
10. 決策建議：在什麼條件下該調整規模或改變方案。

第八項的「餘裕」是新手最常漏掉的。真實的訓練會發散、會 OOM、會因為某個 bug 而重跑——如果你的預算剛好夠跑一次，那你實際上跑不完。本書建議至少保留 20% 到 30%，而且要在預算書中明確列出，而非偷偷藏在數字裡。

第九項的假設清單則是誠實的表現。「我們假設 MFU 為 0.35，這來自第 15 章在同型號 GPU 上的實測；若實際為 0.25，時程將延長 40%」——這樣的寫法讓審查者能判斷風險，遠比一個看起來很確定的數字有價值。

17.6.4 一份預算書的實際範例

把前面的工具用在一個具體案例上：某科技大學想訓練一個 500 M 參數的繁中模型，用於校務與教學應用。

項目	數值	依據	不確定性
目標模型	500 M 參數	由資料量反推 (17.4.3 節)	低
訓練詞元	10 B (20:1)	17.2.1 節公式	低
可用語料	4 B 詞元 (去重後)	第 9 章語料報表	中 (授權待確認)
重複輪數	3 輪，折算 10.8 B	17.4.2 節係數	中
總 FLOPs	3.24×10^{19}	6ND (用折算後的 10.8 B)	低
硬體	2 張 GPU、30 TFLOPS	實驗室現有	低
MFU	0.32	第 15 章實測	中 (± 0.08)
牆鐘時間	約 19.5 天	換算	中
加 25% 餘裕	約 24.4 天	17.6.3 節第八項	—
電費	約 1,700 元	$350 \text{ W} \times \text{PUE } 1.4 \times 3 \text{ 元}$	低

(本表為教學範例，數值依第 17.6.1 節的換算鍵計算。實際專案請用你自己實測的 MFU 與功耗。)

這份估算的結論是：在現有硬體上，這個規模的訓練約需三週半 (含餘裕)，電費不到兩千元——完全在一個大學實驗室的能力範圍內。這正是本章想傳達的訊息之一：許多學生以為「訓練模型」遙不可及，但算完之後會發現，在合理的規模上它是可行的。

真正的瓶頸往往不在算力，而在第三列：可用語料只有 4 B 詞元，且授權狀態待確認。這與 17.4.3 節的結論一致——**在臺灣，資料才是綁住你的那條線**。

17.6.5 敏感度分析：哪些假設最關鍵

一份好的預算書要能回答「如果我的假設錯了會怎樣」。

假設	若偏離	時程影響	應對
MFU 0.32 → 0.22	低 31%	從 24.4 天增至 35.5 天	需重新評估時程
語料 4 B → 2.5 B	少 37%	模型須縮至 400 M	需重新規劃或補資料
重複折算係數過樂觀	有效詞元減少	同上	需做程式 17C 的實測
發散重跑一次	多一次完整訓練	增加約 19.5 天	餘裕不足，需提高至 50% 以上

第四列是一個值得注意的發現：25% 的餘裕不足以吸收「完整重跑一次」。如果你的訓練有相當機率發散（例如你在用較激進的學習率或未經驗證的架構），餘裕應該提高到 50% 甚至 100%。第 16.6.4 節的發散率統計在這裡有實際用途——它讓你能量化這個風險。

17.7 尺度定律的限制

17.7.1 它預測什麼、不預測什麼

能預測	不能預測	為什麼	章節
預訓練損失	下游任務表現	損失與任務表現非線性相關	第 5.5.5 節
整體趨勢	特定能力何時出現	能力的量測本身有問題	第 1.2.5 節
同一設定下的外推	架構改變後的行為	每種架構有自己的曲線	第 20 章
資料充足時	資料受限時的行為	需另行修正	17.4

第一列是最重要的限制。你可以用尺度定律精確預測「模型放大十倍後的損失」，但那不直接告訴你「它能不能正確回答臺灣的升學制度問題」。損失是一個聚合指標，而下游能力是特定的。

第 1.2.5 節討論過的湧現爭議與此相關：某些能力看起來是「突然出現」的，但那部分是評測指標造成的錯覺。無論如何，損失曲線的平滑外推無法告訴你某個具體能力何時可用。

17.7.2 本書的立場

尺度定律是一個**規劃工具**，不是一個**保證**。它的正確用法是：

- 用它決定資源配置（模型多大、資料多少），這是它最可靠的用途。
- 用它做粗略的可行性判斷（「這個目標需要的算力是我們的一百倍」）。
- 不要用它承諾具體的能力（「訓練完就能達到某某分數」）。
- 一律報告不確定範圍，並註明擬合的資料範圍與外推倍數。
- 在關鍵決策前，用一個中等規模的實驗驗證外推是否成立。

最後一項值得堅持。如果你的預算書建議訓練一個 1 B 的模型，而擬合只做到 80 M，那麼在投入全部資源之前，先跑一個 300 M 的驗證——如果它落在預測範圍內，你對外推就有信心；如果偏離，那你省下了大部分的資源。

17.7.3 尺度定律之外的三個變數

本章的公式只涵蓋參數量、資料量與算力。但實際決定模型好壞的還有其他因素，而它們不在尺度定律的框架內。

變數	影響	為什麼不在公式中	該怎麼辦
資料品質	可能等同於數倍的資料量	難以量化	第 9 章的過濾與配比
tokenizer 效率	直接改變同樣內容的詞元數	被吸收進 D 中	用 BPC 而非 loss 比較
後訓練	對下游能力的影響可能大於預訓練規模	完全不同的階段	第六篇
檢索增強	知識可外掛而非內建	改變了問題本身	第 30 章

第三與第四列對臺灣的實務特別重要。如果你的目標是「回答校務問題」，那麼把預算花在 RAG（第 30 章）與指令微調（第 23 章）上，效益幾乎一定高於把預訓練模型放大一倍。尺度定律告訴你「放大一倍能讓損失降多少」，但它不會告訴你「還有別的方法能達到同樣的目的」。

所以本章的最後一個提醒是：****先問「這個問題該用什麼方法解」，再問「這個方法需要多少算力」****。順序顛倒的話，你可能會非常精確地規劃一件不該做的事。

17.7.4 什麼時候不該用尺度定律

- 當你的目標是特定的下游能力而非整體語言建模品質時——用第 36 章的評測而非損失外推。
- 當你在比較架構時——不同架構有不同的曲線，不能用同一組係數（第 20 章）。
- 當你的資料嚴重受限且必須大量重複時——標準公式會高估可達到的效果（17.4）。

- 當你在做後訓練規劃時——微調的資料需求與預訓練完全不同 (第 23、24 章) 。
- 當外推倍數超過十倍時——超出可信範圍，應先做中等規模驗證 (17.5.4) 。

程式實作

本章三個實作的產出是一份可以直接使用的預算書。它會在第 22 章規劃持續預訓練、第 26 章決定小模型規格、第 40 章的總整專題中被反覆使用。

程式 17A 尺度定律擬合器

目標：用多組小規模實驗擬合冪次曲線，外推到目標規模，並正確報告不確定性。

步驟	做什麼	驗收標準	節次
資料收集	至少 4 個尺寸 × 3 個種子	跨越一個數量級以上	17.5.2
擬合	三參數 <code>curve_fit</code>	$R^2 > 0.99$ 且殘差無系統性	17.5.2
殘差圖	畫出殘差對 $\log N$	隨機散布而非曲線	17.5.4
外推	預測目標規模並給範圍	含外推倍數警告	17.5.2
驗證	在模擬資料上測試流程	參數誤差小於 10%	17.5.3

必須先在模擬資料上驗證你的擬合程式碼——本書提供的參考值是：真值 $\alpha = 0.35$ 時應擬合出 0.354 左右，外推 12.5 倍的絕對誤差應小於 0.01。如果你的實作在模擬資料上就不準，那在真實資料上的結果更不可信。

延伸挑戰：分別用 `loss` 與 `BPC` 各擬合一次（第 5.3.5 節的三數字慣例）。兩者的 α 是否相同？如果不同，那代表什麼？（提示：與 `tokenizer` 效率有關）

程式 17B 算力預算計算器

目標：實作 17.6.1 節的完整換算鏈，並支援雙向計算——從模型規格算成本，或從預算反推模型規格。

scripts/budget_report.py (節錄)

```
import argparse, json
from formosa.analysis.budget import ComputeBudget

def full_report(budget: ComputeBudget, n_params, n_tokens,
                reserve_ratio=0.25):
    base = budget.estimate(n_params, n_tokens)

    # 加上餘裕 (17.6.3 節第八項)
    with_reserve = {k: (round(v * (1 + reserve_ratio), 2))
```

```

        if isinstance(v, (int, float)) else v)
    for k, v in base.items()}

# 敏感度分析：MFU 若偏離預期
sensitivity = {}
for mfu in [budget.mfu * 0.7, budget.mfu, budget.mfu * 1.2]:
    b2 = ComputeBudget(**{**budget.__dict__, "mfu": mfu})
    sensitivity[f"mfu_{mfu:.2f}"] = b2.estimate(n_params,
                                                n_tokens)["wall_days"]

return {
    "assumptions": {
        "peak_tflops": budget.peak_tflops,
        "mfu": budget.mfu,
        "n_gpus": budget.n_gpus,
        "power_watts": budget.power_watts,
        "pue": budget.pue,
        "price_per_kwh": budget.price_per_kwh,
        "reserve_ratio": reserve_ratio,
    },
    "base_estimate": base,
    "with_reserve": with_reserve,
    "sensitivity_wall_days": sensitivity,
}

def reverse_plan(budget: ComputeBudget, available_days: float,
                 available_tokens: int, repeat_epochs: int = 1):
    """雙向約束：算力與資料哪個先撞牆？"""
    by_compute = budget.max_model_for_time(available_days * 24)

    # 資料端的限制（見 17.4.2 節的折算）
    decay = [1.0, 0.9, 0.8, 0.7, 0.6][:repeat_epochs]
    effective_tokens = available_tokens * sum(decay)
    by_data = {"n_params": round(effective_tokens / 20),
               "effective_tokens": round(effective_tokens)}

    binding = ("compute" if by_compute["n_params"] < by_data["n_params"]
               else "data")
    return {"by_compute": by_compute, "by_data": by_data,
            "binding_constraint": binding,
            "recommended_n_params": min(by_compute["n_params"],
                                         by_data["n_params"])}

```

`reverse_plan` 中的 `binding_constraint` 是這支程式最有價值的輸出。它明確告訴你「你被算力綁住還是被資料綁住」——而這決定了你該去爭取更多 GPU 還是去找更多語料。第 17.4.3 節說過，臺灣的情況通常是後者。

程式 17C 資料受限的模擬

目標：實測重複訓練的邊際效益，找出你的資料與模型的飽和點。

實驗設計：

- 固定一個小模型（tiny 或 small），固定總訓練詞元數。

- 變因：資料集大小。用 100%、50%、25%、12.5% 的語料，分別跑 1、2、4、8 輪達到相同的總詞元數。
- 量測：驗證損失、BPC、以及第 16.5 節的記憶化程度。
- 每組至少三個種子 (第 16.2 節)。
- 畫出「重複輪數對驗證損失」與「重複輪數對記憶化」兩條曲線。

必須回答的四個問題：

- 在你的設定下，重複幾輪之後驗證損失不再改善？那就是飽和點。
- 記憶化在第幾輪開始明顯上升？它與飽和點的關係是什麼？
- 把重複的效益折算成「有效詞元數」，你的折算係數是多少？與 17.4.2 節的參考值相比如何？
- 如果你只有一半的語料，你會選擇「跑兩輪」還是「訓練一半大的模型」？用你的資料支撐這個決定。

一個實務上的重要觀察

這個實驗在小模型上做，結論未必能直接套用到大模型——大模型的記憶容量更大，重複的風險更高。但它建立的方法論是通用的：用可負擔的實驗量出你自己的折算係數，而不是抄文獻的數字。這與第 16 章的精神一致。

系統案例：臺灣本土 3B 模型的算力預算書

本章的系統案例是撰寫一份完整的算力預算書：假設你要為臺灣訓練一個 3 B 參數的本土模型，比較自建叢集、國網中心與雲端租用三種方案，並提出可執行的建議。

必須回答的五個核心問題

問題	需要什麼資料	本書的工具	節次
資料夠嗎	可用繁中語料的有效詞元數	第 9.7.3 節的語料報表	17.4
算力要多少	FLOPs、GPU 小時、牆鐘時間	程式 17B	17.6.1
3 B 是對的嗎	資料與算力的雙向約束	reverse_plan	17.6.3
多少錢	三種方案的三年期總持有成本	程式 17B + 第 21 章	17.6.2
值得嗎	與適配既有開放權重模型的比較	第 1.4 節的路線分析	第 22 章

第五個問題是這份預算書最關鍵、也最容易被略過的一項。第 1.4.1 節列過四條路線，而「從零預訓練」是成本最高的一條。一份誠實的預算書必須把「不做這件事、改成在開放權重模型上做適配」也算一次成本，並比較兩者。

很多情況下，這個比較的結論會是「適配比較划算」——而那正是本書第六篇的主線。能夠算出這個結論並誠實報告，比硬要證明從零預訓練可行更有專業價值。

交付要求

- 依 17.6.3 節的十項撰寫，每一項都要有數字與依據。
- 三種方案的比較表，含三年期總持有成本與各自的優缺點。
- 雙向約束分析：你被算力綁住還是被資料綁住？
- 敏感度分析：MFU 若比預期低 30%、資料若延遲三個月，時程與成本各變成多少？
- 對照方案：在開放權重模型上做適配的成本估算，與從零訓練的比較。
- 假設清單：至少十個假設，每個附不確定範圍。
- 一頁執行摘要：給不看細節的決策者，三段話說完該做什麼與為什麼。

評分重點

- 所有數字是否可追溯到本書的某個公式或你自己的實測。抄來的數字不予計分。
- 是否計算了餘裕，以及餘裕的比例是否有理由。
- 敏感度分析是否涵蓋最可能出錯的假設。
- 是否誠實比較了「不做」的方案。只論證自己想做的事者扣分。
- 執行摘要是否真的能被非技術決策者讀懂。

最後一項的訓練價值不亞於前面的計算。一份技術上完美但主管看不懂的預算書，不會拿到資源——而拿不到資源的計畫，再正確也不會發生。這是工程能力之外、同樣重要的一項能力。

章末評量

一、概念題

1. 什麼是不可約損失？它的存在對「該把模型放多大」有什麼意涵？
2. 為什麼給定算力預算時，模型大小與資料量存在最適配比？直觀上為什麼太大或太小都不好？
3. 為什麼推論主導的情境應該刻意過度訓練較小的模型？請說明其中的成本結構。
4. 在資料受限時有哪五種選項？各自的代價是什麼？
5. 為什麼說「在臺灣，模型大小往往是被資料決定的」？這與國際大廠的處境有何不同？
6. 外推有哪五種風險？為什麼「看殘差圖比看 R^2 重要」？
7. 尺度定律能預測什麼、不能預測什麼？為什麼不能用它承諾具體能力？
8. 預算書為什麼必須包含餘裕與假設清單？各自防止什麼問題？

二、計算題

1. 一張 GPU 峰值 30 TFLOPS、MFU 0.35，跑 7 天。計算總 FLOPs，以及依 20:1 配比的最適參數量與訓練詞元數。
2. 你有 8 B 詞元的繁中語料，去重後剩 62%，打算跑 3 輪（折算係數 1.0、0.9、0.8）。計算有效詞元數，以及由資料決定的最適模型大小。
3. 一個 3 B 參數的模型訓練 60 B 詞元。計算總 FLOPs。若用 4 張 GPU（各 30 TFLOPS、MFU 0.35），需要幾天？
4. 承上題，單卡功耗 350 W、PUE 1.4、電價每度 3 元。計算總電費。
5. 比較兩個方案：A 是 7 B 配 140 B 詞元，B 是 3 B 配 600 B 詞元。計算各自的訓練 FLOPs，以及服務量達到多少詞元時 B 的總成本較低。
6. 擬合得到 $E = 1.85$ 、 $A = 480$ 、 $\alpha = 0.34$ ，擬合範圍最大為 100 M。預測 $N = 800$ M 時的損失，並說明這個外推的倍數與可信度。

三、除錯題

1. 同學擬合尺度曲線得到 $R^2 = 0.998$ ，但外推到十倍規模時誤差達 0.3。請列出三個可能原因。
2. 同學的預算書顯示訓練只需 5 天，但實際跑了 12 天。請列出四個最可能的原因。
3. 同學按 20:1 訓練了一個 2 B 模型，但驗證損失明顯高於文獻中同規模的模型。請列出四個檢查方向。
4. 同學的資料受限實驗顯示「重複 10 輪後驗證損失仍在下降」。請說明這可能代表什麼，以及該

補做什麼檢查。

5. 同學的預算書把 MFU 假設為 0.5，理由是「規格書上寫的峰值」。請指出這個假設的問題。

四、系統設計題

1. 你的實驗室有一筆經費，可以選擇：買兩張高階 GPU、租三個月的雲端算力、或申請國網中心並把經費用在資料建置上。請設計一套決策流程，說明你需要哪些資訊、如何比較、以及在什麼條件下該選哪一個。
2. 你要為一個「臺灣製造業專用語言模型」規劃三年的技術路線。第一年資料有限、第二年可望取得產學合作資料、第三年可能有國家型計畫支持。請規劃各年的模型規模與訓練策略，並說明每一年的決策依據與檢核點。

五、臺灣情境與倫理題

1. 本章 17.4.3 節指出臺灣的模型規模往往被資料決定。請討論：這個限制是技術問題還是組織問題？有哪些可能的突破方向，各自需要誰來推動？
2. 訓練大模型的能源消耗是一個公共議題。請以本章的計算工具，估算一次 3 B 模型訓練的碳排放（可假設每度電的排放係數），並討論：研究團隊在什麼規模以上應該把環境成本納入決策？這應該是自律還是制度？

評量提示（給自學者）

- 計算題第 1 題： $C = 30e12 \times 0.35 \times 604800 \approx 6.35 \times 10^{18}$ ； $N^* = \sqrt{(C/120)} \approx 230 \text{ M}$ ； $D^* \approx 4.60 \text{ B}$ 。
- 計算題第 2 題： $8 \times 0.62 = 4.96 \text{ B}$ ；有效詞元 $= 4.96 \times (1+0.9+0.8) = 13.4 \text{ B}$ ； $N^* = 13.4/20 \approx 670 \text{ M}$ 。
- 計算題第 3 題： $\text{FLOPs} = 6 \times 3e9 \times 60e9 = 1.08 \times 10^{21}$ ；有效算力 $= 4 \times 30e12 \times 0.35 = 4.2 \times 10^{13}$ ；秒數 $\approx 2.57 \times 10^7$ ，約 297.6 天。這個數字說明 4 張卡遠不足以訓練 3 B 模型。
- 計算題第 4 題：GPU 小時 $= 297.6 \times 24 \times 4 \approx 28,571$ ；度數 $= 28,571 \times 0.35 \times 1.4 \approx 14,000$ ；電費約 42,000 元。
- 計算題第 5 題：A 為 5.88×10^{21} 、B 為 1.08×10^{22} ；損益兩平服務量約 6.15×10^{11} 詞元。
- 計算題第 6 題： $L = 1.85 + 480 \times (8 \times 10^8)^{-0.34} \approx 2.30$ ；外推倍數 $8\times$ ，在建議的十倍以內但已偏高，應報告範圍並先做中等規模驗證。

研究延伸與版本注意事項

- Kaplan 等人 (2020) 與 Hoffmann 等人 (2022, Chinchilla) 是尺度定律的兩篇奠基之作。前者提出冪次關係，後者修正了最適配比——兩篇的結論不同，這個修正過程本身就是很好的科學史教材。
- 推論成本主導下的過度訓練：近年多篇研究探討這個取舍，對 17.3 節有直接補充，也是多數開放權重模型採用高詞元比的原因。
- 資料受限下的尺度定律：關於重複訓練的邊際效益有明確的實證研究，17.4.2 節的數值量級來自這類工作。
- 尺度定律與下游能力的關係：這是一個活躍且尚無定論的方向，與第 1.2.5 節的湧現爭議相關。
- 本章的公式穩定，但係數（特別是 20:1 這個比例）在不同架構、資料與訓練方式下會變動。引用時務必註明來源與適用條件，並優先使用你自己擬合的結果。

常見問題

問：20:1 這個比例可以直接用嗎？答：作為規劃的起點可以，但你應該知道它的來源與限制（17.2.4 節）。如果你的專案有足夠的算力做小規模擬合，用自己的數字會準得多。如果沒有，用 20:1 並在預算中保留餘裕。

問：我的專題只跑 tiny 模型，需要這一章嗎？答：需要，而且很實用。第 13 章你可能是憑感覺決定「訓練 20,000 步」，本章讓你能算出「以我的算力，最適合的模型是多大、該餵多少資料」。這個計算只要幾分鐘，卻能讓你的實驗設計有依據。

問：如果算出來的結論是「我們的算力不足以訓練有用的模型」呢？答：那是一個非常有價值的結論，而且很可能是對的。正確的反應不是硬做，而是調整路線——第 1.4 節的四條路線中，「在開放權重模型上做適配」（第六篇）通常是資源受限時的正解。能算出這一點並及早轉向，比投入六個月做出一個不堪用的模型好得多。

問：預算書寫給誰看？答：至少三種讀者：技術審查者（看你的計算對不對）、經費決策者（看值不值得）、以及未來的你（看當初為什麼這樣決定）。第三種最常被忽略，但半年後你回頭看時會非常感謝當初寫下的假設清單。

教師使用建議

本章排在第 17 週，是第一學期的最後一章，建議 2 小時講授加 1 小時實驗。本章的教學重點是「把感覺變成數字」。

時段	內容	互動設計	注意事項
前 25 分鐘	17.1 至 17.2 節，尺度定律與最適配比	請學生先猜「一張 GPU 跑一天該訓練多大的模型」	多數人會高估好幾倍
中間 25 分鐘	17.3 至 17.4 節，過度訓練與資料受限	現場算 615 B 詞元的損益兩平點	過度訓練的結論很反直覺
後 25 分鐘	17.5 節，擬合與外推	展示殘差圖 vs R^2 的對比	殘差圖的診斷價值要示範
最後 40 分鐘	17.6 至 17.7 節與作業說明	各組現場算自己專題的預算	連結到貫穿式案例
實驗課 1 小時	程式 17A 與 17B	兩人一組互相審查對方的假設清單	假設的合理性是重點

一個效果極佳的開場：在講任何內容之前，請每個學生寫下「如果給你一張 GPU 跑一天，你會訓練多大的模型」。收集答案後公布 87 M 這個計算結果。多數人的答案會高出好幾倍——那個落差就是本章的價值。

另一個建議：讓學生把第 13 章實際跑過的 tiny 訓練代回 17.2 節的公式，檢查當初的設定是否接近最適。多數人會發現自己「模型太小、步數太少」或「資料重複太多輪」。這種回頭檢視自己作品的練習，比抽象的公式推導有效得多。

高中授課的調整：17.1 只講「損失有下限」與「三個變數互相牽制」的直覺；17.2 用「預算固定時買大房子還是買很多家具」的比喻，並讓學生實際算一次 87 M 那個數字；17.3 的過度訓練概念可用「買車 vs 加油」的比喻；17.4 至 17.5 略過；17.6 完整教（算錢對高中生很有感）；17.7 保留「這是規劃工具不是保證」一句話。程式做 17B 的填表版即可。

附：本章速查表

把本章的所有公式與經驗值整理成一張表，可以印出來貼在螢幕旁邊。

要算什麼	公式	典型輸入	節次
訓練 FLOPs	$6 \times N \times D$	參數量、訓練詞元數	第 3.2.5 節
推論 FLOPs	$2 \times N \times \text{服務詞元數}$	同上	17.3.1
最適參數量	$N^* = \sqrt{(C / 120)}$	算力預算 C	17.2.1

要算什麼	公式	典型輸入	節次
最適詞元數	$D^* = 20 \times N^*$	同上	17.2.1
由資料反推	$N^* = \text{有效詞元數} \div 20$	去重與折算後的量	17.4.3
有效算力	峰值 TFLOPS $\times$ MFU $\times$ 卡數	實測 MFU (非規格書)	第 15.6 節
牆鐘時間	FLOPs $\div$ 有效算力 $\div$ 3600	—	17.6.1
電費	GPU 小時 $\times$ 功耗 $\times$ PUE $\times$ 電價	實測功耗	17.6.1
過度訓練損益兩平	$(\text{訓練}_2 - \text{訓練}_1) \div (2 \times (N_1 - N_2))$	兩個方案的規格	17.3.2
重複折算	$D_{\text{eff}} = D \times \Sigma \text{各輪係數}$	1.0、0.9、0.8、0.7...	17.4.2
尺度擬合	$L = E + A \cdot N^{(-\alpha)}$	至少 4 個尺寸	17.5.2
外推上限	不超過擬合範圍的 10 倍	—	17.5.4

使用這張表時請記得三個前提：MFU 必須是實測的、功耗必須是實測的、而所有公式都應該加上 20% 到 30% 的餘裕。用規格書上的峰值算出來的預算，實務上幾乎一定不夠。

本章小結

1. 損失與參數量呈幂次關係 $L = E + A \cdot N^{(-\alpha)}$ ；E 是不可約損失，代表語言本身的不確定性。
2. 損失不會趨近於零；接近 E 時繼續加大模型的邊際效益極低，投資應轉向後訓練或檢索。
3. 給定算力預算 C，最適配比為 $N^* = \sqrt{(C/120)}$ 、 $D^* = 20 N^*$ ——一張 GPU 跑一天約對應 87 M 參數配 1.74 B 詞元。
4. 多數人的直覺會高估該訓練的模型大小；訓練詞元數只有參數量 1.25 倍的模型嚴重欠訓練。
5. 20:1 不是鐵律，會受資料品質、重複、tokenizer、架構與後訓練規劃影響。
6. 推論成本正比於參數量；服務量大時應刻意過度訓練較小的模型。
7. 7 B 配 140 B 與 3 B 配 600 B 的損益兩平點約在 6,150 億個服務詞元。
8. 臺灣的應用場景多為推論主導，因此策略應系統性偏向「小而充分訓練」。
9. 資料受限時，重複訓練的效益遞減：2 至 4 輪可接受，超過 15 輪接近無效且記憶化風險高。
10. 在臺灣，模型大小往往是被資料決定的，而非被算力決定——這與國際大廠的處境相反。
11. 外推應限制在擬合範圍的十倍以內，並一律報告不確定範圍；診斷應看殘差圖而非只看 R^2 。
12. FLOPs 到新臺幣的換算鏈有七步，關鍵輸入是實測的 MFU 與功耗，而非規格書上的峰值。

13. 預算書必須包含十項，其中餘裕（20% 至 30%）與假設清單最常被漏掉。
14. 尺度定律能預測預訓練損失與整體趨勢，但不能預測下游能力或架構改變後的行為。
15. 它是規劃工具而非保證；關鍵決策前應用中等規模實驗驗證外推是否成立。
16. 一份誠實的預算書必須把「不做」的方案也算一次——在開放權重模型上適配往往才是資源受限時的正解。

成果檢核

完成本章後你必須繳交：

- 尺度定律擬合器（程式 17A）：先在模擬資料上驗證，再用你自己的多尺寸實驗擬合，含殘差圖與外推警告。
- 算力預算計算器（程式 17B）：雙向計算、敏感度分析、以及 binding constraint 的判定。
- 資料受限實驗報告（程式 17C）：重複輪數對損失與記憶化的兩條曲線，以及你自己的折算係數。
- 臺灣本土 3 B 模型的算力預算書：依 17.6.3 節的十項撰寫，含三方案比較與執行摘要。
- 回顧分析：把你在第 13 或 15 章實際跑過的訓練代回本章公式，檢查當初的設定是否接近最適，並說明差距的原因。
- 一頁反思：算完之後，你對自己專題的規劃有什麼改變？

第 4 篇到此結束。這四章把「訓練一個模型」從一件憑感覺的事，變成一個有解碼策略、有穩定性保障、有實驗紀律、有資源規劃的工程。你現在能回答四個問題：****輸出怎麼調？訓練怎麼穩？結論可信嗎？資源夠不夠？****

從第 5 篇開始，我們會離開單機的世界，進入算力、系統與現代架構——而本章算出來的那些數字，將決定你在那裡面對的是什麼樣的問題。

索引

依詞條首字排序，數字為出現的章次。英文詞條排在中文詞條之前。索引依本冊內容編製；跨篇的完整索引見全書合訂本。

英文詞條

activation checkpointing 15
beam search 14
BF16 14、15、17
bootstrap 16
checkpoint 15、16
cherry-picking 16
cosine 15
DataLoader 15
decode 14
dropout 15、16
FLOPs 14、15、17
GELU 16
GPT 14
GradScaler 15
greedy 14
IsoFLOP 17
KV cache 14、17
loss scaling 15
LR range test 15
MFU 15、17
min-p 14
MoE 17
no-repeat n-gram 14
prefill 14
PUE 17
PyTorch 14、15
RAG 14、16、17
RMSNorm 15
RNG 狀態 15
RoPE 14
softmax 14、15
SwiGLU 16
tokenizer 16、17
tokens/sec 15
top-k 14
top-p 14
Transformer 15
warmup 15、17
WSD 15

中文詞條

不可約損失 17
反向傳播 15
尺度定律 15、16、17
文法約束 14
可重現 16
可重現套件 16
外推倍數 17
失敗復盤 16
推測解碼 14
敏感度分析 17
梯度 15、17
梯度累積 15
梯度裁切 15、17
梯度範數 15
混合精度 15
逐字複述 16

正規化	14、15	最適配比	17
交叉熵	15、17	嵌入	17
合規率	14	殘差圖	17
因果遮罩	14	結構化輸出	14
多重比較	16	評測	16、17
有效批次	15	詞元	14、15、16、17
有效候選數	14	開放權重	17
污染	16、17	微批次	15
自動回滾	15	微調	14、15、17
自變數	16	損益兩平	17
吞吐量瓶頸	15	溫度	14、15
困惑度	16	解碼參數表	14
批次大小	15、17	運算強度	14
受限生成	14	過度訓練	17
注意力	14、15	預先註冊	16
注意力熵	14	實驗可信度檢查表	16
信賴區間	16	截斷取樣	14
負面結果	16	算力	14、15、16、17
重複折算	17	算力預算書	17
重複懲罰	14	標準誤	16
個資	16	確認集	16
倖存者偏誤	16	餘裕	15、17
核取樣	14	冪次律	17
消融	16	學習曲線	16
草稿模型	14	學習率	15、16、17
訓練哨兵	15	學習率排程	15、17
記憶化	15、16、17	選擇偏誤	16
退化	14	應變數	16
假設	14、16、17	環境指紋	15、16
參數量	16、17	總持有成本	17
國網中心	15、17	繁體中文	14、16、17
探索集	16	續訓	15
控制變數	16、17		

大模型導論 Introduction to Large Language Models

第 4 篇 預訓練、解碼與實驗科學 (第 14–17 章)

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

課程與勘誤 : @prof (<https://page.line.me/prof>)

臺灣自編大學教科書系列 | 2026 年 8 月 第一版